中国科学院大学
University of Chinese Academy of Sciences

博士学位论文

高可靠高性能的Java虚拟机研究

作者姓名：敖　琪

指导教师：胡伟武　研究员

中国科学院计算技术研究所

学位类别：工学博士

学科专业：计算机系统结构

研 究 所：中国科学院计算技术研究所

二〇一五年十一月

Research on Java Virtual Machines of High Performance and High Reliability

by

Ao Qi

A Dissertation Submitted to
University of Chinese Academy of Sciences
in Partial Fulfillment of the Requirement
for the Degree of
Doctor of Philosophy

Institute of Computing Technology
Chinese Academy of Sciences

November, 2015

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书）

作者签名：　　导师签名：　　日期：

摘　要

随着集成电路制造工艺的持续发展，现代计算机系统的性能在大幅度提高的同时，其可靠性也正日益面临着软错误的严重威胁。软错误，指的是元器件吸收并积累外界能量例子的辐射，当达到一定程度时，有可能会造成器件状态的改变，通常表现为位翻转。软错误会改变程序的控制流和数据流，造成程序的错误或者异常，给计算机系统的可靠性和可用性带来极大的损害。对于那些用于极端环境的计算机系统，软错误会造成更大的危害。

当前针对软错误的容错技术大体可以分为两类：基于硬件实现和基于软件实现。基于硬件实现的容错技术错误覆盖率高、性能好，但是成本高。基于软件实现的容错技术，能够在不增加成本的情况下，具有较好的灵活性和错误覆盖率。但是，基于软件实现的容错技术面临的严峻挑战是往往会给计算机系统造成比较大的性能开销。

在Java虚拟机层面实现容错具有移植性好、透明等优势，但是同样面临性能的问题。本文在Java虚拟机层面进行了容错技术的研究，分别针对Java虚拟机的控制流和数据流提出了容错方法，并提高了Java虚拟机的访存和取址性能。本文的主要工作如下：

1. 提出并实现了一种针对Java虚拟机控制流的多位错误恢复方法。该方法是一种主动性的、预防性的错误恢复技术，通过对Java虚拟机分步骤地、并行地、并发地进行再生，达到消除Java虚拟机中软错误的目的。实验结果表明，该方法能够有效提升Java虚拟机的可靠性，同时对Java虚拟机产生的性能损失不到1%。
2. 提出并实现了一种针对Java虚拟机数据流的错误检测方法。该方法基于数据的时间信息和来源信息对数据进行有差别的校验，可以实现对Java虚拟机的堆空间和栈空间中的数据进行错误检测。实验结果表明，通过该方法以及相应的优化算法，比如短生命期数据优化、更新校验码优化、栈顶优化以及获取当前时间优化，Java虚拟机的可靠性和错误检测的效率得到了显著的提高。
3. 提出并实现了一种基于双基址寻址指令的快速访存方法和一种加载地址的优化方法。该方法从硬件和软件两方面优化MIPS 64位Java 虚拟机的访存和取址的过程，减少了生成代码的指令数，提高了程序性能。实验结果表明，该方法能够有效提升龙芯平台64位Java虚拟机的性能并降低Java虚拟机生成代码的体积。

上述部分成果在基于龙芯处理器的Java虚拟机上得到了实际应用。

关键词：Java虚拟机；软错误；瞬态故障；可靠性；容错

Research on Java Virtual Machines of High Performance and High Reliability

Ao Qi (Computer Architecture)
Directed by Hu Weiwu

In recent decades, computer designers employ advanced techniques such as shrinking feature sizes, reducing supply voltage, increasing clock rates to achieve better performance. However modern computer systems become increasingly vulnerable to soft errors. A soft error, appeared as bit flips, is a manifestation of a transient fault. It may change the data or control flow, making applications unpredictable and influencing the reliability and availability of a system. A soft error does more harm to computers or embedded devices used in extreme environments.

The current fault-tolerant technology for soft errors can be divided into two categories: hardware-based and software-based approaches. Hardware-based fault-tolerant techniques achieve high fault coverage and good performance with high cost. Software-based techniques improve system reliability without extra hardware and are gaining popularity. However, software fault-tolerance schemes typically incur significant performance degradation.

The JVM layer provides a great place to include fault-tolerance mechanisms. It works without modifying either applications running on it or an OS under it. However JVM-level approaches also incur high-performance degradation. It is necessary to achieve high-performance fault tolerance towards the reliability requirements and the hardware features. Therefore, this paper proposes software-based fault-tolerance approaches both for the control flow and data flow of JVMs, and improves the performance of memory access and load-address of JVMs. The main contributions of this paper are as follows:

1. A new approach is proposed to correct multibit errors in control flow of JVMs. The approach is a proactive and preventive error recovery mechanism. By performing JVM rejuvenation separately, parallelly and concurrently, the approach eliminates soft errors in JVMs. Experimental results show that this approach effectively improves the reliability of JVMs, and the performance degradation incurred by the approach is less than 1%.

2. A new approach called CST is proposed to detect data-flow errors in JVMs. The approach uses time information and source information of data in JVMs to check data differentially. Several optimization schemes are also proposed to reduce the performance overhead incurred by CST, including short lived variable optimization, top of stack optimization, current time optimization and updating checksum optimization. Experimental results show that the reliability of JVMs and the efficiency of error detection have been improved.
3. An approach based on register + register addressing to access memory and an approach to load 64-bit address are proposed. The memory access and address computation of 64-bit JVMs are respectively optimized by hardware and software schemes. Experimental results show that the number of instructions generated by JIT compiler is reduced and the performance is improved.

Some of the proposed methods have been applied in the JVM based on Godson processors.

Keywords: Java Virtual Machine; JVM; Soft Error; Transient Fault; Reliability; Fault Tolerance

目　录

摘　要 I

目　录 V

图目录 IX

表目录 XI

第一章　绪论 1

1.1　概述 1

1.2　Java虚拟机的基本概念和关键技术 3

1.2.1　Class文件格式 4

1.2.2　类装载子系统 5

1.2.3　执行引擎 5

1.2.4　运行时数据区 6

1.2.5　自动内存管理 8

1.2.6　Java虚拟机面临的主要问题 9

1.3　本文的研究内容及主要贡献 9

1.4　论文的组织 10

第二章　面向软错误的容错技术的研究现状 13

2.1　软错误基本概念 13

2.2　容错技术的基本方法 16

2.2.1　Lockstep技术 17

2.2.2　冗余多线程技术 17

2.2.3　信息冗余技术 19

2.2.4　前向错误恢复 24

2.2.5　后向错误恢复 27

2.3　硬件实现 29

2.4　软件实现 29

2.5　本章小结 31

第三章 Java虚拟机研究平台 33
3.1 引言 33
3.2 有代表性的Java虚拟机项目 33
3.2.1 HotSpot VM 33
3.2.2 JRockit 35
3.2.3 J9 VM 36
3.2.4 Zing VM 37
3.2.5 研究型Java虚拟机 38
3.2.6 Java processor 38
3.3 龙芯高性能处理器 39
3.3.1 GS464处理器核 39
3.3.2 GS464E处理器核 39
3.3.3 四核龙芯3号处理器 40
3.4 龙芯Java虚拟机 40
3.5 本章小结 41
第四章 针对控制流的Java虚拟机容错 43
4.1 引言 43
4.2 相关工作 44
4.2.1 针对控制流的硬件容错技术 44
4.2.2 针对控制流的软件容错技术 44
4.3 软件再生技术 47
4.4 Java虚拟机快速再生技术 50
4.4.1 快速再生Java虚拟机 50
4.4.2 Java虚拟机热再生 51
4.5 实验结果和分析 53
4.5.1 错误注入模块 53
4.5.2 错误覆盖率 53
4.5.3 性能损失 56
4.6 本章小结 56

第五章　针对数据流的Java虚拟机容错 59
5.1　引言 59
5.2　相关工作 59
5.2.1　针对数据流的容错技术 59
5.2.2　动态污点分析技术 60
5.3　方法概述 61
5.4　实现与优化 63
5.4.1　标签存储 63
5.4.2　校验码 65
5.4.3　时间标签 65
5.4.4　来源标签 65
5.4.5　本地方法 66
5.4.6　需要修改的字节码 66
5.4.7　优化 66
5.5　实验结果和分析 68
5.5.1　性能损失 68
5.5.2　内存消耗 70
5.5.3　错误覆盖率 71
5.6　本章小结 73
第六章　Java虚拟机访存与加载地址优化 75
6.1　引言 75
6.2　相关工作 76
6.3　加载地址行为分析 77
6.3.1　加载地址的分类 77
6.3.2　地址分布 78
6.4　快速内存访问 82
6.4.1　概述 82
6.4.2　实现细节 83
6.5　加载地址优化 84
6.5.1　LPC优化 84

6.5.2 GLA优化 84
6.6 实验评测 85
6.6.1 实验平台介绍 86
6.6.2 性能测试 86
6.6.3 内存消耗测试 88
6.7 本章小结 89
第七章 总结及未来工作 91
7.1 本文工作总结 91
7.2 下一步研究方向 92
参考文献 93
致 谢 i
作者简介 iii

图目录

图 1.1 软件实现容错的层次图 3
图 1.2 Java虚拟机内部体系结构 4
图 1.3 由所有线程共享的运行时数据区 7
图 1.4 线程专有的运行时数据区 8
图 2.1 错误分类图[1] 14
图 2.2 容错系统的作用 14
图 2.3 SoR举例（包含处理器） 15
图 2.4 SoR举例（包含处理器和内存） 16
图 2.5 利用编码技术实现容错的基本原理 19
图 2.6 水平垂直奇偶校验例子 21
图 2.7 DMR工作流程图 25
图 2.8 TMR工作流程图 26
图 2.9 二乘二取二系统结构图 27
图 2.10 恢复块的工作流程 28
图 3.1 HotSpot结构图[2] 34
图 3.2 HotSpot client编译器的编译过程 34
图 3.3 J9 VM 36
图 3.4 三大主流Java虚拟机堆空间布局对比 37
图 4.1 看门狗和TMR方式 45
图 4.2 EDDI指令对比 46
图 4.3 未使用软件再生技术的概率状态转换模型 48
图 4.4 使用软件再生技术的概率状态转换模型 48
图 4.5 快速再生Java虚拟机内部结构图 51
图 4.6 SPECjvm2008错误覆盖率执行结果 55

图 4.7 SPECjvm2008性能损失 56
图 5.1 标签结构 63
图 5.2 执行iadd前后Java栈顶的情况。O表示操作数，ST表示源标签，TT表示时间 64
图 5.3 性能测试 69
图 5.4 性能测试：阈值 70
图 5.5 对象累积尺寸 71
图 5.6 最小栈需要 72
图 6.1 不同测试程序的地址分布 80
图 6.2 不对堆大小的地址分布 81
图 6.3 访存和加载程序地址的算法示意图 82
图 6.4 优化前后的访存指令序列 84
图 6.5 优化前后加载地址指令序列 85
图 6.6 SPECjvm2008性能评测 87
图 6.7 Dacapo性能评测 88

表目录

表 2.1 数据$D_1D_2D_3D_4$所对应的汉明(7, 4)码的校验矩阵 22
表 2.2 使用不同容错技术的硬件实现方案及其各自特点 29
表 4.1 模拟结果 54
表 4.2 再生模拟结果 55
表 5.1 Java字节码对应的标签传播规则 67
表 5.2 测试程序列表 69
表 5.3 不同阈值对CST方法效果的影响 73
表 6.1 la指令数和比例 78
表 6.2 la指令分类 79
表 6.3 优化前后la指令数和比例 88

第一章　绪论

随着集成电路制造工艺的持续发展，现代计算机系统的处理器性能和存储性能均得到了大幅度提高。但与此同时，现代计算机系统的可靠性也正面临着瞬态故障的威胁，变得越来越容易受到软错误的影响。一些应用于特殊环境下的嵌入式系统，比如应用于航天中的计算机系统，更是面临着宇宙射线等极端条件所导致的瞬态故障的挑战。因此，针对瞬态故障的容错技术对于当代计算机系统越来越重要。在硬件和软件层面都有许多容错技术提出。在Java 虚拟机层面进行容错由于其移植性好等特点具有很大的优势，但是以往的在虚拟机层面进行的容错技术往往会给计算机系统带来比较大的性能损害。本文通过对Java 虚拟机进行研究和改进，结合软件再生、动态污点分析等技术，实现了从软件层面提高计算机系统可靠性的同时保证了系统的性能。同时，高可靠的、高性能的Java虚拟机技术对于以龙芯为代表的国产处理器破除x86体系壁垒、丰富软件环境、提高国产计算机系统可靠性具有重要的现实意义。

1.1　概述

在过去的几十年中，计算机系统中单位面积使用的晶体管数量成指数级增长，这使得半导体器件，尤其是微处理器的速度得到大幅度提升。1965 年，英特尔创始人之一摩尔预测，集成电路上可容纳的晶体管数目约每隔24个月便会增加一倍，这就是著名的“摩尔定律”。1971年，英特尔制造出第一个处理器，内有2300个晶体管。至2014年，英特尔发布的Haswell-EP处理器，内有高达55.6 亿个晶体管。

随着工艺的发展，计算机系统的速度越来越快，与此同时，计算机也变得越来越容易出错。为了实现更高的性能和更低的功耗，计算机设计者们采取了各种先进的技术来减小体积、降低电压、增加时钟频率和内存容量[3]。但是，这也使得当今计算机系统变得更容易受到软错误的影响。另一方面，计算机系统被用于各种极端复杂的环境中，比如航空航天、核反应堆等[4,5]，这使得计算机系统的可靠性变得尤为重要。随着工艺的进一步发展，以及计算机系统应用的环境更加复杂恶劣，接下来的计算机系统可能要面临“软错误墙”的现象。

半导体电路通过保持或者释放一定量的电荷来表示不同的逻辑状态，这些器件会受到来自于能量例子的辐射，比如α粒子和中子、电压扰动、电磁干扰等。计算机系统的元器件吸收并积累这些能量粒子的辐射，当达到一定程度时，有可能会造成器件状态的改变。这种间歇性的瞬态故障通常体现为从逻辑0变成逻辑1，或者反之，这也被称为软错误。区别于会对硬件产生永久损害的永久故障，瞬态故障是一种只对逻辑电路或内存产生临时效果，并且可以通过重新修改相应存储单元中的值就可恢复的故

障。

软错误是瞬态故障的一种表现形式，通常会表现为位翻转，这会改变程序的数据或者控制流，使得程序变得不可预计，会给计算机系统的可靠性带来极大的损害。计算机系统在运行过程中出现软错误，可能会造成系统崩溃、无响应或者产生错误的结果。应用于极端环境的计算机系统会更容易受到软错误的影响。由于这类系统往往对于可靠性要求更高，比如航天系统，针对软错误的错误检测与错误恢复技术对这类系统更为重要。

针对软错误的容错技术可以通过硬件和软件两个层面实现。在硬件层面，计算机系统通常采用硬件上的冗余或者使用一些专有的硬件模块来实现可容错，比如ECC（Error Correcting Code）或者看门狗（Watchdog）。通过硬件的方式能够提供充足的错误覆盖率以及良好的性能。但是采用硬件方案往往意味着更大的功耗、更大的体积和更高的成本。另外，对于那些已经布置好的系统，采用硬件方案意味着更换设备或者部件，有的时候这是不可行的。因此，很多计算机系统，尤其是嵌入式系统是无法采用硬件方式的。

在软件层面实现容错技术是一种常用的方式，它可以在不增加硬件代价的前提下实现较高的可靠性。基于软件的容错技术可以用于系统的各个层面，比如应用程序、虚拟机以及操作系统。可由软件方法实现容错的的层次在图1.1 中给出。在应用层面容错往往在实现上比较简单，但是有的时候并不能获取应用程序的源代码，而且要修改系统中运行的所有应用程序通常也不现实。在操作系统层面容错可以实现更好的错误覆盖率，但是需要修改操作系统，这对于一些成熟的操作系统来讲会很困难，比如Windows操作系统。在虚拟机层面进行容错，可以做到对下不修改操作系统、对上不修改应用程序，具有很大的优势。但是，不管在软件的哪个层面进行容错都会对性能产生比较大的损害[6]。同错误检测技术相比，通过软件层面实现错误恢复会给系统带来更大的性能损害。

Java虚拟机提供了一个良好的实现容错技术的平台，在Java虚拟机层面进行容错有几个好处。第一，一旦基于某个体系结构的Java虚拟机实现了容错，那么等于对所有可以运行该Java虚拟机的相同指令集的计算机系统均实现了容错，甚至包括那些还没有开发出来的使用相同指令集的计算机系统。第二，从Java 虚拟机层面实现容错，可以避免修改操作系统。修改操作系统往往是一个难度极高而且繁重的任务。第三，从Java虚拟机层面实现容错，解放了应用程序的开发者，使得应用程序开发者不用考虑容错的任务。第四，所有已经开发的Java程序均可以直接运行在实现容错的Java 虚拟机之上，不需要源代码、不需要修改程序、不需要重新编译。最后，目前Java语言依然是最流行的高级语言之一，尤其在一些嵌入式平台，Java程序被广泛使用。

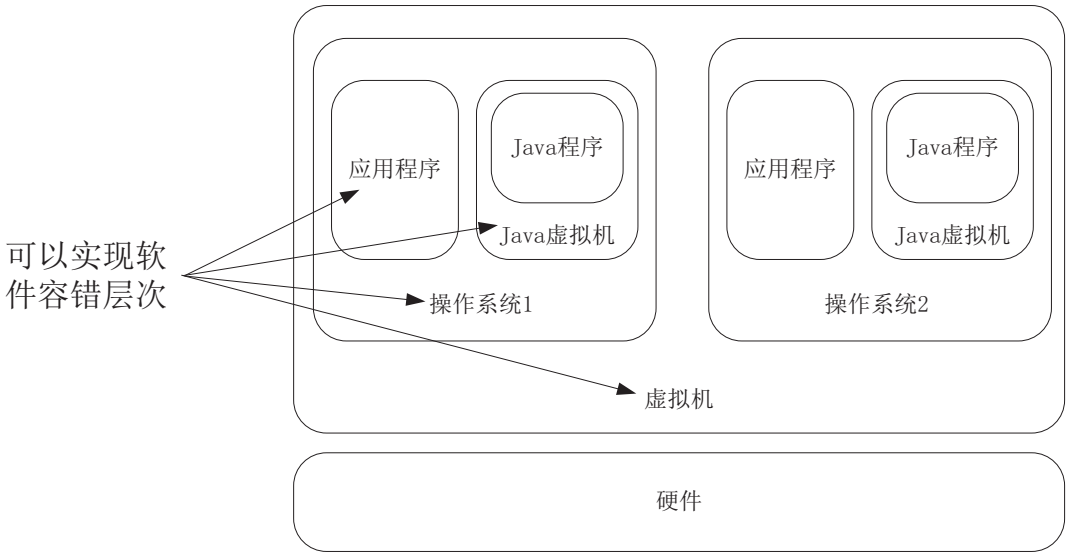


图 1.1 软件实现容错的层次图

1.2 Java虚拟机的基本概念和关键技术

Java技术体系包括以下几个组成部分：

- Java程序设计语言
- 各个硬件平台上的Java虚拟机
- Class文件格式
- Java API类库
- 来自商业机构和开源社区的第三方Java类库

Java虚拟机作为Java技术体系的重要组成部分，是Java程序设计语言的基石。正是通过Java虚拟机，才使得Java语言能够实现硬件无关和操作系统无关。Java虚拟机是一个抽象的计算机，与真实的计算机一样，它具有一套指令集并使用一系列存储区域。Java虚拟机的指令集就是Java字节码（Bytecodes），Java虚拟机的存储区域是一种基于堆和栈的结构。一个典型的Java虚拟机的结构如图1.2所示。

图1.2是Java虚拟机的结构框图，包括在Java虚拟机规范[7]中描述的主要子系统和内存区。类装载器子系统根据给定的全限定名（例如java/lang/Object.class）来装入类型（类或接口）。执行引擎负责执行那些包含在被装载类的方法中的指令。当Java虚拟机运行Java程序时，它需要存储许多数据，例如字节码、局部变量、中间结果、对象实例、传递给方法的参数以及返回值等，这些数据都被Java虚拟机组织在运行时数据区中管理。

Java虚拟机规范对于其各个子系统的描述是抽象的，类装载系统、执行引擎、运行时数据区等结构的实现细节以及采用的技术大都由设计者根据具体的需求决定。

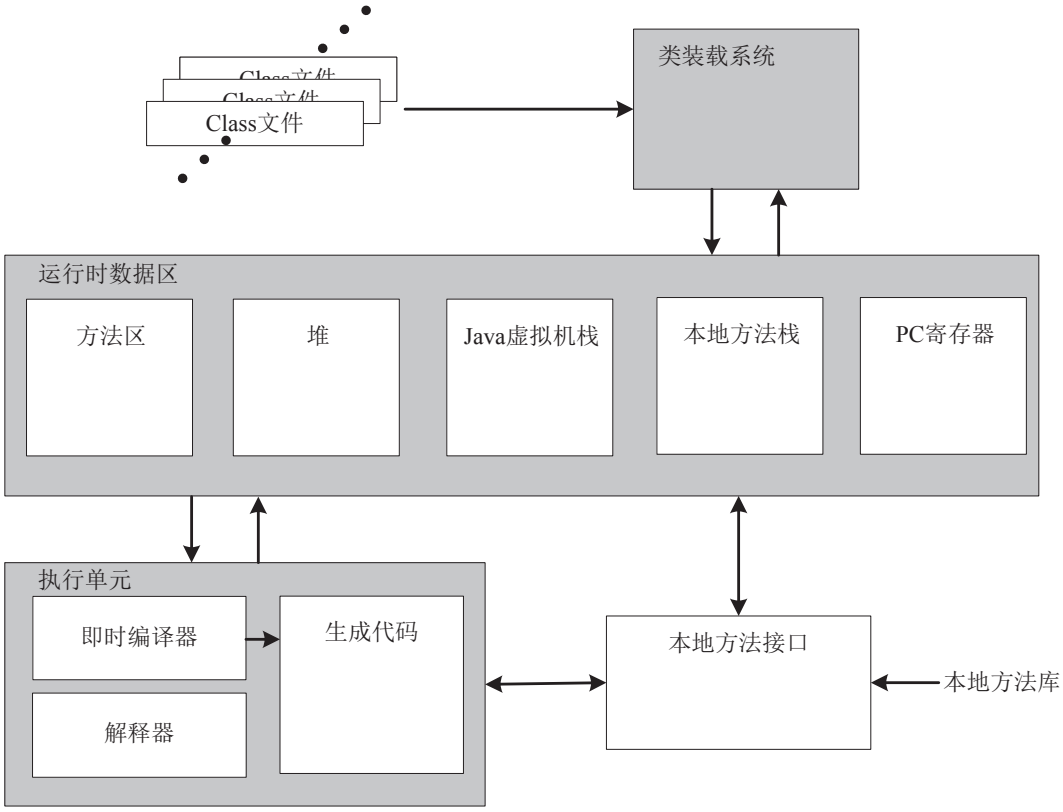


图 1.2　Java虚拟机内部体系结构

1.2.1　Class文件格式

由Java语言编写的程序经过编译，生成一种可以由Java虚拟机直接执行、硬件无关、操作系统无关的二进制格式，这种二进制格式通常以文件的形式存储，即Class文件。每个Class文件对应且只对应一个Java类或者一个Java接口，并对该类或者接口进行了全面的描述。

一个Class文件包含以下信息：魔数（Magic）、版本信息、常量池、访问标志（Access Flags）、类索引、父类索引、接口索引、字段表、方法信表以及属性表。

魔数的作用是表示该文件为Class文件，其值为0xCAFEBABE。版本信息由次版本号（Minor Version）和主版本号（Mager Version）构成，主版本号M和次版本号m共同决定了Class文件的版本M.m。常量池是一个长度可变的有序序列，它是Java程序用到的资源的集合，是占据Class文件中数据最多的一项之一。常量池中存储的常量包括：字符串常量、类和接口的名称、字段名称以及一些其他数据。访问控制用于识别该类或者该接口的访问信息，比如，常用的类访问的属性有：public类型、final类型、interface类型、abstract类型等。类索引和父类索引均是两个字节的常量池索引，而接口索引是由两个字节的常量池索引构成的数组，表示该类实现了那些接口。索引指向常量池中的包含相应类或者接口全限定名的数据，Class文件使用这三个信息确定类的继承关系。字段表用于描述类中或者接口中声明的变量，每个字段对应了字段的信息，

包括字段的名字、描述符和修饰符。方法表同字段表结构类似，描述了类中或者接口中声明的方法。如果一个方法既不是抽象方法也不是本地方法，那么方法表中会存储该方法的局部变量所需的栈空间长度、为方法所捕获的异常表、字节码序列以及局部变量表。Class文件最后部分是属性集合，它给出该类或者接口所定义的属性的基本信息。

Class文件并不一定必须与Java语言相关，其他语言也可经过编译生成Class文件格式，比如Jython、JRuby、JScheme、Groovy、Kawa、Scala、Clojure等语言均可运行于Java虚拟机之上。因此，Java虚拟机不仅是一种平台（操作系统和硬件）无关的虚拟机，还可以实现一种语言无关的虚拟机。

1.2.2 类装载子系统

类装载子系统负责查找并装载类型信息。它把描述类的数据从Class文件装载至内存，然后对其进行验证、准备、解析和初始化，最终形成可以被Java虚拟机直接使用的Java类型。

一个类在被使用之前需要经历装载、链接（包括验证、准备和解析）以及初始化三个过程。装载过程负责查找并将类型的二进制数据放入内存；验证过程确保被载入的类型的正确性；准备过程为类变量分配内存，并将其初始化为默认值；解析过程负责把类型中的符号引用转换为直接引用；初始化则指的是把类变量初始化为正确的值。

1.2.3 执行引擎

执行引擎是Java虚拟机核心的组成部分，Java虚拟机规范只是制定了字节码执行引擎的概念模型，在Java虚拟机实现层面，执行引擎一般会有解释执行和编译执行两种方式。解释执行是通过解释器（Interpreter）逐条解释执行字节码，编译执行则是通过即时编译器（Just-In-Time Compiler，简称JIT compiler）将多条字节码翻译成本地代码执行。

解释器能够把高级编程语言一行一行直接转译执行。解释器不会一次把整个程序编译出来，而是每次执行程序时都要先转成另一种语言再作执行，因此解释器的程序运行速度比较缓慢。它每解释一行程序就立刻执行，然后再解释下一行，再执行，如此不停地进行下去。Java解释器将Java字节码转换成当前平台可以识别的机器代码。Java解释器可以运行所有Java程序，能够实现一个完整的符合标准的Java虚拟机。解释器的优势是省去了编译整个程序的负担，从遇到第一条Java字节码起就可以开始执行Java程序，具有较快的启动速度。但解释器也会降低程序执行的效率，执行速度较慢。这种执行方式结构简单，移植起来也比较容易。一般来讲，开发或者移植一个Java虚拟机，都是以先完成其解释器作为基础。

即时编译器，动态编译的一种形式，是一种提高程序运行时性能的方法。相对于即时编译方式，计算机程序有两种其他的运行方式，一个是解释执行和一个是静态编译。静态编译是在程序执行前，提前将代码编译成本地机器码，并且只需要编译一次即可。即时编译则融合了解释执行和静态编译这两种方式，在程序的运行过程中，将代码编译成本地机器码，并将编译生成的机器码缓存起来以降低性能损耗。在Java虚拟机中，即时编译器通常是以Java程序的方法和循环体为单位，在Java方法被调用或者循环体执行时将其编译为本地代码。以后每次执行该方法或者循环体时都是运行编译后的本地方法。不同于静态编译，即时编译需要在生成代码的质量和编译代码的时间之间作出平衡，无论偏向哪一者都可能造成虚拟机性能的损害。

程序局部性原理认为决定程序运行性能的只是小部分代码，即时编译器对所有Java方法一视同仁地编译是不恰当的。一些Java方法被执行的次数很少，甚至只有一次（如类初始化方法），编译这些方法得不偿失；不同方法执行次数有很大差别，对它们采用同样的编译力度也是不恰当的。因此，出现了解释器和即时编译混合执行以及自适应优化即时编译等技术。混合执行方式根据一定的启发因素（如方法已被调用次数和循环体执行次数等）决定是否编译相应的字节码；自适应优化编译则根据运行时信息决定是否采用更大力度的优化措施重新编译一个方法。

1.2.4 运行时数据区

Java虚拟机会定义一些运行时数据区域。有些运行时数据区是由程序中所有线程共享的，这些区域一般在Java虚拟机创建时创建，在Java虚拟机退出时才被回收。还有一些区域则由线程独有，这些数据区随着每个Java线程的创建而创建，随着每个Java线程的退出而回收。

每个Java虚拟机实例都有一个方法区以及一个堆，它们是由该虚拟机实例中的所有线程共享的。

- **方法区（Method Area）**：当虚拟机装载一个Class文件时，它会从这个Class文件包含的二进制数据中解析类型信息并将相应的信息存储到方法区中。这些信息以Class为单位进行组织和存储。方法区存储的信息包括常量池（Constant Pool）、类信息、静态变量、即时编译器编译后的代码以及类和接口的构造方法等。
- **堆（Heap）**：当程序运行时，虚拟机会把所有Java程序在运行时创建的对象实例和数组都放到堆中。堆中的数据是所有线程共享的。堆中的数据无需显式的回收，不再被使用的对象实例或者数组会被垃圾收集器自动回收。堆的大小可以是固定的也可以是按需动态扩展的。如果程序所需要的堆的大小超过了虚拟机设置的限制，那么Java虚拟机抛出一个OutofMemoryError异常。

图1.3描述了这些线程间共享的内存区域。

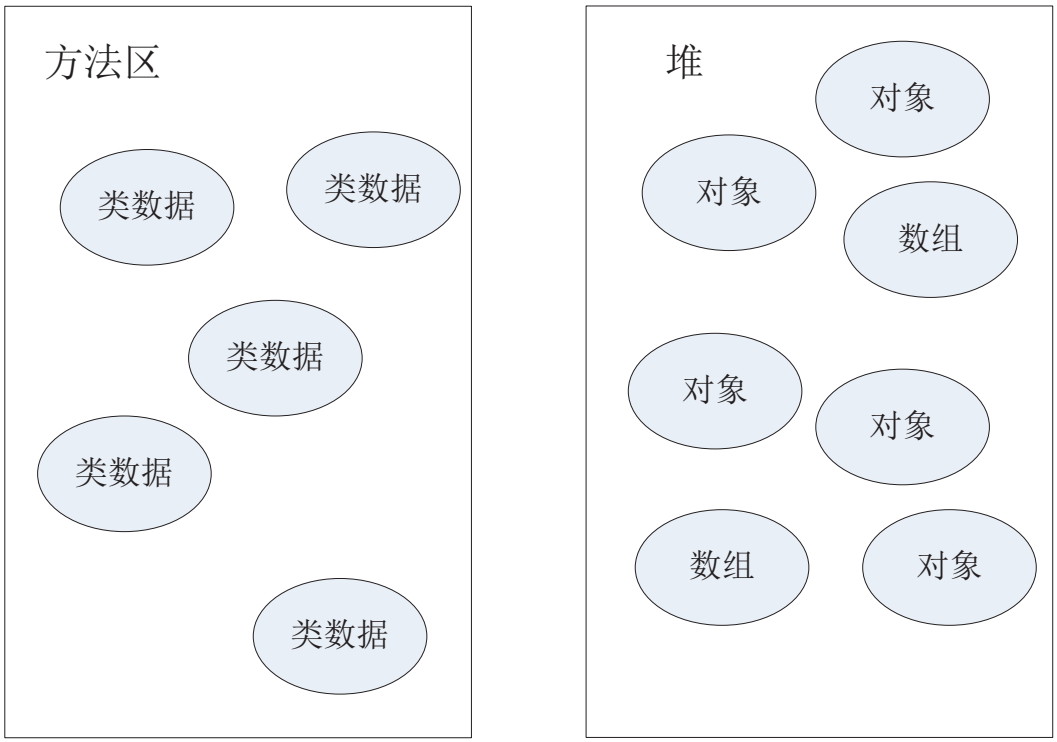


图 1.3 由所有线程共享的运行时数据区

当每一个新线程被创建时，该线程都将得到它自己的PC寄存器以及一个Java虚拟机栈。这些数据区域是每个线程独享的。

- **PC寄存器**：Java虚拟机支持多个线程同时执行。每个线程都有它自己的PC寄存器。如果线程正在执行的是一个Java方法（非本地方法），那么PC寄存器的值将总是指示下一条将被执行的指令。
- **Java虚拟机栈（Java Virtual Machine Stack）**：每个Java线程拥有一个私有的Java虚拟机栈。当前正在执行的线程对应当前栈。Java虚拟机栈类似于C语言中栈的概念，存放Java线程需要的局部变量和中间结果，并在方法调用的时候传递参数和返回值。Java虚拟机栈是由许多栈帧（Stack Frame）或者称帧（Frame）组成的，一个栈帧包含一个Java方法调用的状态。当线程调用一个Java方法时，虚拟机压入一个新的栈帧到该线程的Java虚拟机栈中；当该方法返回时，这个栈帧被从Java虚拟机栈中弹出并抛弃。Java虚拟机没有寄存器，其指令集使用Java虚拟机栈来存储中间数据。这样设计的原因是为了保持Java虚拟机的指令集尽量紧凑，同时也便于Java虚拟机在通用寄存器很少的平台上实现。如果在Java程序执行的过程中某个线程需要使用的栈的大小超过了设置，则会抛出StackOverflowError。在第一版Java 虚拟机语言规范中，Java虚拟机栈被称为Java栈（Java Stacks）。本文不严格区分Java虚拟机栈与Java栈。
- **本地方法栈（Native Method Stack）**：Java语言支持调用本地方法，也就是C函

数。在调用本地方法时，需要将当前栈由Java 虚拟机栈的结构转化为C栈的结构。这部分的实现是平台相关的，方法的参数是以某种依赖于具体实现的方式存储在本地方法栈中，也可能是在寄存器或者其他某些与特定实现相关的内存区中。

图1.4举例说明了这些线程专有的内存区域，它描绘了Java虚拟机为每一个线程创建的内存区，这些内存区域是私有的，任何线程都不能访问另一个线程的PC寄存器或者栈。图1.4展示了一个虚拟机实例的快照，它有三个线程正在执行。线程1和线程2都在执行Java方法，而线程3则正在执行一个本地方法。在此图中，Java栈是向下生长的，而栈顶都显示在图的底部。当前正在执行的方法的栈帧以浅色表示。对于一个正在运行Java方法的线程而言，它的PC寄存器总是指向下一条将被执行的指令。在图1.4中，线程3的PC寄存器——以深色显示的那个，其值是不确定的。

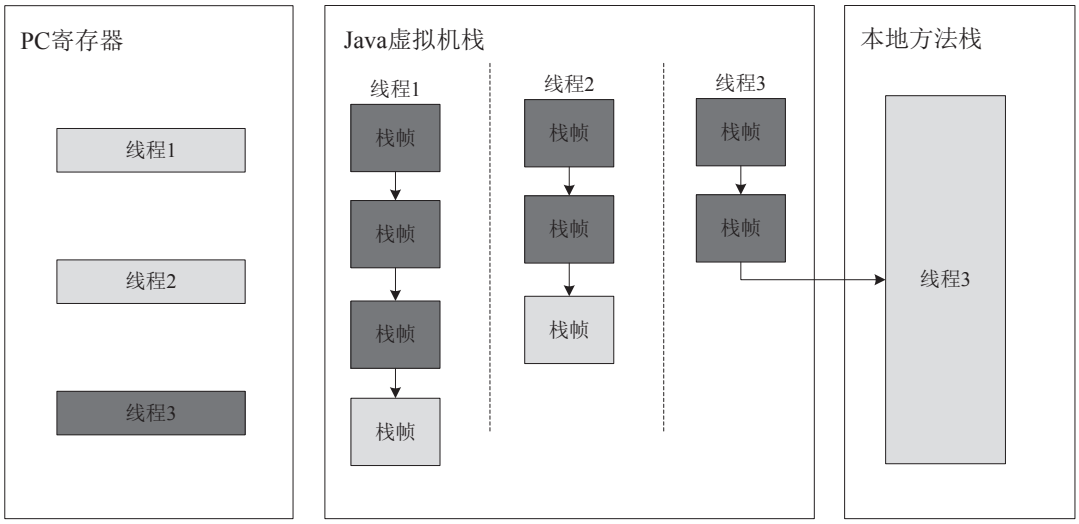


图 1.4　线程专有的运行时数据区

1.2.5　自动内存管理

自动内存管理也称为垃圾收集（Garbage Collection，简称GC），其作用是自动回收无用单元的内存空间。释放内存的工作由 Java 虚拟机自动管理，减轻了应用程序员的负担，避免了Java应用程序的内存泄漏。垃圾收集算法的主要评价标准为吞吐量（Throughput）和停顿时间（Pause Time）。吞吐量指在程序运行中，非垃圾收集的时间与整个应用程序运行时间的比值，比值越高代表垃圾收集算法的整体效率越高。停顿时间主要指垃圾收集造成的应用程序的停顿时间，这可以从多个方面衡量，如每次垃圾收集的停顿时间、总的停顿时间等。如果每次垃圾收集的停顿时间恒定且较小，则该算法的可预测性较好，实时性较高。

Java虚拟机规范并没有规定如何实现自动内存管理，甚至并没有规定必须要实现自动内存管理。自动内存管理的实现可以根据实现者的具体需求进行选择，甚至在一些

精简的嵌入式系统中并没有实现自动内存管理[8,9]。

1.2.6 Java虚拟机面临的主要问题

Java程序语言给广大程序员带来了很多好处，比如一次编译到处执行、自动内存管理以及相对安全。Java虚拟机在实现这些特点的同时，是以一定程度上牺牲了程序的执行效率为代价。Java虚拟机需要在执行过程中动态地编译字节码，甚至解释执行字节码；Java虚拟机需要额外的时间而且需要停止整个Java 程序的执行用来回收不再使用的堆空间；在加载Class文件的过程中，Java虚拟机需要对Class文件进行加载验证、解析、准备、初始化等一系列动作，这些都给Java程序的执行带来了额外的时间开销。目前，通过多种优化手段，许多Java虚拟机[10–15]已大幅提升了性能，但是同C 和C++等语言相比，Java程序的执行效率仍有增长的空间。

另外，在软错误存在的前提下，如何实现一个高可靠、高性能的Java虚拟机成为挑战。Java程序设计语言以及Java虚拟机的设计目标是移植性和安全性。为此，Java提供了很多特性，比如类型检查、RMI（Remote Method Invocation）、沙箱等。但是，Java规范并没有对容错提供直接的支持。近年来，学者们提出了一些提升Java虚拟机或基于Java系统的可靠性的技术[16–22]。但是这些技术往往对Java虚拟机产生较为严重的性能损害，或是只能应用于某些特定的计算机系统具有很大的局限性。

综上所述，提升Java程序的性能一直以来都是研究的热点。面临软错误愈发严重的新形势，在提高可靠性的同时兼顾Java虚拟机的性能成为Java虚拟机技术发展的一个新难题。

1.3 本文的研究内容及主要贡献

随着以龙芯处理器为代表的国产高性能处理器的日益发展，我国的处理器设计水平也跻身于世界先进水平。龙芯处理器采用简洁高效的 MIPS 兼容指令系统，具有处理器设计高效、低功耗等诸多优点。

然而，不可否认的是，目前服务器和桌面领域的主流指令集架构采用x86及其衍生架构，而移动领域和嵌入式领域则由ARM占据主流地位。商用高性能处理器市场长期被Intel等厂商垄断，导致大量优秀的软件无法在龙芯处理器上运行。随着龙芯等国产处理器产业化大踏步发展，软件库不够丰富成为限制其推广的一个瓶颈。而Java程序正是以跨平台著称，同时市场上存在大量以Java编写的优秀软件。因此，实现一个高性能的Java虚拟机有助于龙芯处理器的产业化发展，提高龙芯处理器的竞争力。

本文旨在研究面向软错误的Java虚拟机容错技术，重点研究如何在保证程序性能的前提下提高Java虚拟机的可靠性。主要的贡献包括：

1. 提出一种基于Java虚拟机控制流的容错方法。本文将软件再生模型与软错误容错相结合，提出了一种新颖的、预防性的、主动性的Java虚拟机热再生方法。通过

该方法可以对Java虚拟机的方法区实现容错。该方法有别于传统的利用冗余执行对错误先检测后恢复的思想，而是不进行错误检测，直接主动性地修复Java虚拟机中可能存在的错误。该方法可以并行执行也可以和Java程序并发执行，因此具有性能损耗小和低延迟的特点。该方法可以和其他容错技术配合使用，作为其他容错的预防和补充。

2. 提出一种基于Java虚拟机数据流的容错方法。该方法结合污点分析技术，对Java虚拟机的堆空间和栈空间中的数据实现容错。该方法通过对数据的时间信息和来源信息进行动态追踪，对数据进行有差别的检查，消除了一些不必要的校验。针对该方法，本文还提出了一系列优化算法，包括短生命期数据优化、更新校验码优化、栈顶优化以及获取当前时间优化，这些优化能够大幅提升错误检测的性能。
3. 提出了一种基于龙芯处理器的64位Java虚拟机访存与取址优化方法。该方法分别从硬件和软件两方面优化龙芯64位Java虚拟机的访存和取址过程，减少了生成代码的指令数，提高了程序性能。该方法通过在硬件上实现双基址寄存器访存指令，简化了Java虚拟机方法区的访存过程；通过优化地址计算，简化了程序地址和数据地址的加载过程。最终的评测结果显示，该方法可以使SPECjvm2008整体性能提升5.1%，使Dacapo 整体提升7.3%，个别用例性能提升近25%。在内存消耗方面，本方案可以使Java虚拟机编译器生成代码的体积减少16%。

1.4 论文的组织

本文共分七章，组织如下：

第一章，从软错误对计算机系统的危害出发，分析了在各个层面进行容错的特点尤其是在Java虚拟机层面进行容错的优势。着重介绍了Java虚拟机的基本概念和关键技术。最后指出从Java虚拟机层面进行容错，不论是从技术特点出发还是从当今计算机系统尤其是龙芯处理器的实际需求出发，均具有重要的意义。

第二章，首先介绍了软错误的一些基本概念。基于这些基本概念，阐述了错误检测技术和错误恢复技术的方法和研究现状。最后，结合这些容错方法，本章列举并介绍了一些具体实现，这些实现包括硬件层面、应用程序层面、编译器层面、软硬结合方式以及Java虚拟机层面。

第三章，介绍了本文所使用的实验平台。本章首先介绍了一些有代表性的Java 虚拟机，其次介绍的龙芯处理器以及基于龙芯处理器实现的Java虚拟机平台。

第四章，提出一种基于Java虚拟机控制流的容错方法。首先描述了软错误可能对Java虚拟机控制流带来的危害，并对相关工作进行介绍。在介绍了背景之后描述了Java虚拟机热再生方法。最后对该方法的错误覆盖率和性能进行了评测和分析。

第五章，提出一种基于Java虚拟机数据流的容错方法。首先介绍了软错误对Java虚拟机数据流产生的危害和针对这一问题进行的相关工作。之后描述了基于污点分析的CST校验方法以及实现细节和相关优化。最后对该方法对Java虚拟机的性能和内存消耗进行了评测和分析。

第六章，提出了一种基于龙芯处理器的64位Java虚拟机访存和取址优化方法。本章首先阐述了64位Java虚拟机在MIPS处理器上访存和取址的问题，然后介绍了相关工作。接下来本章对Java虚拟机的地址加载行为进行了分类和分析，在此基础上提出了基于硬件的快速访存方法和基于软件的地址加载方法。最后对方法的性能优化效果和内存优化效果进行了评测与分析。

第七章，对整个论文进行了总结，并给出了将来的研究方向。

第二章　面向软错误的容错技术的研究现状

针对软错误的容错技术有很多，而且应用于计算机系统的各个层面，包括硬件、操作系统、虚拟机、编译器等。不管应用于哪个层面，一个基本思想都是利用冗余来实现容错机制。在硬件层面实现容错，错误覆盖率高，性能好，但是需要额外的硬件支持，成本高；在软件层面实现容错也可以实现较好的错误覆盖率，而且不需要额外的硬件支持，尤其适用于嵌入式系统，但是软件实现的容错技术普遍会对系统造成比较大的性能开销和内存开销。如何在错误覆盖率、内存消耗和程序性能之间进行优化和权衡是目前容错技术研究的重点。

本章的结构如下：第一节介绍一些本文需要用到的容错技术的基本概念；第二节对容错技术的研究进展进行总结和介绍；第三节介绍一些容错技术的硬件实现；第四节介绍一些容错技术的软件实现；最后一节总结本章。

2.1　软错误基本概念

用户可见的错误（Error），比如软错误，是计算机系统潜在故障（Fault）的一种表现形式。有了故障才会导致错误，但是并不是所有故障都会导致错误。比如某一个计算机系统的内存中出现了一个故障，由于该计算机系统的内存具有硬件容错机制，因此该故障并不会体现为一个用户可见的错误。或者含有故障的内存并没有被程序访问到，这个故障也不会体现为一个错误。计算机中的故障又分为永久性故障、瞬态故障和间歇性故障。本文接下来并没有严格对故障与错误、瞬态故障与软错误进行严格区分。

不同情况下的软错误对系统最终的影响有所不同，一般可以按照对系统的影响把软错误分成三类：Benign Fault，SDC（Silent Data Corruption）和DUE （Detected Unrecoverable Error）。发生了软错误的部件没有被程序访问到，或者被访问到但是并没有改变程序的正常执行，这类错误被称为Benign Fault。SDC指的是发生了软错误，但是没有被检测到，并且最终产生了用户可见的结果，比如错误的输出。Benign Fault和SDC不属于同一范畴，Benign Fault对系统无害，SDC对系统有害。DUE指的是被检测到但是没有恢复的错误，这类错误可能会最终造成系统的崩溃。DUE又分为False DUE和True DUE。False DUE是那些本来是Benign Fault而被检测到的错误，True DUE则是指那些如果没有被检测到，则会成为SDC的DUE。这些错误的分类以及来源如图2.1所示。

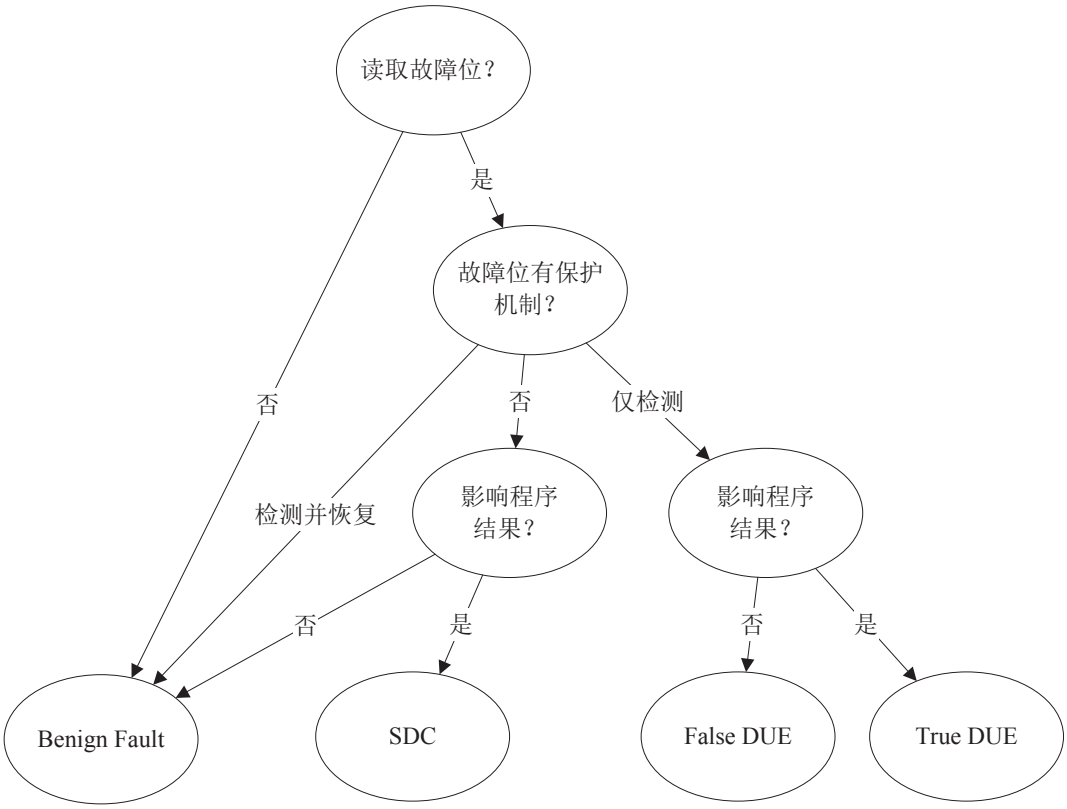


图 2.1　错误分类图[1]

一般来讲，相对于DUE，SDC对系统更为有害。SDC会造成数据的丢失以及一些潜在的不可预知的错误，而DUE通常只是会造成系统停止从而无法继续提供服务。例如，在网上支付时发生了某个错误，用户宁可希望支付失败也不希望支付了一个错误的数额或者支付到一个错误的地址。

错误检测技术的目的就是要尽可能的将SDC转换为DUE，也就是尽可能多的发现系统中的软错误。另一方面，False DUE会造成程序不必要的中断和性能开销，错误检测技术应避免将系统中不会对程序产生影响的错误，即Benign Fault，识别为DUE。错误恢复技术的目的是尽可能的将DUE转换为Benign Fault。错误检测和错误恢复与错误之间的转换关系见图2.2。

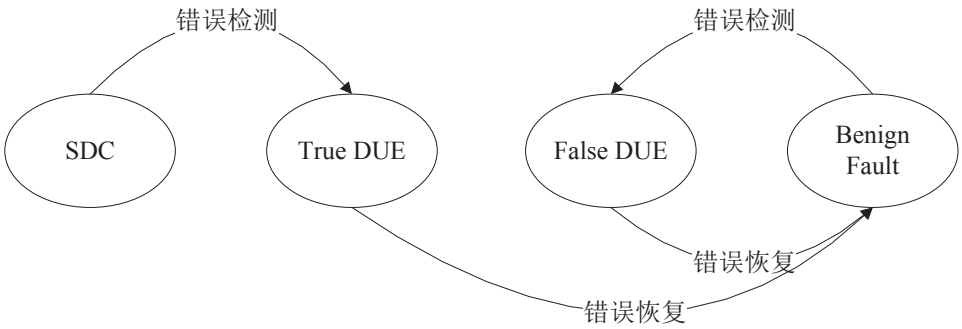


图 2.2　容错系统的作用

SoR

大部分针对软错误的容错技术的核心思想是冗余计算。为了有助于描述基于冗余计算的容错方案，SoR（Sphere of Replication）[23]是一个被普遍使用的概念。SoR描述了容错方案所保护的逻辑区域。一个容错方案的SoR表明任何发生在SoR内的错误都可以被这个容错方案检测到。图2.3的例子显示了一个容错系统的SoR包含处理器，但是不包含内存、硬盘以及I/O系统。

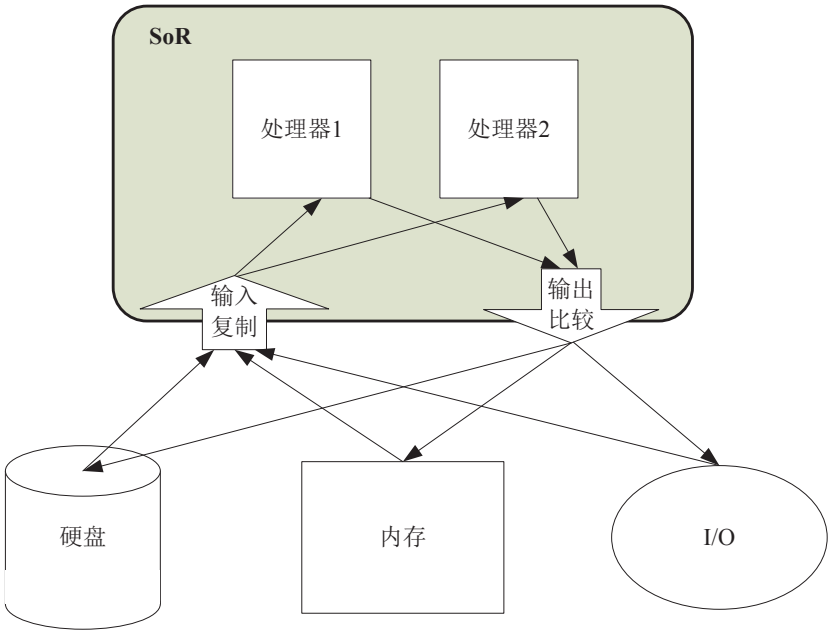


图 2.3 SoR举例（包含处理器）

SoR概念包含三部分内容。

1. **输入复制（Input Replication）**：进入SoR之前，数据需要被复制为SoR需要的副本形式并存储至相应的位置。比如，在双模冗余系统中，数据需要被复制成两份，在三模冗余系统中，数据需要被复制成三份。
2. **冗余执行**：在SoR内的程序会进行多次执行。冗余执行又分为时间冗余和空间冗余。时间上的冗余执行指的是在同一个部件上串行地重复执行SoR中的程序，一般多用于串行程序。空间上的冗余执行指的是在多个部件上并发地执行SoR中的程序，一般需要硬件支持，多用于并行程序。比如在图2.3中的例子中，SoR中采用的就是空间冗余。
3. **输出比较（Output Comparison）**：在离开SoR时，输出数据需要进行比较以判断SoR中的程序是否正确地完成了执行。如果输出结果出现差异，那么说明SoR至少有一个冗余执行出现了错误，则容错系统会采取相应的错误应对措施。输出比较模块通常被称为Checker。在设计输出比较模块时，要尽量减少无效的比较，这会增加性能代价同时还不会提升错误覆盖率。有的方案采用专门的硬件电路来进行输出比较，比如使用看门狗。

SoR的大小

不同容错系统的SoR的大小有所不同。SoR的尺寸会影响计算机系统的一些属性，SoR过大会影响计算机系统各个部件之间的独立性，但是会简化输入复制和输出比较的设计。相反地，SoR过小会增加硬件之间的独立性，但是会增加输入复制和输出比较的设计复杂度和它们带来的性能损失。另一方面，SoR的大小由容错系统的设计者所处的层面决定。一个软件系统提供商最多只能提供软件层面上的容错；一个系统集成商可以将SoR控制在两三个冗余芯片的范围；而处理器设计者可以将SoR缩小至单个芯片内部的流水线或寄存器层面。比如，容错公司Stratus的产品ftServer[24]的SoR就如图2.4所示，包含处理器和内存。而Hewlett-Packard Himalaya系统[25]的SoR如图2.3所示，只包含了处理器，没有包含内存和I/O系统。

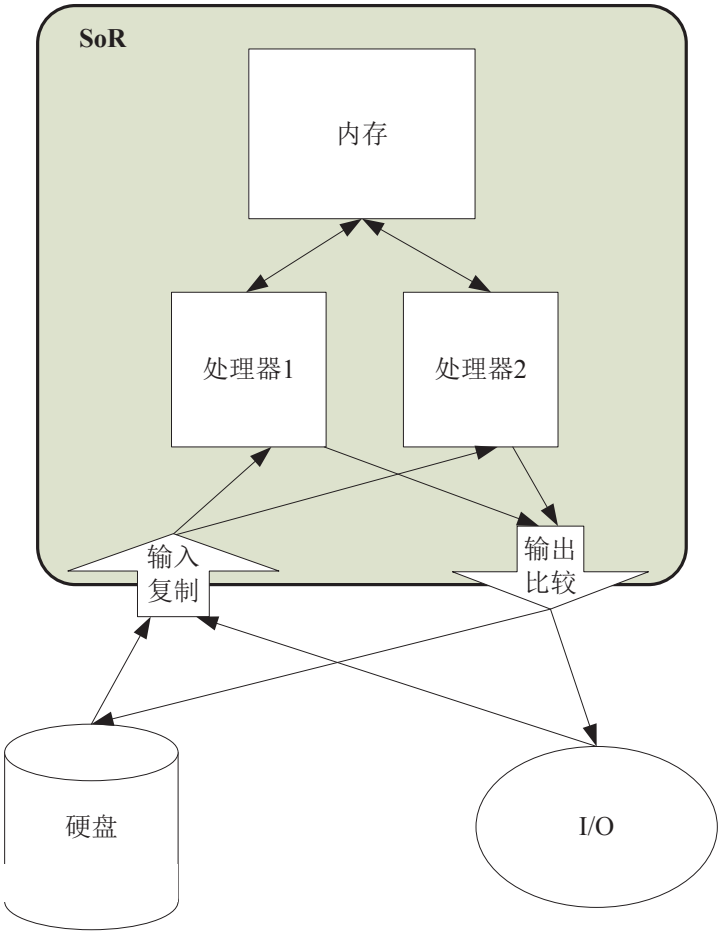


图 2.4　SoR举例（包含处理器和内存）

2.2　容错技术的基本方法

容错技术的基本思想就是利用冗余。它可以从多个角度进行分类：从实现的功能可以分为错误检测和错误恢复两类；从冗余资源形式可以分为空间冗余、时间冗余以及信息冗余；从实现层面可以分为硬件容错、软件容错以及软硬结合容错；错误恢复

又可以从实现方式上分为前向错误恢复和后向错误恢复两种。本节列举并介绍一些容错技术中的经典技术方法和研究现状。

2.2.1 Lockstep技术

自从上个世纪80年代以来，Lockstep（锁步，也称时钟同步）技术就开始被应用于高可靠的计算机系统。当时，越来越多的公司需要提供7×24小时不间断的在线服务，这对当时的计算机系统的可靠性提出了很高的要求。为了满足这一市场需求，Stratus、Tandem和IBM等公司先后推出了支持Lockstep技术的系统。在Lockstep系统中，需要对硬件进行完全复制，程序在两个冗余硬件上以Lockstep方式执行，即按节拍对比二者的执行结果。如果在比较的过程中发现不一致，那么则检测到了错误，将SDC转换为DUE。Lockstep技术是一种错误检测技术，并不具有错误恢复的功能。典型的Lockstep系统有：Stratus ftServer、Hewlett-Packard NonStop Himalaya和IBM Z-series处理器。

Lockstep技术给系统带来了两个限制。第一，它要求冗余的副本是完全确定性的，即同样的输入必须给出同样的输出。第二，它要求硬件必须支持确定性重置（Deterministic Reset），也就是通过重置，所有冗余的处理器或者芯片必须能够恢复到一个相同的确定的状态。例如，在没有使用Lockstep技术时，分支预测器的初始状态并不会影响程序的正确执行。但是，当冗余处理器的分支预测器具有不同的初始状态时，它们的执行可能会产生偏离并造成执行不匹配。

Lockstep的优点

Lockstep技术可以给系统带来很高的错误覆盖率，它可以检测到SoR内的所有软错误和永久性错误。唯一一种Lockstep技术无法检测的错误就是所有冗余硬件中发生了同样的错误，这种情况下的输出比较的结果是相同的，从而程序会被判定为正确执行。这种在冗余硬件中发生相同错误的概率是非常低的，除非是硬件设计上的缺陷所导致。Lockstep技术可以通过纯硬件的方式实现，实现对软件层面透明。

Lockstep的缺点

Lockstep技术也存在几个比较显著的缺点。第一，由于要使用冗余硬件，采用Lockstep技术的系统的成本要比普通计算机系统高很多。第二，实现处理器或者芯片数月甚至数年以确定状态执行是比较困难的。第三，Lockstep技术不具备错误恢复功能，往往需要搭配错误恢复技术的支持。第四，Lockstep技术会产生很多False DUE。因为Lockstep技术会将所有Benign Fault和SDC均转换为DUE，而False DUE会造成系统不必要的停顿。

2.2.2 冗余多线程技术

类似于Lockstep技术，冗余多线程（Redundant Multithreading，简称RMT）也是同时执行程序的多个副本，并且比较它们的输出从而判断是否有错误的发生。不同的是，

Lockstep是以时钟周期为界限进行比较，而RMT是以指令为界限进行比较。就像“多线程”一样，RMT也是一种概念，一种容错思想，它可以有具体不同的实现。RMT几乎可以在所有具备多线程功能的系统中实现。在RMT系统中，需要对哪些输出进行比较由SoR的大小而定，比如，如果像图2.3中的例子一样，SoR只包括处理器，那么就需要对写入到内存和I/O的数据进行比较；如果像图2.4 中的例子一样，SoR包括了处理器和内存，那么只需要对输出到I/O的数据进行比较。RMT对系统的限制要小得多，这有助于减少硬件代价并提高运行效率。其主要优点有以下几个方面：

1. 抽象级别高，因此实现容易，适应性好，对具体的工艺和电路技术依赖小；
2. 设计灵活，相对宽松的限制使RMT可以从多个层面进行实现。例如，RMT可以在系统间、单个系统内、多个处理器核之间、单个处理器核内部进行实现。另外，RMT也可以通过纯软件的方式进行实现；
3. 透明性好。

典型的通过硬件实现的RMT系统有：Marathon Endurance Server、Hewlett-Packard NSAA、SRT、CRT和DIVA。后三者均在同一个处理器芯片中实现了RMT。典型的通过软件实现的RMT系统有：EDDI（Error Detection By Duplicated Instructions）[26]、SWIFT（Software-Implemented Fault Tolerance）[27]和Spot[28]。EDDI和SWIFT通过改造编译器，使编译器生成含有冗余执行的代码从而实现错误检测。Spot 则不需要程序的源代码，它是通过二进制翻译的方式，直接在二进制代码层面实现容错。从软硬结合层面实现RMT的系统有CRAFT（Compiler-Assisted Fault Tolerance）[29]。软件实现的RMT将在第四章详细介绍。

RMT优化

RMT的缺点是有可能会产生较大的性能开销，尤其是通过软件实现的方式。近年来，如何保证RMT在提供足够的错误覆盖率的同时，减少其对系统带来的性能损害和内存开销是研究的一个热点。降低RMT性能开销的方法可以从三个思路出发：减少输入复制的开销；减小输出比较的开销；减小冗余执行过程中执行的指令数。

- **优化输入复制**。
- **优化输出比较**：过于严格的输出比较并不会提高系统的错误覆盖率，反而还会降低性能，可以通过减少输出比较实现对系统的优化。比如，有些系统通过只对特点指令集合的输出进行比较实现了对系统整体性能的优化[27]；在基于事务的系统中，以事务为单位对输出进行比较，也能够优化系统的效率。
- **优化冗余执行**：另一种优化性能的手段是减少SoR内执行的指令数。[30]提出了一种指令重用的方式，作者提出了一种指令重用缓存，如果一条或者一组指令被重新执行并且跟指令重用缓存中的指令匹配，那么缓存中的输出会被直接使用，而无需再次执行。还有一些作者提出在系统高IPC、高功耗或者低AVF（Architectural Vulnerability Factor）时关闭冗余执行。[31]提出了一种在

程序高IPC时关闭副线程的方法。

2.2.3 信息冗余技术

信息冗余技术指的是在原始数据中增加一些冗余位，从而达到容错的目的。信息冗余主要是通过编码技术实现，分为错误检测和错误恢复两类。编码技术是目前使用最广泛也最有效的容错技术之一，常用于信息的传输、存储和处理过程中。通过编码技术可以发现一位错误也可以发现多位错误。常见的编码有奇偶校验码（Parity）、汉明校验码（Hamming Code）、循环冗余码（Cyclic Redundancy Check，简称CRC）等。本小节首先介绍编码技术的基本概念和原理，然后分别介绍奇偶校验码、汉明码和循环冗余码。

编码技术的基本原理由图2.5中的流程图所示。原始数据经过编码过程生成包含数据位和编码位的编码后数据，经过传输、存储或者处理，编码后的数据经过解码过程恢复出数据位和编码位。若恢复后的数据位和编码位符合相应的编码规则，那么说明没有发生错误，反之则说明在数据的传输、存储或者处理的过程中发生了错误。

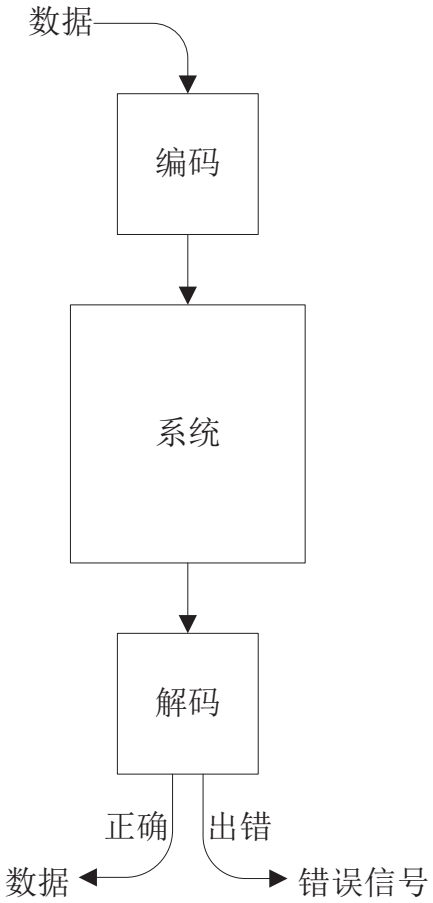


图 2.5 利用编码技术实现容错的基本原理

最简单的编码方式是编码位等于数据位。例如，假设图2.5中的数据*data_bit*为1位，

那么编码$code_bit$也为1位，那么编码后的数据为$< data_bit, code_bit >$，编码规则为：

$$code_bit = data_bit \tag{2-1}$$

因此，在该例中，编码后数据可能为00或者11。经过系统之后的数据如果是00或者11，那么则说明在系统中没有发生错误。如果经过系统后的数据为10或者01，那么则说明系统中发生了错误。该例中的编码方式能且只能检测出1位错误，并不能纠正错误。

将例子中的编码位扩展成2位则可以实现1位的错误恢复。编码后的数据形式为$< data_bit, code_bit_1, code_bit_2 >$，编码规则为：

$$\begin{aligned} code_bit_1 &= data_bit \\ code_bit_2 &= NOT(data_bit) \end{aligned} \tag{2-2}$$

使用这种编码方式编码后的数据可能为001或者110。除此之外的数据均为出错数据。比如，如果经过系统后的数据为010，那么说明数据位出现了错误，将数据位恢复为1即可；如果经过系统后的数据为011，那么说明编码位$code_bit_1$在系统中出现了错误，数据位并没有出现错误，解码后直接使用数据位0即可。

奇偶校验码

在实际应用中，奇偶校验码是最简单的编码方式，它能够检测出奇数位错误，不能纠正错误。奇偶校验码有两种形式：奇校验码和偶校验码。奇偶校验的形式是多位数据位对应1位校验位。如果数据位和校验位中1的个数为奇数个，那么就是奇校验；如果数据位和校验位中1的个数为偶数个，那么就是偶校验。如果发生奇数位个数据翻转，那么编码中1的个数就会发生变化，从而检测出错误。k位数据$a_0 a_1 \cdots a_{k-1}$的偶校验位$parity_code_{even}$可以通过以下公式计算（$\oplus$表示按位异或）：

$$parity_code_{even} = a_0 \oplus a_1 \oplus \ldots \oplus a_{k-1} \tag{2-3}$$

在设计具体的实现时，需要考虑奇偶校验粒度。常见的两种方式是以字节为单位进行校验和以字为单位进行校验。这两种方式各有优缺点。使用以字节为粒度的奇偶校验时，在读取某一字节时，不需要读取字节所在的整个字长来进行校验。例如，在一个计算机系统中，一个字包含两个字节，二进制形式为 00000001和00000010。当需要读取第一个字节时，以字为单位进行校验的系统需要读取整个字已完成校验计算，这可能会增加读取内存的操作，还可能会影响Cache 效率，从而造成性能开销。另外，假设第二个字节发生了位翻转，以字为单位进行校验的系统在读取第一个字节的时候也会发生校验错误，从而造成校验不够精确。另一方面，以字为单位进行校验的优点是节省内存空间。

使用奇偶校验码的扩展形式可以实现多位错误检测或者1位错误恢复。水平垂直奇偶校验是其中典型的代表，通过水平方向和垂直方向的校验，可以通过坐标定位错误的位置。图2.6给出了一个水平垂直校验的实例。该例中每12位原始数据以4×3的方式排列成一个二维数组。数组每行和每列均对应一个偶校验位。若数据中某一位发生错误，则相应的行与列的校验会失效，从而根据行号和列号定位出错位的位置。

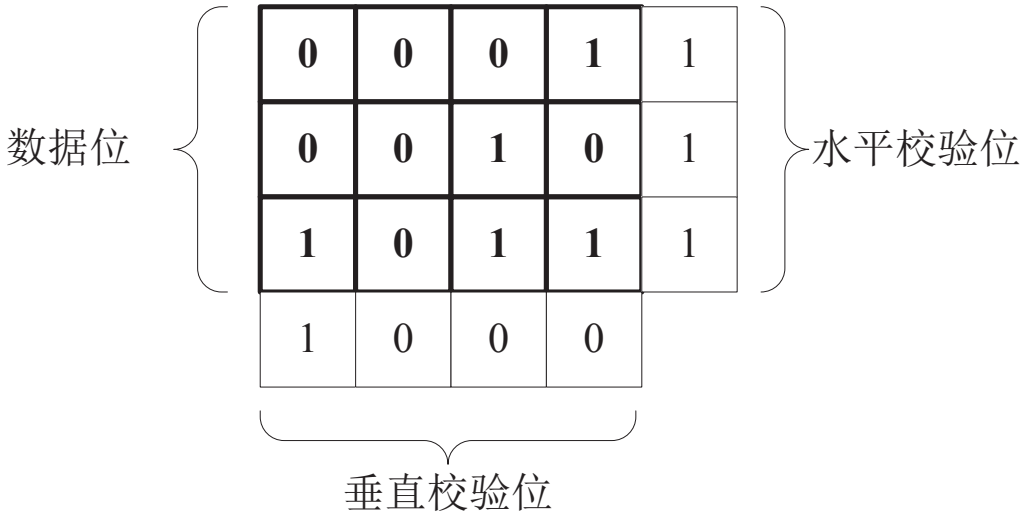


图 2.6　水平垂直奇偶校验例子

汉明校验码

汉明码，又称海明码，也是一种利用奇偶性来校验数据的编码，以其发明者Richard Hamming的名字命名。汉明码在原始数据中插入校验码，已检测并纠正错误。汉明码有多个版本，包括SEC（Single Error Correction）、SECDED（Single Error Correct, Double Error Detection）和DECTED（Double Error Correct, Triple Error Detection）。汉明码也可以使用汉明(n, k)的形式表示，其中n表示总的编码后的位数，k表示数据位数。

本小节以最基础的SEC码、汉明(7, 4)码为例说明汉明码的基本思想。汉明(7, 4)码说明原始数据位有4位，编码位有3位。编码后的数据E以矩阵的形式表示为$[C_1C_2D_1C_3D_2D_3D_4]$，其中C表示编码位，D表示数据位。编码C_n的计算公式为：

$$\begin{aligned} C_1 &= D_1 \oplus D_2 \oplus D_4 \\ C_2 &= D_1 \oplus D_3 \oplus D_4 \\ C_3 &= D_2 \oplus D_3 \oplus D_4 \end{aligned} \tag{2-4}$$

根据公式可以计算出校验矩阵P，如表2.1所示。

表 2.1　数据$D_1D_2D_3D_4$所对应的汉明(7, 4)码的校验矩阵

C_1	C_2	D_1	C_3	D_2	D_3	D_4
0	0	0	1	1	1	1
0	1	1	0	0	1	1
1	0	1	0	1	0	1

最终结果S可以通过以下公式计算得出：

$$S = P \cdot E^T \tag{2-5}$$

公式2-5中E^T为E的转置。假设现在有数据$D_1D_2D_3D_4$为0101，那么根据公式2-4计算得出E为[0100101]，这是一个符合汉明码的数据。该数据通过公式2-5计算得出：

$$S = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{2-6}$$

如果S表示为$[S_1S_2S_3]$，则$S_1 = (0\ AND\ 0) \oplus (0\ AND\ 1) \oplus (0\ AND\ 0) \oplus (1\ AND\ 0) \oplus (1\ AND\ 1) \oplus (1\ AND\ 0) \oplus (1\ AND\ 1) = 0$。计算结果$S_n$均为0表示没有发生错误。如果数据$D_3$的发生了位翻转，则数据$E'$变为：[0100111]，那么公式2-5的计算结果为：

$$S' = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \tag{2-7}$$

S'的二进制形式为110，转换为十进制为6，表示$C_1C_2D_1C_3D_2D_3D_4$的第6位D_3出现了错误。

CRC校验码

CRC，即循环冗余校验，由Wesley Peterson于1961年发表[32]，该方法多适用于检测传输通道干扰引起的错误。CRC不具备错误恢复功能，只能检测到错误。因此，在传输过程中如果接受者发现错误的话，一般的处理方式是发送错误信号让发送者重新发送一遍数据。多项式除法是进行CRC编码和解码的核心思想。CRC被广泛应用于通信和软件协议中，三种最常用的CRC为：

- CRC-12：$G(x) = x^{12} + x^{11} + x^3 + x^2 + x^1 + 1$
- CRC-16：$G(x) = x^{16} + x^{15} + x^2 + 1$
- CRC-CCITT[3]：$G(x) = x^{16} + x^{12} + x^5 + 1$

Arithmetic Code

上述编码技术一般适用于对传输部件或者存储部件进行检测。如果数据要进行算数运算等运算操作时，需要先对数据进行解码和校验，然后运算，最后再对结果进行编码。这种方式有两个问题，一是不能检测运算部件中是否发生了错误；一是如果程序包含大量的运算，这可能会导致大量的编解码操作，从而造成过高的性能开销。Arithmetic Code[33,34]可以缓解这一问题。Arithmetic Code的特点是编码也可以跟随数据一起参与运算，并且符合编码规则。例如，$C(a)$ 表示数据a的编码，那么Arithmetic Code符合：

$$C(a\ op\ b) = C(a)\ op\ C(b) \tag{2-8}$$

在等式中，op表示算数操作。通过该等式可以看到，a和b的运算结果的编码可以通过直接对a的编码和b的编码进行同样的运算得到。这样就省去了对操作数解码和运算结果的编码过程。AN Code和Residue Code是两种Arithmetic Code的代表。

AN Code的编码方式很简单，正如其名称所示，将数据N乘以A即得到了N的编码。AN Code可以应用于加法和减法运算，因为$A \times (N_1 + N_2) = A \times N_1 + A \times N_2$以及$A \times (N_1 - N_2) = A \times N_1 - A \times N_2$。但是AN Code并不适用于乘法和除法运算，比如$A \times (N_1 \times N_2) \neq (A \times N_1) \times (A \times N_2)$。在实际应用中，A的选择根据系统的具体特点而定，常用的做法是把A设为3，这样$A \times N$可以通过对N进行左移1位再加上N实现。

Residue Code是另一种Arithmetic Code，它的运算规则为求余运算。因此Residue Code可以被应用于加法、减法、乘法和除法操作：

$$\begin{aligned}
&(N_1 + N_2)\ MOD\ M = (N_1\ MOD\ M + N_2\ MOD\ M)\ MOD\ M \\
&(N_1 - N_2)\ MOD\ M = (N_1\ MOD\ M - N_2\ MOD\ M)\ MOD\ M \\
&(N_1 \times N_2)\ MOD\ M = (N_1\ MOD\ M \times N_2\ MOD\ M)\ MOD\ M \\
&(D\ MOD\ M - R\ MOD\ M)\ MOD\ M = ((Q\ MOD\ M) \times (I\ MOD\ M))\ MOD\ M
\end{aligned} \tag{2-9}$$

在上述除法公式中，D是被除数，I是除数，Q是商，R是余数[6]。Residue Code还可以进一步扩展支持更多的运算。在[35]中，作者实现了Residue Code对移位计算的支持。[36]研究了Residue Code对浮点运算的支持。在IBM的z6处理器的流水线中，也加入了对Residue Code的支持[37]。在[38]中，作者实现了一个能够产生自检测乘法器的CAD系统。

一些非Arithmetic Code可以通过使用Check Prediction适用于整数运算。奇偶校验[34,39]，校验和（Checksum）[36]和Berger码[40]都已被用于对算术运算单元进行保护。

2.2.4 前向错误恢复

错误恢复技术不同于错误检测，它要求程序在遇到错误的时候能够从错误中恢复出来，使程序继续正常执行。错误恢复技术可以分为两大类：前向错误恢复（Forward Error Recovery）和后向错误恢复（Backward Error Recovery）。前向错误恢复指的是当遇到错误时，程序可以继续执行，而不用恢复到之前的状态。有纠错能力的编码技术就是一种前向错误恢复，在发现错误时，纠错码可以修正错误保证程序继续正确执行。由硬件实现的前向错误恢复技术往往需要比较高的成本。常见的前向错误恢复方法有：双模冗余（Double Modular Redundancy，简称DMR）系统，三模冗余（Triple Modular Redundancy，简称TMR）系统以及N版本编程（N-Version Programming）方法。

双模冗余系统

双模冗余系统，也称DMR系统，指的是两个模块同时执行同一个程序。可以通过Lockstep技术和RMT技术进行错误检测，其工作流程如图2.7 所示。在检测到错误以后，DMR系统需要判断两个模块中的哪一个模块出现了错误。常用的办法是在输出比较部分发现了输出不匹配时，输出比较模块会继续等待两个模块中发生错误的模块发出的信号。收到信号后，DMR系统冻结未出错的模块并将其拷贝至出错模块。在拷贝完成之后，错误模块中的错误被消除，两个模块继续同时进行。

对于硬件全系统冗余，按照其工作方式可以分为双系统冷备、双系统温备、双系统热备和双系统双工4种[41]。双系统冷备指的是一个系统处于工作状态，一个系统处于关机状态，在工作状态的系统发生故障时未加电的系统启动替换故障系统；双系统温备指的是一个系统处于工作状态，一个系统处于带电等待状态，工作状态系统发生故障时由等待系统替换工作；双系统热备指的是两个系统同时工作，工作机出现故障时由值班机替换工作；双系统双工指的是两个系统同时工作，对输出结果进行比较判断故障。双系统双工即DMR的一种形式。

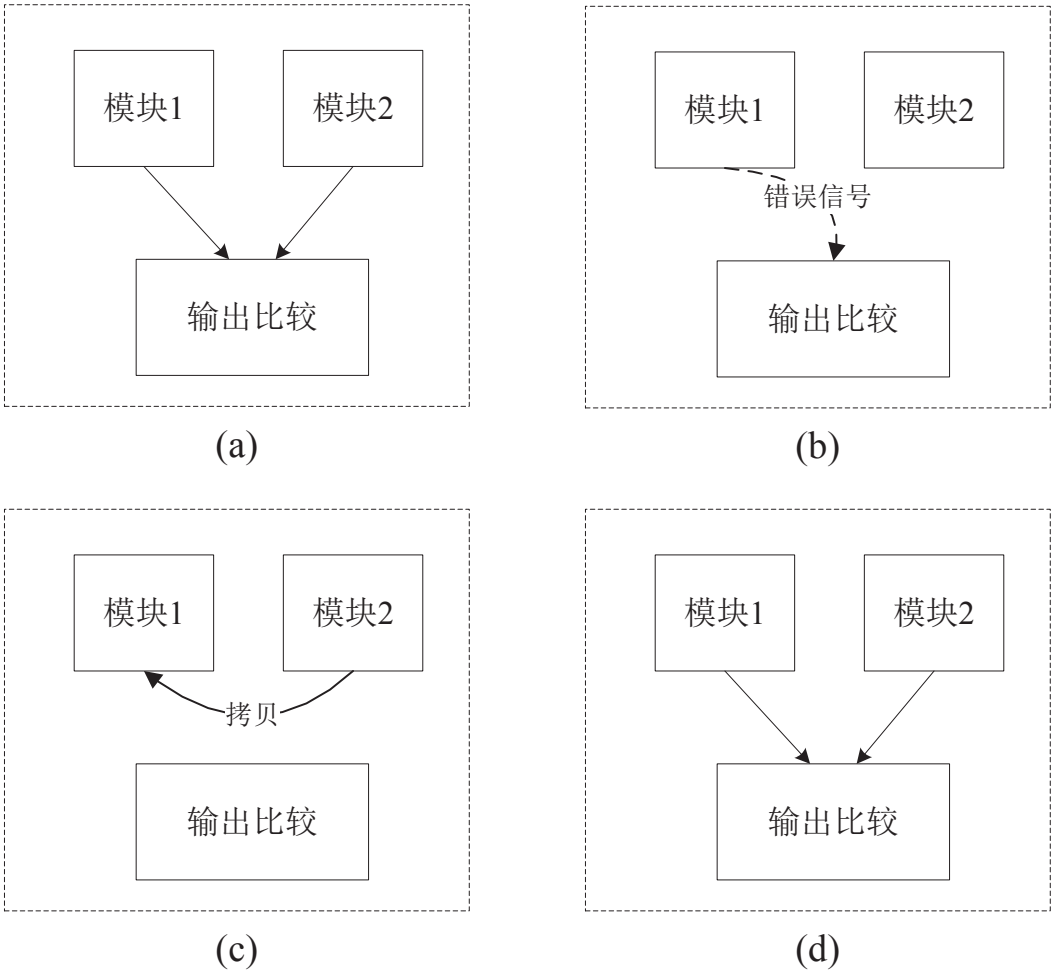


图 2.7 DMR工作流程图

DMR系统要求模块能够识别错误并发出错误信号。对于那些不能识别的错误，即使输出比较部分发现了不匹配，也不能识别是哪个模块发生的错误。因此这种情况会导致DUE，并不能消除DUE实现错误恢复。

三模冗余系统

三模冗余，即TMR，解决了恢复不能识别的错误的问题。TMR指的是三个模块同时执行同一个程序（图2.8）。在输出比较时，如果三个输出相同，说明没有发生错误；如果有一个模块的输出与另外两个不同，说明该模块发生了错误。TMR系统可以同时容忍一个模块发生故障而不影响系统的正确执行。TMR方式不需要模块内部发出错误信号，通过输出比较就可以完成错误的识别。但是，TMR的代价也要高于DMR。

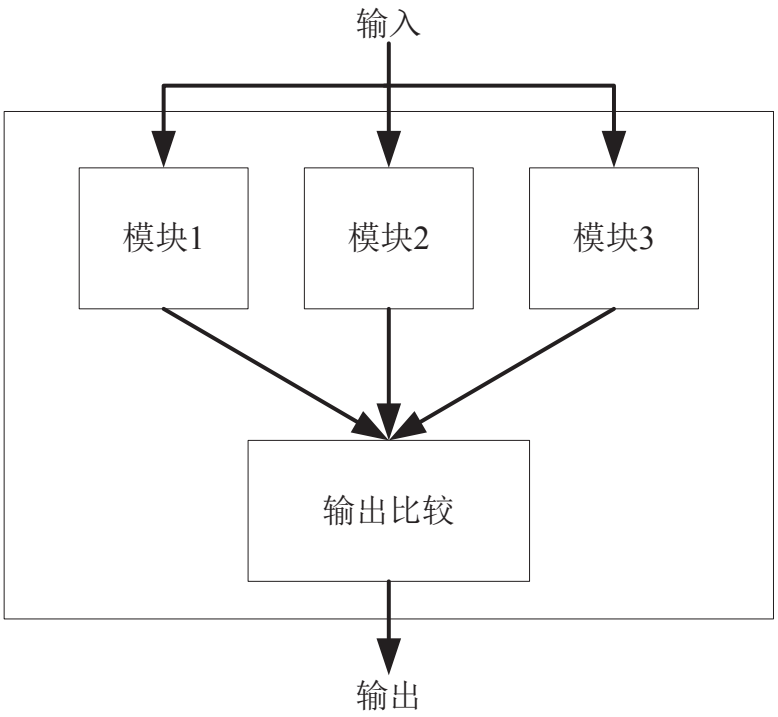


图 2.8　TMR工作流程图

在检测到错误之后，TMR通常有两种方式进行处理。第一，停止出错的模块，将整个系统降级为DMR模式继续运行。停止模块的信息可以记录在日志文件中或者发送出错信号通知相关人员对出错模块进行调试或者重启。第二，发现错误后，TMR系统选择一个合适的时间停止整个系统的执行，然后将正确的模块覆盖至出错的模块。覆盖之后，错误模块被恢复，三个模块继续执行。对于处理软错误来讲，第二种方法更为合适。但是第二种方法会因为数据的拷贝导致系统出现停机时间，降低了系统的可用性。为了解决这一问题，通过增加额外硬件电路，有些系统可以实现程序执行与拷贝同时进行[42]。

对一些可靠性要求特别高的系统，多模冗余（N–Modular Redundancy，简称NMR，N＞3）是一种经常采用的方案，其基本思路与上述DMR和TMR类似。这类系统可以容忍多个模块同时发生错误，但也会付出更高硬件代价或者性能代价。二乘二取二（Double 2-Vote-2 Redundancy）就是其中一个例子（图2.9）。二乘二取二系统结合了DMR双系统热备的优点，具有高安全性和高可靠性。

N版本编程方法

N版本编程[43]指的是使用多种不同的软件版本实现同一个功能，并用一个表决器判断输出。不同的软件版本由不同的团队独立设计开发，使用不同的方法、不同的程序语言、不同的开发工具来实现。使用这种方式开发的容错系统可以将各版本之间的软件相关性降至最小，因此能够避免由于软件设计上的缺陷所导致的错误。

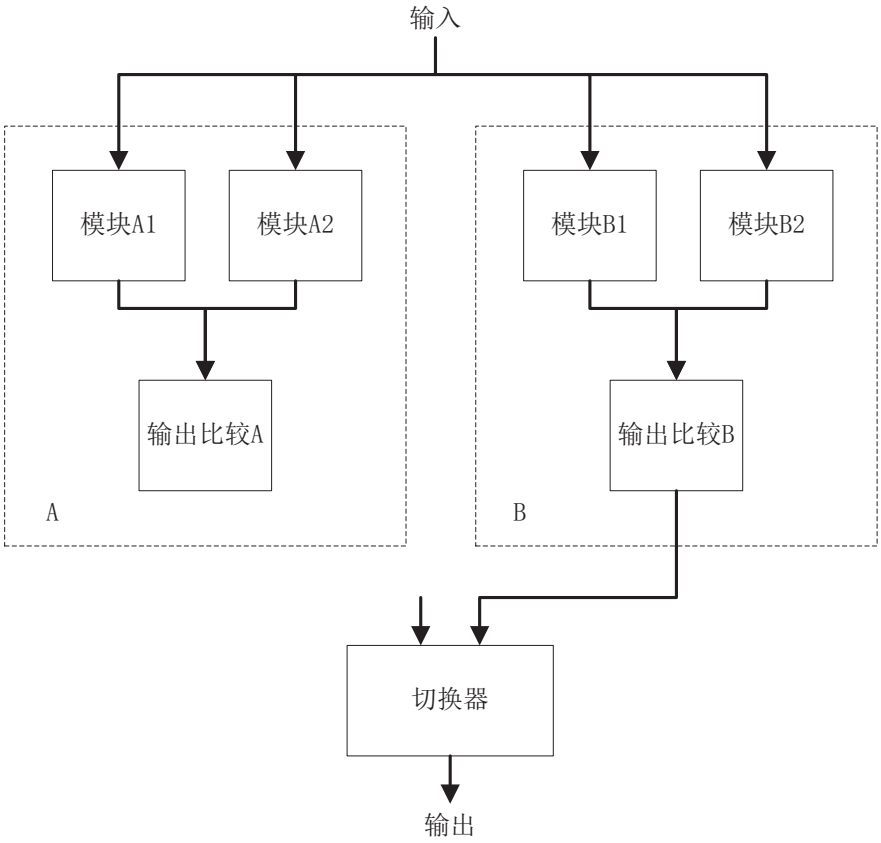


图 2.9 二乘二取二系统结构图

2.2.5 后向错误恢复

后向错误恢复指的是在发生错误之后，系统可以回退至之前的某个正确状态继续执行。后向错误恢复一般比前向错误恢复需要更少的硬件代价。典型的后向错误恢复技术有恢复块（Recovery Block）和回卷恢复（Rollback）。

恢复块方法如图2.10所示。程序在进入恢复块的入口时创建检查点，然后开始执行主块。执行完主块后进行验证，若验证通过则说明程序没有错误，此时程序退出恢复块并舍弃之前保存的检查点信息。如果验证没有通过，或者执行过程中发生了异常，那么程序回退至入口处，开始执行下一个备选块并进行验证。如此反复，直到有一个备选块能够正确执行为止[44]。如果所有备选块都不能正确执行，那么抛出一个错误异常。在设计程序时，要保证主块和备选块之间的独立性，以防止由于软件设计缺陷所引起的错误。

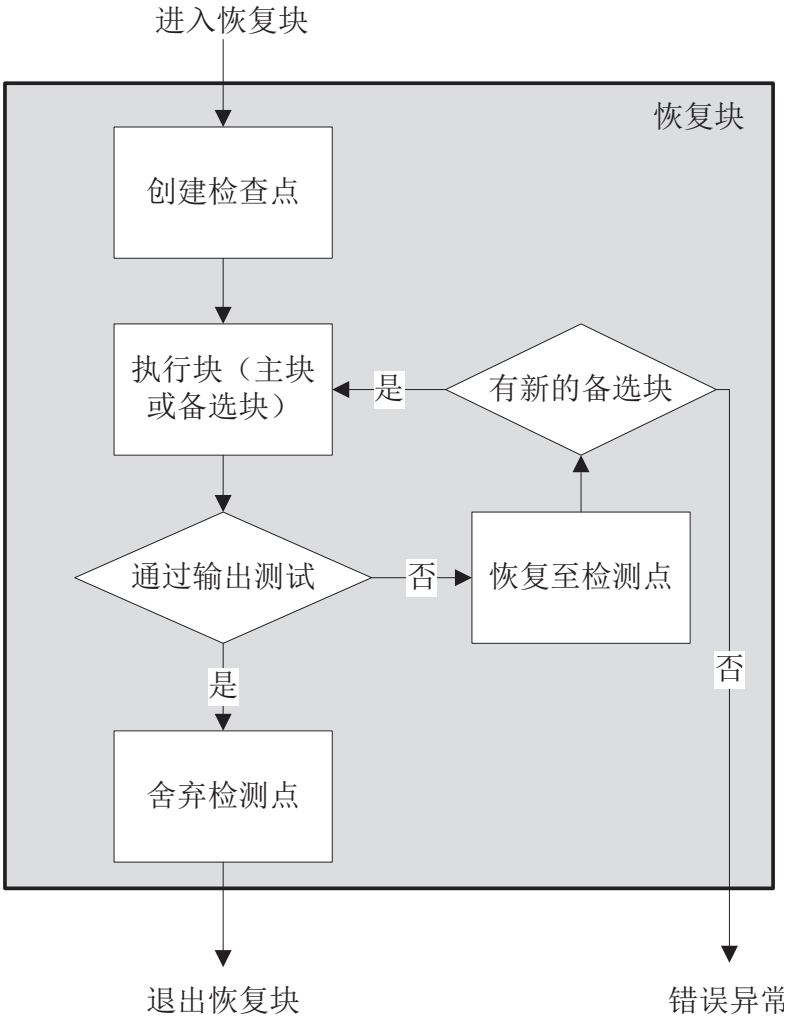


图 2.10　恢复块的工作流程

回卷恢复可以分为基于检查点（Checkpoint-Based）的回卷恢复和基于日志（Log-Based）的回卷恢复两种。

基于检查点的回卷恢复的核心思想是将系统运行的状态定期存储起来，这样在遇到错误时，系统可以退回至最近的检查点重新执行。它不依赖PWD 假设，因此它不需要记录日志或者重放非确定的事件。基于检查点的回卷恢复实现起来较为简单。这种方式不适合用于与外界交互比较多的系统。基于检查点的系统又可以细分为独立式检查（Uncoordinated Checkpointing），协作式检查（Coordinated Checkpointing）和通信引导式检查（Communication-Induced Checkpointing）[45]。

基于日志的回卷恢复将不确定事件通过日志的形式记录，同时系统也会定期记录检查点以减少发生错误时回退的距离。当错误发生时，系统首先恢复到上一个检查点记录的无错状态，然后再根据日志信息处理先前处理过的消息。因此，基于日志的回卷恢复能处理多种类型错误，尤其适合与外界交互性强的程序。基于日志的回卷恢复也有三类[45]：

- **悲观日志（Pessimistic Logging）**：程序在处理消息前，必须保证消息已经记录到稳定存储器中。这种方法也称同步记录。这种方式实现简单，但是性能开销大；
- **乐观日志（Optimistic Logging）**：这种方式认为在发生错误前消息记录能够完成。采用异步方式记录消息，在消息纪录时不会阻塞程序的执行，因此性能很好，但是系统比较复杂；
- **因果日志（Causal Logging）**：结合了悲观日志和乐观日志的优点，既能避免以同步方式访问稳定存储器，又能实现检查点的异步设置。

2.3 硬件实现

根据上一节介绍的错误检测和错误恢复技术，本节在表2.2中列举了一些有代表性的通过硬件实现的例子及其各自的特点[46]。

表 2.2 使用不同容错技术的硬件实现方案及其各自特点

	硬件冗余		多线程RMT	多核RMT
	DIVA[47]	Lockstepped流水线[48]	[23]	[49]
机制	部分Replication	完全Replication	时间冗余	时间冗余、空间冗余
错误类型	所有	永久，瞬态	瞬态	永久，瞬态
性能代价	低	非常高（大于2倍）	高（1.5至2倍）	中等（小于1.5倍）
功耗代价	低	非常高（大于2倍）	高（1.5至2倍）	中等（小于1.5倍）
面积代价	低	高	无	无

2.4 软件实现

从软件层面实现容错有很多好处：实现成本低；不依赖具体硬件；应用灵活；充分利用现代处理器的并行计算能力；利于实现优化；误报率更小。

很多软件容错技术的原理同硬件容错技术一样，核心思想都是利用冗余执行实现错误检测和错误恢复。软件容错是通过软件的方式实现了RMT、信息冗余、前向错误恢复、后向错误恢复等技术。如图1.1所示，软件的多个层面均可以实现容错功能，本小节列举一些在软件层次实现错误检测和错误恢复的例子以及一些跟容错相关的研究。

有的容错方法是针对控制流的，比如CCA[50]；有些容错方法是专门针对数据流的，比如EDDI[26]；还有一些容错方法是兼顾数据流和控制流的，比如SWIFT[27]和TSOtool[51]。

编译器层次

在[52]中，作者提出了一种通用的重复执行指令的方法。该方法在编译程序时，所有指令都使用不同的寄存器进行重复执行，比如指令$r_a = r_b + r_c$，会被编译器转换为由$r_{a1} = r_{b1} + r_{c1}$，$r_{a2} = r_{b2} + r_{c2}$以及一些输入复制和输出比较组成的指令序列。EDDI对该方案进行了改进，EDDI只在数据被写回至内存时才进行输出比较。SWIFT对EDDI做了进一步扩展，SWIFT将内存排除在SoR之外，因此提升了错误检测的效率。SWIFT适用于内存有保护机制的系统。TSOtool的目标是检测多核处理器共享内存中的错误行为。

应用程序层次

在应用程序层面可以获得更多的信息，因此在应用层面可以更高效地进行容错。但是，在应用层面进行容错对编程人员要求较高，而且应用层面进行容错往往实现不了很高的错误覆盖率。基于算法的容错（Algorithmic Based Fault Tolerance，简称ABFT）[53,54]可以根据具体应用，有针对性的设计专门的算法实现高效的容错。同时，由于算法的特殊性，这种方式无法适用于大多数应用程序。比如，一些研究专门针对矩阵乘法[55,56]、QR 分解[56]、FFT[57]以及多重网格方法[58]进行了算法容错设计。还有一类通过应用程序层面实现容错的方式是程序员在编程时通过添加断言和不变量实现[59–61]。

Java虚拟机层次

即使Java虚拟机提供了很多错误处理机制，比如异常处理、类型检查、数组越界等，但是还是有很大一部分比例的错误（45.03%）是Java虚拟机的机制所不能处理的[62]。因此，很多在Java虚拟机层面提升系统的可靠性的研究被提出。在Java虚拟机层面提高可靠性可以比编译器或者操作系统层面获得更多的信息，同时，对应用程序开发者是透明的，不会给编程人员带来负担[16–22,63–65]。

软硬结合

如前文所述，一个容错系统若要达到很好的错误覆盖率，要么会给系统带来较大的硬件成本，要么会给系统带来较大的性能代价。软硬结合的方式能够结合二者的优点，使用少量的硬件支持，在保证性能的前提下，给系统提供较高的错误覆盖率。这类方法具有比较好的弹性，可以动态地在可靠性和性能之间调节以满足应用程序的最佳需求。Relax[66]、FaulTM[67]、CRAFT[29]、Argus[68] 是这方面的代表。

Relax使用一个低延迟的硬件来进行错误检测，使用软件配合处理状态的保存和恢复。程序员可以标记代码块，编译器负责对标记的代码块进行处理，硬件通过调节频率和电压的方式控制被标记的代码块的执行，从而实现有针对性的容错。FaulTM借助事务内存来实现容错。CRAFT使用很少的硬件支持实现指令复制和检测的功能。Argus使用软硬结合的方式进行控制预测。在运行过程中，Argus的硬件模块产生动态的控制和数据流图，然后跟编译器预先生成的流图进行比较。

错误分析

在[69]中，作者建立了一个错误在软件中传播的概率模型。在[70]中，作者分析了位翻转和浮点算数运算之间的关系。在[71]中，作者计算了对于一个特定的应用程序，软错误对该程序产生的各种结果的概率。这些结果包括非法指令、系统异常以及未定义错误。 [72]和[73] 以不同的视角分析了一个程序对于瞬态故障的敏感程度。在[74]中，作者提出了一个称为EPIC的框架，能够对软件中的数据错误进行动态监测。软错误注入也是容错技术领域的一个研究点，一些软错误注入工具和软错误注入技术有FIAT[75]、FERRARI[76]、FINE[77]和Xception[78]。

2.5 本章小结

本章首先介绍了容错技术的基本概念，在此基础上介绍了容错技术的主要方法和主要思想，最后列举并说明了一些运用这些容错方法的硬件实现和软件实现。

第三章　Java虚拟机研究平台

3.1　引言

Java虚拟机位于上层应用程序与下层操作系统和硬件之间，提供了一个良好的实现容错技术的平台。除了具有软件实现容错技术的优点以外，在Java虚拟机层面实现容错，既可以无缝使用已有的Java程序，对程序员实现透明，又可以不修改底层操作系统。而且相对于操作系统和静态编译器，Java虚拟机可以在运行时获得更多的程序运行信息，实现更有针对性的容错。

龙芯处理器是国产处理器的优秀代表，由于指令集兼容的问题，无法使用大量已有的优秀软件。另一方面，龙芯处理器因其应用特点，对可靠性、安全性以及可用性均要求较高。因此，在龙芯平台上实现一个高可靠的、高性能的Java 虚拟机尤为重要。本章以现有Java虚拟机项目为出发点，结合龙芯平台的特点，介绍本文研究所使用的软硬件环境。

本章的结构如下：本章第二节首先介绍已有的Java虚拟机以及各自的技术特点；第三节介绍龙芯处理器平台；第四节介绍了基于龙芯处理器我们实现的Java虚拟机，本文提出的针对Java虚拟机的容错方法和优化方法有一部分是在此平台上进行的。

3.2　有代表性的Java虚拟机项目

本小节将介绍目前存在的一些比较有代表性的Java虚拟机项目。这些项目各有特点，本节将结合第一章介绍的Java虚拟机的基本概念和关键技术对这些项目进行说明和阐述。

3.2.1　HotSpot VM

HotSpot是最主流的Java虚拟机，最初由Sun公司开发并在1999年发行。目前HotSpot虚拟机由Oracle公司负责维护和运营。HotSpot实现了Java虚拟机标准，包含Java运行时库。作为一个Java字节码的执行引擎，它支持大多数操作系统和体系结构，提供一系列运行时支持，包括线程同步和对象同步等。它可以自适应地将Java字节码编译成优化的本地代码。HotSpot的垃圾收集兼顾延迟和吞吐率。它还提供了一系列调优、监控和调试的工具。HotSpot提供丰富的命令行参数，可以对Java虚拟机内部的参数进行设置，以及在运行时输出虚拟机的运行状态。HotSpot的编译器、堆结构和垃圾收集的许多特性也均可以通过命令行参数进行设置。这给用户提供了强大的调试和调优的功能。同时，这也需要用户对Java虚拟机的原理有比较深入的理解，否则很可能适得其反。

HotSpot的特点是它的JIT编译器以及自适应编译技术。Hotspot的JIT编译器分为Client编译器和Server编译器两种，其结构如图3.1所示。

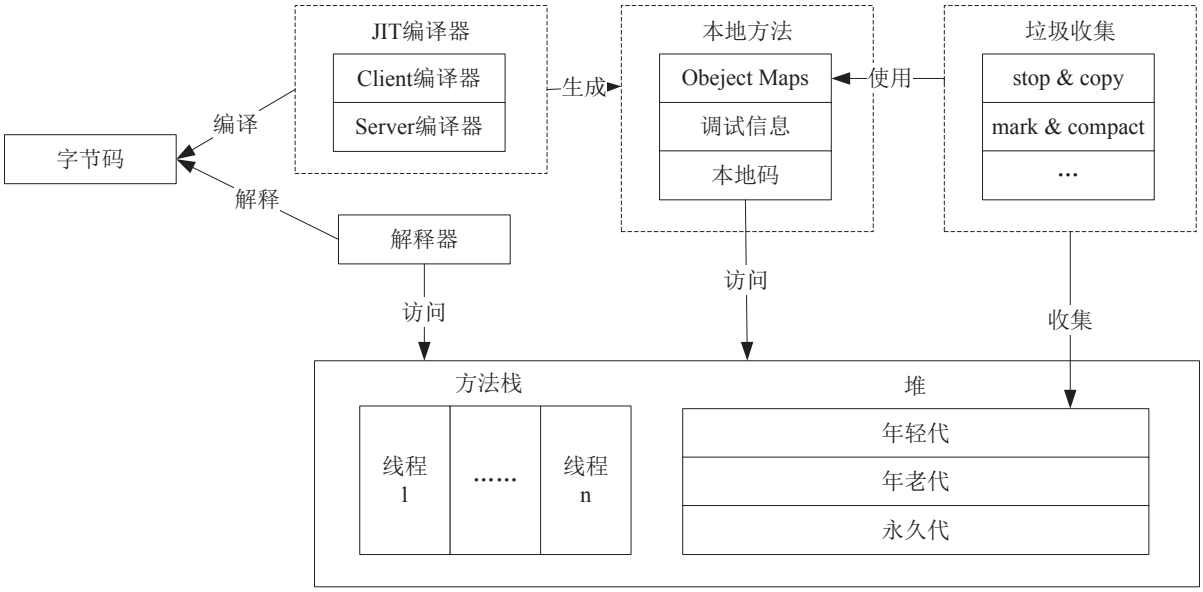


图 3.1　HotSpot结构图[2]

Client编译器适用于交互需求较高的桌面程序。这类程序对快速启动和低延迟的要求更高，对峰值性能相对没有那么高的需求。Client编译器是一个简单快速的、分三步编译的编译器，其编译过程如图3.2所示。第一步，平台无关的前端将字节码构建为一种高级的中间表示HIR（High-Level Intermediate Representation），为了便于优化，HIR使用SSA（Static Single Assignment）来表示；在第二个阶段，平台相关的后端会从HIR产生更低级的中间表示形式LIR（Low-Level Intermediate Representation）；最后一步是进行寄存器分配、窥孔优化并产生机器码。Client 版编译器侧重于局部代码的质量，很少做全局的优化，以避免增加编译方法的代价。

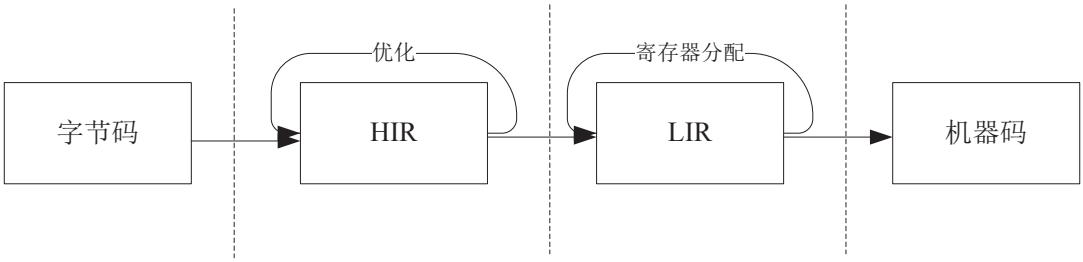


图 3.2　HotSpot client编译器的编译过程

而Sever版编译器则相反，需要的是峰值性能，对启动速度要求不高。Server 版编译器对Java字节码进行全面的优化。它使用基于SSA的中间表示形式IR进行优化。大部分经典的优化方法均会在Server编译器中使用，包括无用代码删除（Dead Code

Elimination），循环不变量外提（Loop Invariant Hoisting），公用子表达式去除（Common Subexpression Elimination），常量传播（Constant Propagation），值编号（Global Value Numbering）以及Global Code Motion。Server编译器还会使用一些针对Java的优化技术，比如空值检查消除（Null Check Elimination），范围检查消除（Range Check Elimination）以及异常处理路径优化。Server编译器是高度可移植的，因为它的实现主要基于一个硬件描述文件，这个文件可以描述目标硬件的各个方面。

HotSpot还支持一系列既可以应用于传统的编程方法也适用于面向对象的编程方法的优化技术。这些优化技术包括内联(Deep Inlining)和潜在虚调用内联（Inlining Of Potentially Virtual Calls）、快速instanceof/checkcast、循环展开（Loop Unrolling）以及反馈式编译优化（Feedback-Directed Optimization）。

OpenJDK[2,15]是Oracle公司为Java平台构建的Java开发环境的开源版本，它的核心部件就是HotSpot VM，另外还有Java运行时库和Java编译器（javac）。目前最新的OpenJDK版本为OpenJDK 9。

3.2.2 JRockit

JRockit最初是由Appeal Virtual Machines开发的，2002年被BEA Systems收购，2008年成为Oracle融合中间件的一部分。Oracle正逐步将JRockit 的代码和OpenJDK的代码整合起来，目标是在发布JDK 8时发布集成的代码库。 JRockit针对Intel平台的企业级应用做了深入的优化，在多平台之间可以实现无缝互操作（Seamless Interoperability）。JRockit捆绑了一套称为JRockit Mission Control的工具，可以提供Java虚拟机诊断功能。JRockit Mission Control包括：

- Management Console，交互式管理控制台，该工具可以监视和管理多个JRockit实例，可以捕获并显示垃圾收集停顿、内存和CPU使用等实时数据，以及JMX XBean的运行信息。
- JRockit Runtime Analyzer (JRA)，运行时性能分析工具。JRA可以详细的记录Java虚拟机和Java程序的运行信息，这些信息可以被储存下来，通过JRA Mission Control插件进行线下处理。统计信息包括方法和锁的调优信息、垃圾收集数据、优化决策、对象数据以及延迟事件。
- Memory Leak Detector，通过这个工具可以找到内存泄露的原因。该工具可以发现内存中非常缓慢的泄露过程，它会显示堆的详细数据，比如每个类型的实例占有多大的空间、堆中的碎片信息、堆中有多少个实例、以及这些实例占有空间每秒增长的速度。
- Latency Analyzer，延时分析器，JRockit R27.3之后增加。可以可视化同步、文件和网络的I/O、内存分配和垃圾回收造成的线程停止的统计数据。

3.2.3 J9 VM

J9是一款由IBM开发的Java虚拟机，它最初是由IBM Ottawa实验室的一个SmallTalk的虚拟机扩展得来。当时，这个虚拟机有一个因8K值定义问题所导致的错误，花了很长时间修正了这个错误之后，这个虚拟机就被称为8K了。后来，在此虚拟机基础上开发的Java虚拟机就被称为了J9。J9的主要组成模块包括（图3.3）：

- JVM API：外部应用同J9交互和调用的接口；
- Diagnostic：提供Java虚拟机可靠性、可用性和可服务性方面的管理；
- Memory Management：内存管理单元；
- Class Loader：类加载器；
- Interpreter：解释器；
- Platform Port Layer：平台支持，使J9可以运行在多达12个平台之上。[79]

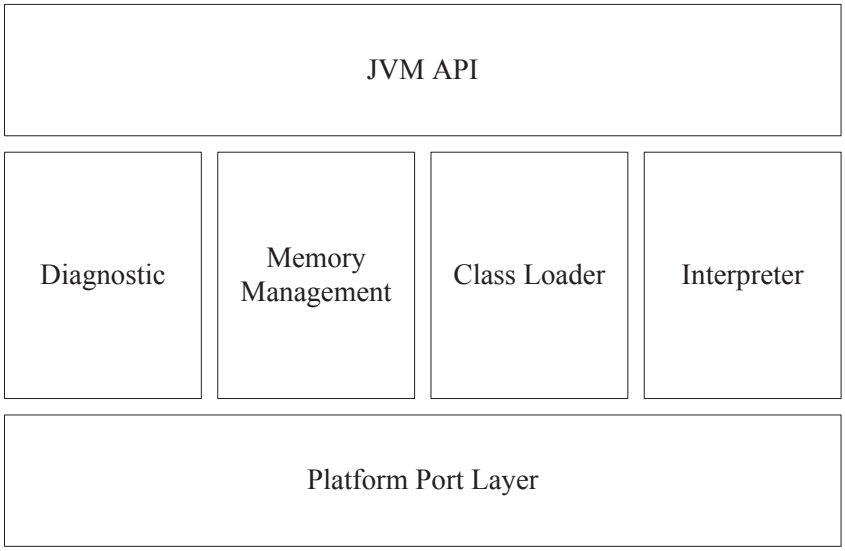


图 3.3 J9 VM

J9虚拟机同HotSpot虚拟机有一些不同。比如，它们使用不同的JIT编译器和垃圾收集器。J9使用的编译器叫做AOT Compiler（Ahead Of Time Compiler），它们使用不同的堆结构，J9虚拟机没有PermGen空间。

HotSpot、JRockit和J9是目前三大最主流的Java虚拟机，其中HotSpot占主导地位。他们的设计思想大体相同，在一些具体实现或者提供的工具上有所区别。接下来简要介绍一下三者的垃圾收集的区别。

HotSpot使用一种分代的思想，对象首先会被分配至年轻代，即Eden区。当Eden区满了，会触发年轻代的垃圾收集过程。年轻代的垃圾收集会将活的对象放入空的Survivor区，同时会将另一个Survivor区的活的对象也移至空的Survivor区。垃圾收集之后，对象只有可能存在一个Survivor区中，另一个Survivor区和Eden区都是空的。在Survivor区中经历过多次垃圾收集的对象会被移至年老代Tenured。

JRockit也是使用了一种分代的思想，在堆空间的设计上它与HotSpot有所不同。年轻代被称为Nursery，包括两块区域。新分配的对象会被放在Keap Area。JRockit没有Perm区，即永久代。

IBM的Java虚拟机的垃圾收集过程也是使用分代的思想。Allocate和Survivor两个区的作用同HotSpot中Eden和Survivor区的作用类似，新的对象会被分配至一个区中，然后经过垃圾收集以后被移动到另一个区。

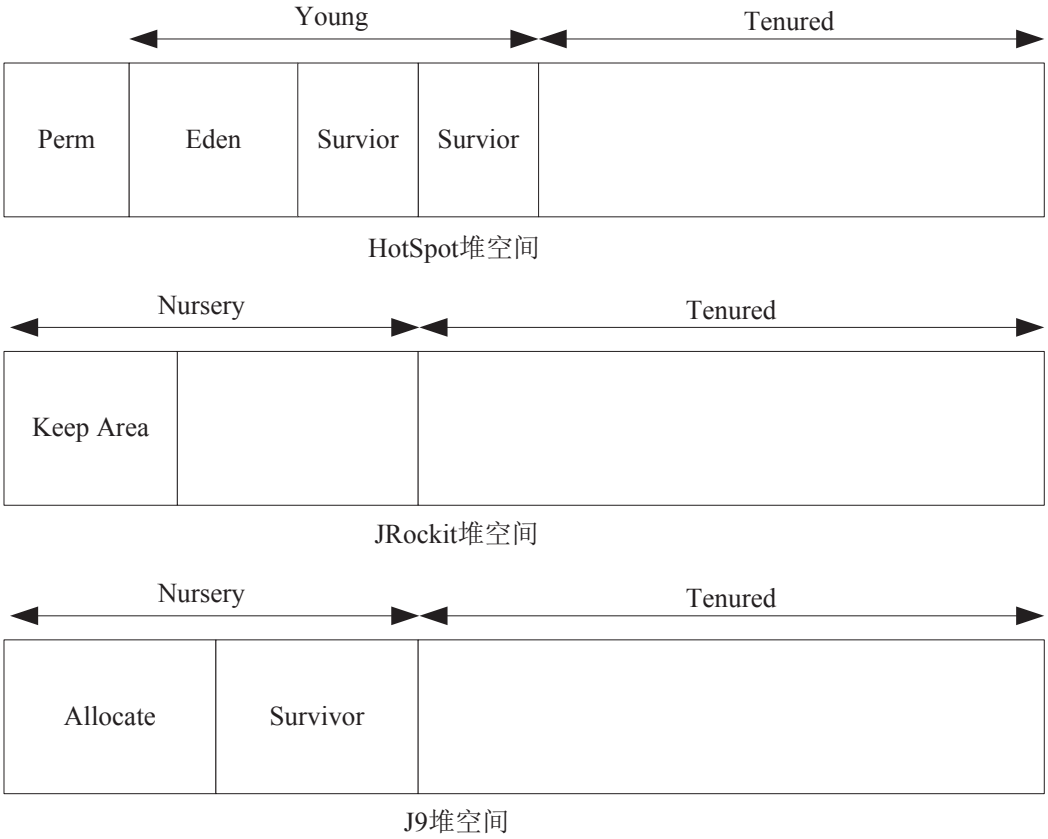


图 3.4 三大主流Java虚拟机堆空间布局对比

3.2.4 Zing VM

Zing VM是由Azul公司基于HotSpot开发的一款Java虚拟机。Azul公司重新开发了垃圾收集器，也对虚拟机进行了一些修改。在需要低延迟、快速预热的场景下，Zing VM具有一定优势。

大部分垃圾收集器的缺点是会导致长时间程序暂停。这是因为垃圾收集会对堆空间进行压缩和清理，此时需停止应用程序的执行，以防止产生对堆空间的改变。很多技术能够缓解这一问题，但是始终无法避免程序暂停。Zing的垃圾收集器C4（Continuously Concurrent Compacting Collector）基于Azul Pauseless GC Algorithm，实现了无停顿的垃圾收集。C4垃圾收集的不同之处在于它支持一种不同的代之间并发地进行垃圾收集的方式。C4支持大内存，可以支持高达670GB的堆空间[80]。C4还方便调

优，只需设置简单的参数，C4便可提供高效的垃圾收集。HotSpot等虚拟机的垃圾收集提供了大量的调优参数供用户使用，这对没有经验的使用者来说很难迅速上手实现对垃圾收集的调优。

Zing的ReadyNow!功能可以利用收集的调优数据，引导Java虚拟机启动后快速进入高性能运行状态，减少解释执行到编译执行的时间，提高启动速度。Zing还提供ZVision和ZVRobot的工具，方便用户监控虚拟机的运行状态，包括热点代码、对象分配监控、锁竞争监控等。

3.2.5 研究型Java虚拟机

Jikes RVM（Research Virtual Machine）[81–83]是这类虚拟机的代表，它提供了一个灵活的测试平台，可以用来建立新的虚拟机技术的原型和试验各种各样的设计方案。Jikes RVM支持多平台，它包括动态编译、自适应优化、垃圾收集、线程调度和同步等最新的虚拟机技术。Jikes RVM最突出的一个特点是它是由Java语言开发的，是一种self-hosted 的虚拟机。也就是说，Java 虚拟机运行在自身之上，不需要其他虚拟机对其进行支持。大部分Java虚拟机都是由C/C++编写的。由Java语言实现的Java虚拟机易于移植，使Java虚拟机和Java程序的资源之间实现无缝结合，比如对象、线程和操作系统接口。正如其名，Jikes RVM是一个研究项目，有许多研究和上百篇论文是基于Jikes RVM进行的。

Maxine Research VM[84]，是一款由Oracle实验室开发的全部由Java语言编写的Java虚拟机。同Jikes RMV一样，它也是一种采用了元循环技术（Metacircular Java VMs）的虚拟机。与Jikes RMV不同，Maxine没有使用第三方库，而是集成了标准的JDK。Jikes RVM需要底层工具进行调试，比如gdb。而Maxine提供了专门的工具可以从更高的层次对Java虚拟机和Java程序进行调试。

OVM[85]是一款应用于研究实时Java虚拟机的项目。该Java虚拟机一部分由Java实现一部分由C实现，它提供一种代码转换工具可以将Java代码转换为C/C++代码。OVM不支持JNI，同标准Java虚拟机兼容，可以执行大量的Java程序。

3.2.6 Java processor

Java处理器（Java processor）指的是通过硬件实现Java虚拟机，也就是Java字节码就是Java处理器所能直接运行的指令集。picoJava[86,87]是一款不需要解释器和编译器就可以直接执行Java程序的处理器标准。它的目标是使同样的Java程序运行在picoJava上的性能是运行在Intel处理器上的Java虚拟机的20倍。基于picoJava的处理器也可以执行C/C++程序。picoJava 标准不包含内存和I/O接口设计，因此开发者可以根据自己的需要添加内存和I/O。这一标准从未在产品中实现过。[88] 通过FPGA实现了picoJava标准的一个开源版本。

其他的一些Java处理器包括：Cjip[89,90]、aJile's JEMCore [91,92]、LavaCOR[93]、Komodo[94]、FemtoJava[95]、JOP [96,97]、jHISC[98]等。

3.3 龙芯高性能处理器

3.3.1 GS464处理器核

GS464是一款通用的处理器核，最初由龙芯2F[99]芯片设计。近年来，多款龙芯高性能处理器均使用了GS464的架构，包括龙芯3A[100,101]，龙芯3B[102–104]等。该处理器核使用64位数据宽度，支持4发射和乱序发射、动态流水线技术，并支持多核一致性扩展。

为了支持x86仿真，GS464芯片在硬件上提供对x86指令到MIPS指令的二进制翻译的支持[100]。因为x86和龙芯在体系结构上的差别（EFlags、浮点寄存器栈、地址模式等），完全从软件层面很难实现高效的二进制翻译。GS464使用最少的硬件大幅度提高了从x86到龙芯处理器的二进制翻译性能。通过对MIPS指令系统的UDI（User Defined Interface）进行扩充，增加MIPS中不具备但是x86中具备的核心指令和功能，实现优化二进制翻译的目的。具体来说，GS464增加了一些硬件机制：

1. x86标志位模拟指令；
2. x86浮点栈模拟指令；
3. x86多媒体指令（MMX/SSE）；
4. 双基址寄存器寻址模式；
5. 有界访存；
6. 改进的定点乘除指令；
7. 位提取和插入指令；
8. 数据Cache和指令Cache的自动同步；
9. CAM表（Content-Associated Memory）。

3.3.2 GS464E处理器核

龙芯公司于2012年开始研发GS464E处理器核，该处理器核的设计采用多项创新技术。使用GS464E的龙芯3A2000四核处理器已于2015年研制成功。GS464E 的流片采用40nm CMOS工艺，在该工艺下，每个处理器核的大小为4.30mm×5.22mm，面积约22.72mm^2。每个共享Cache（Shared Cache，SCache）的大小为3.00mm×4.29mm，面积约12.86mm^2。

GS474E的处理器核可用于搭建一个共享SCache的一致性多核芯片，使用基于目录的MESI缓存一致性协议。在一个芯片中，最多可支持集成32个处理器核和32个SCache。相比于GS464，GS464E在取址部件、寄存器重命名、指令重定序、定点部件、浮点部件、访存部件、缓存失效队列以及Victim Cache等方面都有大幅度的改进。

GS464E处理器核架构的主要特点有：

- 重点提升了单线程的执行性能；
- 新的片上共享 Cache设计；
- 定义了 LoongISA 指令集结构；
- 采用 AXI 接口，GS464兼容。

3.3.3 四核龙芯3号处理器

四核龙芯3号处理器是龙芯3号多核处理器系列的第一款产品，最高工作主频为1GHz，采用65nm工艺制造。四核龙芯3号处理器的主要技术特征如下：

- 片内集成4个64位的四发射超标量GS464高性能处理器核；
- 片内集成4MB的分体共享二级Cache；
- 片内集成32位100MHz PCIX/66MHz PCI；
- 片内集成2个64位400MHz的DDR2/3控制器；
- 片内集成2个16位800MHz的HyperTransport控制器；
- 每个16位的HT端口拆分成两个8路的HT端口使用;
- 通过目录协议维护多核及I/O DMA访问的Cache一致性；
- 片内集成1个LPC、2个UART、1个SPI、16路GPIO接口。

3.4 龙芯Java虚拟机

龙芯公司从2011年开始开发MIPS版本的OpenJDK。目前已经成功将OpenJDK 6、OpenJDK 8 移植至龙芯平台，包括解释器、Client 编译器和Server 编译器，实现了功能与x86版本的OpenJDK高度兼用。并且先后完成了32位版本和64位版本的移植。这些版本的Java虚拟机被广泛应用于龙芯的各个产品中。

龙芯Java虚拟机除了实现了OpenJDK中原有的大部分优化方法以外，还提出了一些新的优化方法，修改了OpenJDK官方版本超过10%的代码。这些优化方法包括高复杂度即时优化算法（寄存器分配和指令流水线优化）、双路板NUMA负载均衡优化、Java线程指针访问优化、64位Java虚拟机的压缩对象指针优化、JNI调用优化、加载立即数优化、memcpy优化、Safepoint 优化、2D 图像渲染优化、获取当前线程优化等。

针对龙芯平台自身的特点，龙芯Java虚拟机还提出了一些有针对性的优化措施：

- 类装载器虚方法调用的优化，利用龙芯处理器的硬件CAM查找表，实现了一种软硬件协同混合虚方法调用机制[105]；
- 运行时环境的访存优化，利用龙芯处理器的有界访存和128位访存指令，对Java虚拟机的访存过程实现了优化，有效地降低了数组越界检查的开销，提高了内存拷贝的效率；

- 执行引擎编译器系统的优化，利用龙芯处理的硬件Cache锁机制，对Java虚拟机的即时编译过程进行了优化[106]。

3.5 本章小结

本章列举并介绍了目前工业界主流的Java虚拟机项目，以及一些在学术界比较有代表性的Java虚拟机。接下来本章介绍了龙芯处理器以及龙芯Java虚拟机。本章的内容是后续章节的基础与平台。

第四章　针对控制流的Java虚拟机容错

4.1　引言

在计算机系统中，瞬态故障通过改变存储单元中的值或者电子器件内部的传输值等方式影响系统的正确执行。研究表明，软错误作为瞬态故障的表现形式，是造成计算机系统失效的主要原因。如果软错误发生在程序计数器中、跳转指令处或是地址计算单元，则可能导致指令的执行序列被打乱从而造成控制流错误，比如造成程序死循环、系统崩溃等。更严重的错误是产生SDC，即程序能够执行结束，却产生了错误的执行结果。相对于死循环或程序崩溃这种错误，SDC错误难以被发现，造成的损害也是难以预测的。实验表明，瞬态故障总数的22%-77%来自于控制流错误。

基于控制流的错误检测和错误恢复是一种提升系统可靠性的有效技术。自从上个世纪80年代以来，学者们提出了很多技术方案用于克服这类问题，这些方法可以划分为基于硬件的方法和基于软件的方法两大类。基于硬件的方案一般具有较好的错误覆盖率和较低的性能损失。但是使用冗余硬件或者使用专门硬件模块（比如ECC或者看门狗）进行容错的方法会增加系统的成本、功耗以及体积，这对于很多计算机系统，尤其是嵌入式系统是无法采用的。基于软件的容错方法不需要额外的硬件成本，可以通过纯软件的方式有效地提升系统的可靠性。但是大部分基于软件的容错方法会包含大量的冗余执行、内存访问和比较操作，这会给系统带来较大的性能开销和内存开销。因此，基于软件的方法需要面临的一个问题是如何在错误覆盖率、性能代价和内存代价中进行取舍和优化，以求实现一个低代价的高可靠系统。

本章提出了一种新颖的称为Java虚拟机热再生的技术，这是一种主动性的、预防性的Java虚拟机容错技术。该技术通过软件再生这种纯软件的方式，在Java程序运行的过程中，消除存在于Java虚拟机内存空间中潜在但还没有暴露出的故障，从而降低软错误出现的概率。该技术在获得较好错误覆盖率的同时，可以保证Java虚拟机的性能和延迟基本不受影响。我们基于OpenJDK7实现这种Java虚拟机热再生技术并进行了测试和分析，实验结果显示该方法在影响Java程序性能不到1%的情况下增加Java虚拟机可靠性多至48%。

本章的贡献可以归纳为以下几点：

1. 提出了一种新颖的、主动性的错误恢复方法，可以实现对Java虚拟机中多位软错误恢复；
2. 该方法能够实现并行执行，从而提升Java程序的响应时间、提高再生过程的吞吐率；该方法能够一定程度实现同Java程序的并发执行，从而进一步提升Java程序

的响应时间；

3. 该方法同其他容错方法并不冲突，可以作为一种预防性的容错方法同其他方法同时使用。

本章接下来的篇幅构造如下：第二节介绍了相关工作；第三节介绍软件再生的相关背景以及Java虚拟机进行软件再生的问题；第四节介绍了快速再生Java虚拟机的体系结构以及热再生方法；第五节介绍实验并对实验结果进行分析；最后对本章进行总结。

4.2 相关工作

4.2.1 针对控制流的硬件容错技术

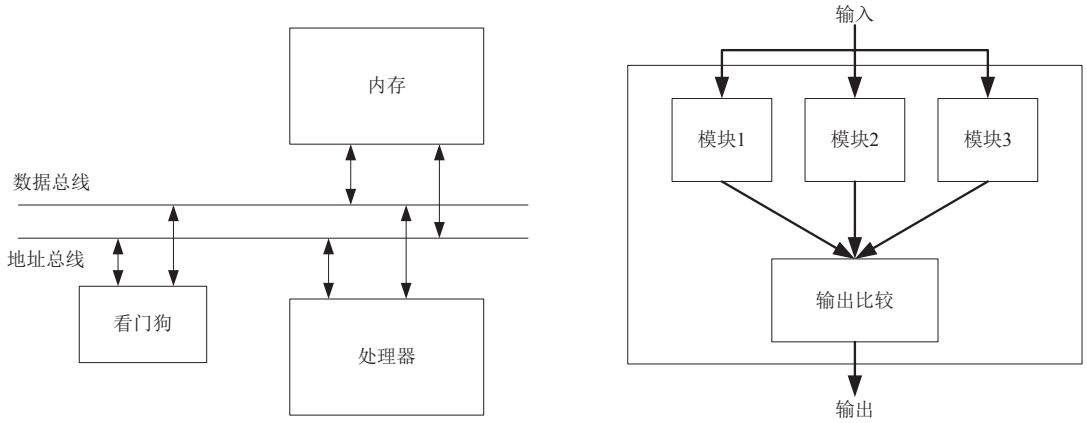
一种经典的基于硬件的容错方法是通过专门的硬件检测电路来实现，比如采用看门狗（Watchdog Processor）[60]。看门狗的方式既可以通过与主处理器并发地执行来检测控制流错误（主动方式），也可以通过使用看门狗处理器进行标签计算和比较的方式来检测控制流错误（被动方式），如图4.1(a)所示。主动方式的代价很大，因为看门狗处理器相当于一个真正的处理器[107]；被动的方式中，看门狗可以由简单的处理器实现，但是被动方式需要大量的访存操作用来存储代码块的标签。另外，被动方式还需要在程序中插入一些代码来实现错误检测功能。看门狗的方式大多用来检测控制流错误，对于数据错误，这种方式只能实现有限的检测。

另一种常用的硬件方法是通过使用重复的硬件电路来实现错误检测和错误纠正，比如TMR[108]。TMR系统同时执行程序的三个拷贝，如图4.1(b)所示。如果输出比较模块检测出三个拷贝的输出是一样的，那么没有错误发生。如果输出比较模块检测出三个拷贝中的两个是一样的，那么TMR系统认为不一样的那个拷贝发生了错误。一般来讲，TMR系统会舍弃发生错误的拷贝，并使系统降级为DMR模式。为了恢复为TMR模式，此时系统有几个选择，一个是记录下相关日志文件并通知技术人员到达现场进行调试和重启。另一个方案是系统尝试修复错误并恢复至TMR模式。该方案选择在一个合适的时间停止整个程序的执行，并将正确的拷贝覆盖至错误的拷贝，然后再继续整个程序的执行。这种方案会增加系统的停机时间并有可能减少系统的可用性。

4.2.2 针对控制流的软件容错技术

现有的针对控制流错误的软件方法大多可以分为Checker和RMT冗余多线程两大类。

第一种方式大多是采用标签检测(Signature Checking或者Signature Monitoring)的办法，这类方法同使用看门狗技术的原理类似，它们的区别是软件实现的容错技术是使用处理器进行控制流检测，而非看门狗。程序中的代码块会对应一个标签，这些标签在指令的执行过程中被创建。在执行过程中，这些标签还会同一些已经预先生成的标



(a) 看门狗　　(b) TMR

图 4.1　看门狗和TMR方式

签进行比较，从而判断程序的控制流是否正确。这些预先生成的标签可能是程序执行之前就生成的，也可能是在没有错误的情况下执行生成的。标签的判断可以在各个层面进行，比如程序语言层面、编译器层面或者二进制翻译层面。具有代表性的标签检测方法是斯坦福大学CRC实验室提出的CFCSS（Control Flow Checking by Software Signatures）[109]。该文提出了一种纯软件实现的标签分析方法。该算法为程序的每个基本块分配一个唯一的标签并在基本块首部插入少量错误检测指令。CFCSS在编译阶段将标签插入到程序当中，在程序的运行时期对标签进行比较。该方法是标签检测的典型算法，不需要增加额外的硬件部件，也不需要操作系统支持多线程，就可以实现控制流错误检测。

另一类软件实现的容错方法基于RMT冗余多线程思想。这类容错方案在指令级、高级语言级、线程级或者进程级等不同层次进行冗余计算从而达到错误检测和错误恢复的目的。

在指令集冗余层面，在[26]文章中提出的EDDI（Error Detection by Duplicated Instructions）是这方面的代表。EDDI 在编译程序时产生冗余的指令，可以在一个线程实现两个执行流。EDDI使用冗余的寄存器保存同一个变量，并且在相应的位置插入比较指令用于检测是否存在软错误。这种方式可以对寄存器和内存同时容错。图4.2显示了一个EDDI的实例。通过实验评测，在一个双发射的计算机上，该方法给程序带来的额外性能代价介于36%和111%之间。

```
ld r12=[GLOBAL]

add r11=r12,r13



st m[r11]=r12
```

(a) 原始代码

```
ld r12=[GLOBAL]
ld r22=[GLOBAL+offset]
add r11=r12,r13
add r21=r22,r23
cmp.neq.unc p1,p0=r11,r21
cmp.neq.or p1,p0=r12,r22
(p1) br faultDetected
st m[r11]=r12
st m[r21+offset]=r22
```

(b) EDDI代码

图 4.2　EDDI指令对比

SWIFT（Software-Implemented Fault Tolerance）[27]与EDDI在很多方面类似。SWIFT也是通过一个单线程对执行流实现冗余执行。同EDDI 一样，SWIFT使用两个寄存器版本保存同一个数据，因此SWIFT 实现了寄存器层面的容错。区别在于，SWIFT假定内存是由相关机制保护的，比如ECC，因此是可靠的。该方法将内存子系统排除在了SoR之外，从而实现了更好的性能。

EDDI和SWIFT都是基于编译器层面的，因此需要程序的源代码并且重新编译程序。Spot[28]则在二进制翻译的层面实现了指令冗余，Spot方法基于SWIFT，在程序运行时动态的复制除写入内存以外的所有指令，因此Spot可以不需要程序的源代码，适用于库和动态生成代码等。

一些论文提出了在Java虚拟机层面进行错误检测和错误恢复。Chen在[16]中研究了内存错误尤其是Java虚拟机堆空间中的内存错误对Java 虚拟机和Java程序的影响，并且实现了一个检测内存错误的原型。该系统可以通过校验码的方式对堆中的对象进行校验。Friedman[18]和Napper[19]使用Replication的方式实现在Java虚拟机层面对分布式系统进行容错。Chen[20]使用一种双冗余执行和检查点的方式进行软错误的检测和恢复。除了内存保护机制以外，Chen的方法不需要任何硬件支持。因此该方法适用于资源比较紧张的嵌入式系统。Stilkerich[22]提出了一种针对类型系统（Type System）的容错方案。Java 语言是一种强类型语言，提供了很多类型保护机制，基于Java的这一特点，该文方法进行了进一步地扩展。该文主要思想为使用LRC （Load Reference Check）对类型系统进行保护，又提出了许多进一步的优化方法。该文主要针对嵌入式系统，需要假定有一块non-volitile 区以及一个定制的、静态的、不需要加载新的类和线程的Java虚拟机。

软件层面的容错方法普遍会对程序的性能带来较大的影响。另一方面，软件层面

的容错一般都采取指令冗余的方式，这种方式在消除软错误的同时也一定程度增加了软错误发生的可能性。

4.3 软件再生技术

当软件长时间运行时，会因为系统内存膨胀和泄露、未释放的文件锁、数据腐烂、存储空间碎片以及舍入误差的累积等原因导致软件性能的衰退或者故障，这种现象称为软件老化（Software Aging）[110]。Huang等在[111]中提出软件再生（Software Rejuvenation），或称软件恢复技术，用于解决软件老化的问题。软件再生技术是一种周期性的终止并重启程序的行为，使程序恢复到正常的初始状态，它可以有效地缓解软件老化。

在[110]中，作者总结并归纳了软件老化和软件再生的相关技术。软件再生技术可以从以下多个层面进行。应用程序重启是最简单的一种软件再生方式。这种方式借助于操作系统级的资源分配机制，对应用程序实现初始化，包括资源的释放、套接字的释放等。操作系统重启也是一种有效的、常用的软件再生方法，包括操作系统的重新启动以及运行于该系统之上的所有应用程序的重新启动。Kourai[112]提出了一个基于虚拟机的快速软件再生方法，该方法通过暂停和恢复虚拟机实现了高效的虚拟机重启。在[113]中，作者提出了一种方法用于评测Java系统的软件老化现象。该文指出在Java虚拟机中存在一些老化现象，包括由运行单元导致的吞吐率下降、由JIT编译行为导致的缓慢的内存泄露以及由垃圾收集器引起的快速的内存耗尽。

软件再生技术的一个重要问题是再生策略，也就是在程序运行过程如何安排与选择软件再生的时间和频度。有效的再生策略可以减少软件再生所引起的性能损失和总的停机时间（包括软件老化所导致的低效、停机、重启的时间，以及软件再生所引起的停机、重启时间）。为了解决这个问题，Huang提出了一个基于连续时间马尔科夫链的简单模型[111]。他将软件的执行过程分成若干个状态。当没有采用软件再生技术时，程序的执行模型如图4.3所示，程序最初运行在高度健壮态S_0；在软件运行较长一段时间后，由于软件老化，程序将进入衰老可能态S_P；在衰老可能态执行一段时间后，由于出现错误，程序会进入故障停机态S_F；经过重新启动，程序需要一段时间再次恢复到高度健壮态S_0。各个状态之间的平均转换速率为r_1、r_2以及λ。平均转换速率与转换周期成倒数关系，例如，程序由衰老可能态S_P平均经过两个小时进入故障停机态S_F，那么λ为$\frac{1}{2}$（以小时为单位）。

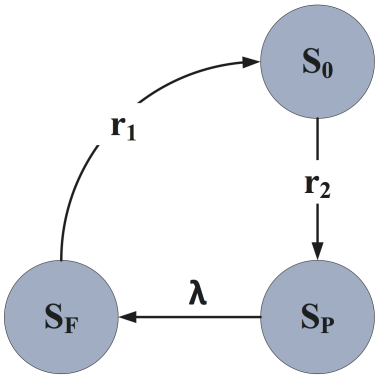


图 4.3　未使用软件再生技术的概率状态转换模型

当系统采用软件再生技术时，程序的执行状态转换如图4.4所示。程序最初运行在高度健壮态S_0；在进入衰老可能态S_P之后会出现两条路径，一条如前所述会进入故障停机态S_F，另一条则以r_4的平均再生速率进入再生态S_R；与重启一样，经过再生之后，软件以r_3的速率重新恢复到高度健壮态S_0。r_4 刻画了程序多久进行一次再生操作，r_3则刻画了一次再生操作需要的时间。

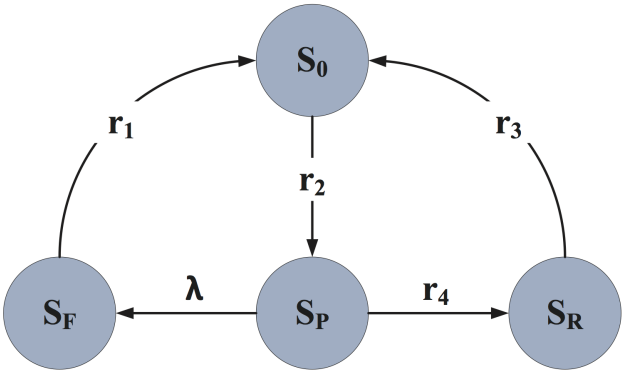


图 4.4　使用软件再生技术的概率状态转换模型

根据这个模型，通过列出各个状态间的平均转换速率和各个状态的概率分布关系的等式，可以分别得出采用和不采用软件再生技术情况下系统总的停机时间与各状态间的转换速率的函数关系。对于不采用软件再生的系统，在L时间间隔内，总的预期停机时间公式为：

$$DownTime(L) = L \times \frac{1}{1 + \frac{r_1}{\lambda} + \frac{r_1}{r_2}} \tag{4-1}$$

对于采用软件再生技术的系统，在L时间间隔内，总的预期停机时间（包括重启时

间和再生时间）的计算公式为：

$$DownTime_r(L) = L \times \frac{\frac{\lambda}{r_1} + \frac{r_4}{r_3}}{1 + \frac{\lambda}{r_1} + \frac{\lambda}{r_2} + \frac{r_4}{r_2} + \frac{r_4}{r_3}} \tag{4-2}$$

例如，一个系统平均运行2天会进入故障可能态，那么，r_2的值为$\frac{1}{2\times24}$（以小时为单位）；由故障可能态S_P变为故障停机态S_F平均需要一个小时的时间，因此λ为1；如果发生错误，系统需要30分钟的时间完成重启恢复至高度健壮态态S_P，因此r_1为2。那么，根据公式4-1计算得出L时间内，系统总的预期停机时间约为0.0101L。如果该系统采用再生技术，且一次再生平均需要2秒，则r_3为180；再生过程每隔10分钟进行一次，则r_4为6。根据公式4-2 计算得出L时间内，系统总的预期停机时间约为0.0016L。因此，在这个例子中，使用软件再生技术有助于降低总体停机时间。

为了得出软件再生速率r_4如何影响总的预期停机时间，我们对4-2求导：

$$\begin{aligned}\frac{\mathrm{d}}{\mathrm{d}r_4}DownTime_r(L) &= L \times \frac{\lambda}{r_1 r_2 r_3} \\ &\times \frac{1}{(1 + \frac{\lambda}{r_1} + \frac{\lambda}{r_2} + \frac{r_4}{r_2} + \frac{r_4}{r_3})^2} \\ &\times [r_1(1 + \frac{r_2}{\lambda}) - r_3]\end{aligned} \tag{4-3}$$

由此可见，上述函数的符号与r_4无关，也就是当r_4变化时，总体停机时间的增加或者减小只取决于$r_1(1 + \frac{r_2}{\lambda}) - r_3$的值的正负。当$r_1(1 + \frac{r_2}{\lambda})$小于$r_3$时，符号为负，意味着$r_4$增加则停机时间减少。这表示越频繁地进行再生，能够越好的降低总体停机时间。相反的，当$r_1(1 + \frac{r_2}{\lambda})$大于$r_3$时意味着频繁地进行再生会增加总体停机时间。

我们认为软错误也可视为一种软件老化现象并可以通过软件再生技术进行修复。当存在软错误时，Java虚拟机的执行也符合该模型。最初Java虚拟机工作在高度健壮态S_0；经过一段较长时间的运行，某些瞬态故障（比如数据或者指令部分的位翻转）在系统中发生，但此时程序还没有访问相应的数据或者程序，因此并不会出现错误，这个时候系统进入故障可能态S_P；随着程序的运行，在某个时刻访问了故障所在的数据或者代码，产生了程序错误，使得程序进入了故障停机态S_F；经过错误恢复或者重启等过程，程序恢复到最初的高度健壮态S_0。而本章的工作是给Java虚拟机加入再生态S_R，由故障可能态S_P进入再生态S_R；通过对Java虚拟机再生，最终恢复到高度健壮态S_0，其状态转换同图4.4相同。

接下来以一个量化的例子来说明Java虚拟机软件再生模型。假设一个Java虚拟机平均2天会发生一个软错误，那么，r_2的值为$\frac{1}{2\times24}$（以小时为单位）；访问到软错误导致停机平均需要一个小时的时间，因此λ为1；如果发生软错误，系统需要30分钟的时间完

成重启恢复至高度健壮态态S_P，因此r_1为2；如果Java虚拟机采用再生技术，那么进行一次再生平均需要2秒，因此r_3为180。带入公式计算得出$r_1(1+\frac{r_2}{\lambda})$小于$r_3$，因此对于这个系统，越频繁地进行软件再生会越有益于降低Java虚拟机总体停机时间。

但是，这是一个理想化的模型。对于实际来讲，频繁的软件再生对于Java程序性能未必总是有益的。在Java虚拟机进行再生的过程中，通常Java程序也会暂停甚至停止，这会产生一些问题。第一，软件再生会导致Java程序的停机时间。在Java虚拟机进行再生的过程中，Java程序的执行也需要停止直到虚拟机完成再生。在Java程序停止的这段时间无法提供相应的服务，这可能会影响一些实时性要求较高的程序，比如网页服务器。第二，Java 程序的性能会在Java虚拟机再生之后有所下降。由于Java程序和Java虚拟机都进行了重启，导致已经编译了的方法需要重新编译，在这些方法执行一定次数并重新编译完成之前，这些方法只能采用解释执行的方式执行，这会给Java程序的性能造成很大损失。另外，再生也会导致很多已经装载了的Class文件需要重新装载。第三，程序必须结束并且重头执行，这对于一些应用场景是无法接受的，比如科学计算或者传输系统。

4.4 Java虚拟机快速再生技术

我们认为Java虚拟机的再生过程应尽量不影响Java程序的执行。也就是说Java虚拟机的再生过程要尽可能减少Java程序的暂停时间，降低对Java程序性能的影响，尤其要避免造成Java程序的结束。基于这一思想，本章利用了Java虚拟机模块化的结构，实现了一种叫做Java虚拟机热再生的快速软件再生技术，并实现了一个快速再生Java虚拟机。

4.4.1 快速再生Java虚拟机

我们开发的快速再生Java虚拟机是以OpenJDK 7为基础的。OpenJDK是目前使用最广泛的Java虚拟机之一，本章采用的版本为jdk7u80-b03。快速再生Java虚拟机可以进行不同速率的再生操作，其结构在图4.5中给出。它是由执行单元、运行时数据区、垃圾收集器、快速再生单元等子系统组成。执行单元主要包括解释器、编译器和异常处理。解释器逐条解释执行字节码，在识别某个方法或者循环体为热点之后，编译器将相应字节码编译成本地代码以便于Java程序更快速地执行[2,114]。存储单元则主要包括方法区和堆，在Java虚拟机中，被加载的类型信息存储于方法区中。当虚拟机加载某个类型时，它定位并读取相应的Class文件，然后虚拟机提取其中的类型信息，并将这些信息组织起来存储到方法区中。类型中的类（静态）变量也存储在方法区中。Java程序在运行时创建的所有类实例和数组都放在堆中。一个Java虚拟机实例只有一个堆，因此所有线程都共享这个堆。垃圾收集器的主要工作就是自动回收不再被Java程序所引用的对象所占用的内存。此外，它也可能移动使用中的对象，以减少堆中的碎片[115,116]。快

速再生单元按照配置的参数，对Java虚拟机的各个模块进行周期性的再生操作。

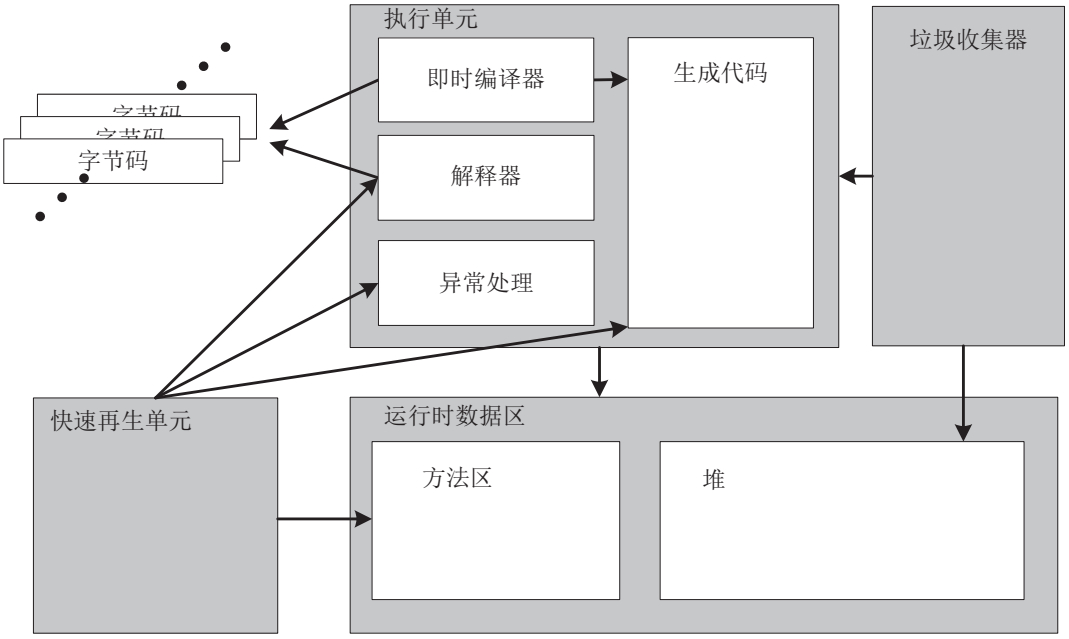


图 4.5　快速再生Java虚拟机内部结构图

4.4.2　Java虚拟机热再生

为了能够减少软件再生对Java程序的影响，我们提出了一种叫做Java虚拟机热再生的方法。这个方法的基本思想是利用Java虚拟机模块化的结构，将再生过程也分模块地进行，并且在再生过程中保存Java虚拟机的中间信息，在再生完成以后继续使用这些信息。以往的方式往往需要结束Java程序的执行，我们称之为冷再生或者冷启动。区别于以往的方式，在Java虚拟机热再生时，不需要结束Java程序，甚至有些时候再生可以与Java程序并发执行。

如前文所述，Java虚拟机中的子模块包括方法区、解释器、编译器以及异常处理。这些模块中的潜在的错误可以通过再生过程进行消除。下面分别介绍这些模块是如何实现热再生过程的。

方法区再生：　为了进行方法区的再生，我们需要维护一个表记录所有已加载的类，这个表叫做Loaded Class Table。在进行再生时，将该表中的类重新加载至方法区，在相同位置覆盖原有的数据，因此消除了原来方法区中可能隐藏的错误。在加载的过程中，Java虚拟机会重新对Class文件进行装载、链接（验证、准备、解析）、初始化。即使Class文件（一般来源于硬盘或者网络）中存在位翻转，也能在链接阶段发现错误，从而保证加载至内存中的数据是正确的。在方法区再生时，不需要一次再生操作完成所有类的加载，这样有可能会增加Java程序的停机时间。本章采用的方式是每次再生操

作重新加载10个类。我们还需要维护这张表的可靠性以防止表中出现软错误。通过添加校验码，在每次读取表数据时对表进行校验，可以发现并纠正表中的位翻转。

解释器再生： 解释器中有一个称为模板表的结构，对解释器进行再生，主要是对模板表进行初始化。模板表存储了每条Java字节码所对应的本地代码，执行时以逐条解释字节码的方式执行字节码所对应的本地代码。模板表是一种与运行状态无关、一次生成、一直有效的一张表，相当于常量。因此，对模板表再生，需重新创建模板表，并覆盖原有模板表，从而消除模板表中潜在的错误。解释器中还包含一些Java栈帧创建、参数传递、方法返回、本地方法调用等代码，这部分代码同样需要重新创建并覆盖原有代码。

生成代码再生： 我们将所有已经编译的方法全部记录在一个称为Compiled Method Table的表中。每次再生时，需要将此表中的方法重新进行编译并覆盖原来的生成代码。通过对方法区进行再生操作，我们已经消除了存储于方法区的类信息中的错误，因此保证编译后的代码也是正确的。同Loaded Class Table一样，我们需要对Compiled Method Table采用容错机制，每次在读取该表之前需要进行错误校验。在生成代码再生时，不需要一次编译所有的方法，分批编译会有助于降低Java程序的响应时间。

异常处理模块再生： 异常处理模块的再生与模板表类似，再生过程会重新生成异常处理代码并在相同的地址覆盖原有代码。通过这种方式，可以覆盖原有异常代码区中潜在的错误。使用相同的地址可以确保不需要修改其他代码，比如跳转地址。另外，使用相同的地址有利于再生线程和Java程序的并发执行。

Java虚拟机中的这些模块相对比较独立，具有良好的局部性。因此在热再生的过程中，可以分模块地进行，并不需要一次性完成所有模块的初始化。对于多核处理器，这些模块之间还可以并行地进行，这有助于减少再生时间，提升再生吞吐量。更进一步来讲，在每个模块内部同样都具有良好的局部性：方法区是以类为单位组织的，模板表是以字节码为单位组织的，生成代码是以方法为单位组织的，异常处理是以代码块为单位组织的。因此在每个模块的再生过程中，可以进一步地拆分成小的子模块进行分步骤地、并行地再生。通过模块的拆分与并行执行，可以很好的提高Java虚拟机热再生的性能，降低Java程序的延迟。

另外，由于Java虚拟机中很多模块是只读的，比如代码部分。这样，一方面可以通过减少锁操作增加Java虚拟机热再生的性能；另一方面，可以让Java虚拟机热再生线程与Java执行线程并发地执行。

该方法可以实现多位错误恢复。很多容错方法只能恢复一位或者有限位数的错误，有的容错方法甚至只能检测出有限位数错误而不能恢复它们。

该方法通过周期性地对Java虚拟机内部的模块进行再生，可以一定程度消除软错误，但是并不能够保证杜绝软错误。比如，如果在两次再生过程中间发生了位翻转并且被程序访问，则Java虚拟机可能会出现错误。另一方面，本章方法与其他容错技术并不冲突，可以配合使用。例如，可以使用[16]中的校验方法对Java虚拟机进行错误检测，根据检测的结果和错误发生的频率，对再生周期进行自适应调整。

该方法主要是针对方法区中的数据进行容错。由于堆和栈中的数据是随着Java程序的执行而不断变化的，因此无法使用该方法进行容错。如何对这部分数据进行容错将在本文第五章介绍。

4.5 实验结果和分析

在本节，我们评测了该方法对于Java虚拟机可靠性以及对于性能所产生的影响。本章实验所使用的计算机配有128核Intel(R) Xeon(R) CPU E7- 8830 @ 2.13GHz 处理器，1T内存，操作系统为Linux 2.6.32。为了对比，我们还评测了原始的OpenJDK 7。本章使用SPECjvm2008[117]作为可靠性测试以及性能测试的测试程序。

4.5.1 错误注入模块

为了模拟软错误在内存中的行为以及评测不同软错误率对Java虚拟机可靠性的影响，本章实现了一个可配置参数的软错误注入模型。该模型通过一个软错误注入线程随机地向Java虚拟机中注入软错误。该软错误注入线程在Java虚拟机启动时创建，在Java虚拟机运行一段时间后开始向Java虚拟机的指定内存区域注入位翻转。该线程通过读取配置文件按照参数设置向Java虚拟机指定内存范围注入错误，可配置的参数为内存地址范围、最多翻转位数（Maximum Faulty Bits，简称MFB）以及最大注入间隔（Maximum Injection Interval，简称MII）。内存地址范围通过设定起始地址和范围来指定，注入线程会向该区间等概率的注入错误。最多翻转位数MFB表示每次注入错误可能的最多翻转位数，比如最多翻转位数设置为5时，软错误注入线程会向目标地址随机注入1位到5位的位翻转，并且位数越多，概率越低。最大注入间隔MII的单位为秒，比如当该值设为30时，每次软错误注入线程会在0-30秒之间随机生成一个时间间隔，并在该时间到达时注入错误。

4.5.2 错误覆盖率

在本小节我们评测了本章方法对于可靠性产生的效果。首先，我们测试在没有进行再生的情况下，不同软错误率对程序产生的影响。我们使用SPECjvm2008中的compiler.compiler作为测试程序，Java虚拟机的初始内存和最大内存均设置为2G（参数-Xms和-Xmx设为2G），8线程运行（参数-bt设为8）。我们将测试分成三组，每组的最多翻转位数5位，最大注入间隔分别为120秒、60秒和20秒，每组实验测试了200次，这600次实验的统计结果在表4.1中给出。实验结果表明，随着最大注入间隔的逐步降

低，Java虚拟机的的崩溃次数逐步升高，表示虚拟机的稳定性逐步降低。当最大注入间隔为120秒时，有97个正确结果，另外一半以上的程序不能正确完成。当最大注入间隔降低到60秒时，正确完成的次数变化不大，但是程序的完成数减少，Java虚拟机的崩溃次数增加。当最大注入间隔降低至20秒时，正确完成的次数明显减少，Java虚拟机崩溃数明显增多。

表 4.1　模拟结果

最大注入间隔 (MII)	120s	60s	20s
正确结果	97	96	44
错误结果	88	79	98
崩溃	12	23	50
超时	2	1	2
不支持指令	0	1	1
其他	1	0	5

接下来我们测试本章方法对可靠性的影响。在最多翻转位数为5位、最大注入间隔为20秒的情况下，我们在Java虚拟机中加入再生机制，并测试了效果。我们分别对再生间隔设为20秒、5秒和1秒时compler.compiler的执行进行测试，并且将结果同表4.1中无再生机制时的结果进行了对比。实验结果在表4.2中给出。由表4.2可以看出，再生机制可以有效提高Java虚拟机的可靠性。在再生间隔为20 秒、5 秒和1秒时，程序正确完成的次数分别提升至54次、61次和65次，分别提高了23%，39%和48%。可见，在再生间隔为20秒时本方法就对Java虚拟机的稳定程度有了效果。在再生间隔为5秒时Java 虚拟机稳定性有了进一步提升。当再生间隔为1秒时，Java虚拟机稳定性进一步提升的幅度有所减少。从表4.2中看到，在再生间隔由5秒降低到1秒的过程中，Java虚拟机的崩溃次数有所降低，错误完成数、超时次数大体不变，这也一定程度上体现了本方法的效果。另一方面，由于我们对整个Java虚拟机的地址空间进行了软错误注入，但是我们的方法主要针对方法区实现容错，堆和栈都没有在容错范围之内，因此实验效果比较有限。

最后，我们对SPECjvm2008中所有的测试程序进行了测试。实验所使用的Java虚拟机的初始内存和最大内存均为2G，8线程执行。最大注入间隔20 秒，注入区间为Java虚拟机的方法区。我们对每个测试程序都进行了4组实验，分别为再生间隔无穷大（不使用再生）、再生间隔20秒、再生间隔5 秒以及再生间隔1秒，每组实验进行100次，即我们对每个测试程序进行了400次实验，测试结果在图4.6中给出。图中每个测试程序的四个柱状图分别表示再生间隔为无穷大、20秒、5秒和1秒时程序的执行结果。由图4.6可见，再生间隔在20秒时，已经对一些测试程序产生了效果。许多

表 4.2　再生模拟结果

再生间隔时间	∞	20s	5s	1s
正确结果	44	54	61	65
错误结果	98	108	98	100
崩溃	50	30	40	30
超时	2	3	1	2
不支持指令	1	3	0	0
其他	5	2	0	3

测试程序的正确完成数有所增加，比如startup.crypto.aes、compress、scimark.fft.large以及scimark.monte_carlo。在正确完成数没有明显提升的测试程序中，我们发现有一些测试程序总的完成数（正确结果和错误结果的和）有所增加，比如mpegaudio。在再生间隔降低至5秒时，各个测试程序的正确完成数均有较大幅度增加，正确完成率基本均在95%以上，甚至许多程序达到了100%。在再生间隔进一步降低至1秒时，可靠性的提升效果已经不明显。由于本次实验的软错误注入区间为Java虚拟机的方法区，并没有向堆和栈中注入错误，因此本实验稳定性的提升效果要好于表4.2中所体现的效果。

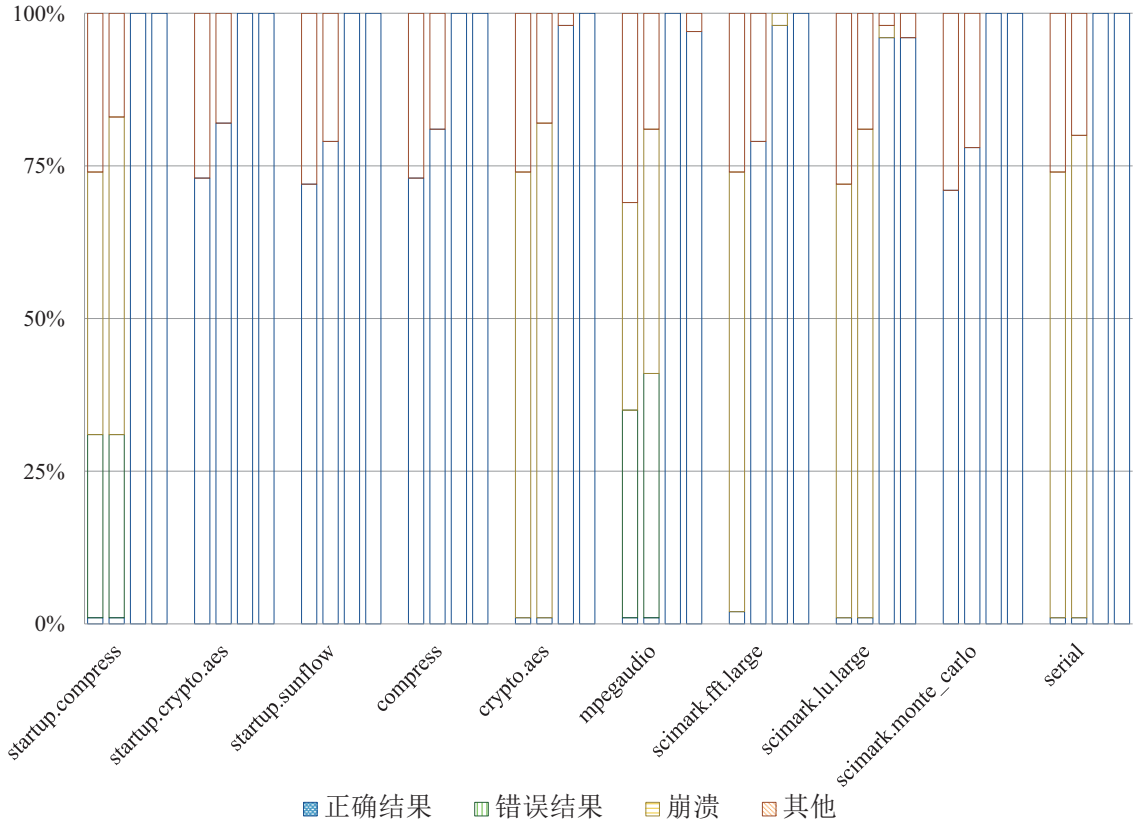


图 4.6　SPECjvm2008错误覆盖率执行结果

4.5.3 性能损失

在本小节我们评测了本方法对于Java虚拟机性能产生的影响。在实验中，我们分别对不使用再生、使用20秒再生间隔、使用5秒再生间隔和1秒再生间隔的情况进行对比，实验结果如图4.7所示。图4.7中的柱状图表示与没有使用再生时的对比。实验显示，在存在再生操作时，大部分程序的性能会有所下降，而且随着再生间隔的降低，程序性能下降幅度增大。不过，即使在对程序性能影响最大的再生间隔为1秒的情况下，程序性能的下降幅度也大部分都在1%以内。对于再生间隔为5秒时，程序性能下降的幅度基本均在0.5%以内。由上节我们知道，在再生间隔为5秒时，就已经能够对Java虚拟机的可靠性带来很大的提升。

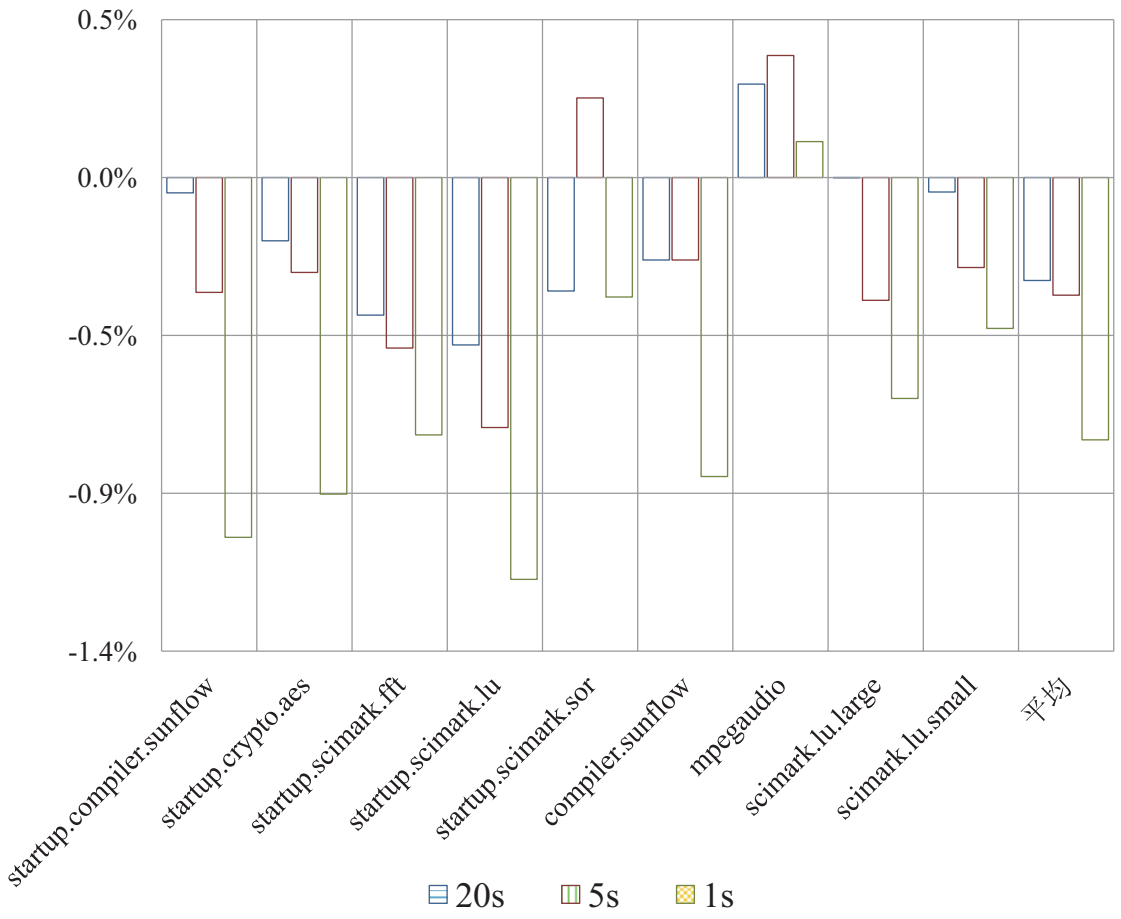


图 4.7 SPECjvm2008性能损失

4.6 本章小结

本章首先介绍了软件再生技术的背景以及Java虚拟机应用软件再生技术进行容错所面临的问题。基于这些问题，本章提出了一种新颖的称为Java虚拟机热再生的方法用于实现对Java虚拟机的快速再生从而消除存在于Java虚拟机内存中的软错误。通过该方法可以实现对Java虚拟机方法区中的任意多位的软错误进行恢复。该方法利用Java虚拟

机模块化的结构将Java虚拟机的再生过程进行划分，使得各模块之间的再生过程可以分开进行、并行进行。该方法进一步利用Java虚拟机各个模块内部良好的局部性特点，对各模块内部的再生过程进行进一步地拆分，使得各模块内部的再生过程可以分开进行、并行进行。Java虚拟机热再生方法还可以在Java程序运行的过程中实现对Java虚拟机的初始化，因此该技术还是一种并发执行方法。通过分步骤地、并行地、并发地执行，该方法可以在保证Java虚拟机的性能和延迟的情况下，提供较好的错误覆盖率。

为了评测Java虚拟机热再生方法的效果，本章实现了一个软错误注入模块，该模块可以根据配置向Java虚拟机中注入位翻转。基于OpenJDK7我们实现了再生方法并进行了错误覆盖率和性能的评测实验。实验显示，该方法能够有效提升Java虚拟机的可靠性，同时对Java虚拟机产生的性能损失不到1%。

第五章　针对数据流的Java虚拟机容错

5.1　引言

软错误不仅会对程序的控制流产生影响，还会对程序的数据流带来危害。错误的数据可能造成程序的崩溃或者造成程序进入死循环。更严重的是程序运行结束，却产生了错误的结果。这种情况难以发现，有可能会带来数据损坏、财产损失甚至安全方面的灾害。

考虑到成本因素，一些系统往往无法采用基于硬件的容错方案。而现有的软件容错方案往往又会造成很大的性能开销。因此，使用软件方式，在保证性能的前提下，实现较好的错误覆盖率是研究的一个目标。本文第四章的工作满足这个要求，但是该方法主要针对的是Java虚拟机方法区的容错。从内存体积上讲，Java虚拟机的数据区，即堆和Java栈一般会占用更大的体积，因此也会有更大的概率出现瞬态故障。然而这部分数据一般都是运行时产生的中间结果，无法使用本文第四章的方法进行周期性的再生。

针对这一问题，本章提出了一种称为CST的纯软件实现的容错方法。该方法利用动态污点分析技术的思想，通过动态追踪数据的时间信息和来源信息，对Java虚拟机中的数据的校验时间和传播过程进行自动分析，实现对数据进行有差别的校验。该方法能够在不大幅损害性能的前提下，对Java 虚拟机的数据流，即堆和Java栈中的数据进行容错。

本章的贡献可以归纳为以下几点：

1. 提出一种基于动态污点分析的容错方法CST，可以消除一些不必要的数据校验，从而提升程序的性能；
2. 提出一系列优化算法，有效提升了CST方法的性能。

本章接下来的篇幅构造如下：第二节介绍了相关工作；第三节介绍了基于Java虚拟机的污点分析方法CST；第四节介绍了CST的具体实现以及一些优化方法；第五节介绍实验并对实验结果进行分析；最后对本章进行总结。

5.2　相关工作

5.2.1　针对数据流的容错技术

在[118]中，作者提出DDFV（Dynamic Data Flow Verification）系统，该系统在编译时为每个基本块创建一个标签，标签根据寄存器的历史决定。在程序运行时，DDFV硬件计算每个基本块的标签，并把这些标签同预先的值进行比较以检测错误。在[119]

中，作者提出了一种C/C++源代码级的容错方法。该方法通过对代码重排和变量复制提高C/C++程序可靠性。该方法通过RECCO工具可以由纯软件和自动化的方式实现，RECCO在分析程序之后，对程序的源代码进行转换。该工具会分析程序中变量的生命周期等特性，计算出变量的权重。基于所有变量的权重排序，用户可以选择数据复制的百分比，从而在错误覆盖率和性能之间取舍。在[69] 中，作者建立了一个概率模型用来描述错误在软件中的传播过程。该文把错误分成两类，原始错误和传播错误。为了分析错误的传播属性，该文首先建立了一个数据流模型，然后基于该模型增加一个错误流模型。在此基础上，作者给出了一些错误传播规则，通过这些规则，可以计算错误在程序中发生的概率。在[70]中，作者分析了软错误和浮点算数运算的关系。

基于校验码的Java虚拟机容错技术

基于校验码的容错技术主要用来实现对程序数据流进行错误检测和错误恢复。该技术的一个重要指标是校验的粒度。在Java虚拟机中，有的方案采用对象级别的校验粒度，有的采用字段（Field，或称域）级别的校验粒度，还有混合方式的校验粒度。

在[16]中，作者研究了瞬态错误对Java虚拟机产生的影响，作者还在Java虚拟机的堆和静态数据区实现了校验码来检测数据错误。该文将Java对象分成小对象和大对象，小对象是那些体积小于页大小的对象，大对象则为体积大于页大小的对象。对于每个小对象，作者额外增加两个字节的空间用于存储校验码；对于大对象，作者将大对象分成相等大小的块，并对每个块增加相应的校验码空间。该方法对系统的性能产生了32%-57%的代价，但是能够检测出多至39%的内存错误。该方法对性能影响较大，而且只保护了Java虚拟机的堆空间。

在[20]中，作者以嵌入式系统的Java虚拟机为平台，对比了以对象为粒度进行校验（Object-Level Checksum，简称CSOBJ）和以字段为粒度进行校验（Field-Level Checksum，简称CSFLD）这两种方案各自的特点。作者还研究了以这两种方案为基础的两个混合方案：基于阈值的校验 (CSOBJ with a Threshold，简称CSTHR)以及分组方式的校验（Group-Level Checksum，简称CSGRP）。基于阈值的校验是指为每个对象增加一部分空间用于记录上次校验的时间，在间隔时间超过阈值时对该对象进行校验；分组方式指的是对对象中的字段进行分组校验，这样既不会像对象级校验那么消耗校验时间也不会像字段级校验那么消耗校验空间。该文通过实验对比了各个方案在可靠性、性能和内存消耗方面各自的优缺点。本章的方案也是一种基于阈值的校验方式，在[20]中对对象的计数是基于字节码的执行次数，这种方式并不足够准确，因为每个字节码的执行所占用的时间不尽相同，另外字节码执行可能会被Java虚拟机其他的线程，比如垃圾收集所打断，这都会造成计时的不准确。

5.2.2 动态污点分析技术

动态污点分析技术是指在程序运行中实时标记程序的输入，并在数据的传播和

计算过程中对数据进行自动的跟踪[120]。在[121] 中，作者提出了一种基于硬件的污点分析技术。在[122]中，作者提出了一种通过污点分析技术防止SQL注入攻击的方法。Panorama[123]利用污点分析技术实现了一种自动提取病毒关键代码的功能。DTA++[124]实现了一种增强的污点分析技术，通过分析给定污点信息的执行路径，能够诊断发生错误的位置。

Haldar[125]提出了一种动态的方法，可以在运行时标记、跟踪和检测用户的输入，以防止用户不恰当的行为导致程序的异常。该方法运行于Java虚拟机上，通过对Java类库进行修改，实现对用户输入的字符串做标记，可以不需要Java程序的源代码。

在[126]中，作者提出了TaintDroid系统，该系统通过修改Android的虚拟机实现对智能手机应用程序的实时分析。该系统假定来自网络下载和来自第三方的应用都是不可信的，并对这些应用如何操作用户个人数据进行监测。TaintDroid对来自不可信的数据做标记，并在数据传播的过程中维护标记的传播。

在[127]中，作者提出了一个基于Java虚拟机的污点跟踪系统Phosphor，该系统能够在性能、准确度和移植性上均取得较好的效果。跟以往污点跟踪系统不同，该系统并不需要修改Java虚拟机，也不需要修改Java程序的源代码和操作系统，它是在字节码层面实现了污点跟踪。通过对Java字节码进行扩展，实现标签创建、标签传递等功能。因此该系统可以直接应用于现有的Java虚拟机中，比如Oracle的Hotspot和OpenJDK。该系统对性能的影响平均为53%。

5.3 方法概述

本章的目标是设计一种能够有效检测内存中的软错误又不给系统带来过高性能代价的方法。以往的方式大部分都是对Java虚拟机中的数据无差别地进行检查，这种方式虽然能够带来较好的错误覆盖率，但是往往也会给系统带来很大的性能损耗。本章提出了一种基于数据的时间信息和数据的来源信息进行数据检查的方法。

本方法的一个主要思想是对刚进行过检查的数据短期内不再进行检查。也就是说本方法基于的一个假设是：访问一个近期刚检查过的数据，发生错误的概率要远低于访问一个很久没有检查过的数据。比如，在一个循环中连续对同一个数据进行访问，那么本文认为没有必要每一次访问都进行一次检查。相应地，只需在第一次访问数据时以及每隔一段时间对该数据进行一次检查即可达到较好的容错效果。本方法另一个主要思想是对那些与含有软错误的数据来源相同的数据着重检测。这是因为我们认为如果某一数据被检测出错误，那么与它相同来源的数据以及由它产生的数据发生错误的概率也会更高。

基于这两个想法，我们提出了CST（Checksum，Source and Time）方法。我们给Java栈中以及堆中的每个数据增加一个标签，标签记录数据的校验时间、来源信息以及校验码等信息。标签的结构如图5.1所示。在数据进行移动和计算的过程中，标签也

进行相应的移动和计算。整个方案描述如下：

1. 对于所有新创建的数据（包括基本数据类型、对象、数组），为其创建一个标签并赋初始值，其中时间标签为当前时间，来源标签根据用户的需要进行配置，校验码根据数据的内容计算得出。
2. 对于栈中的每个数据（包括基本数据类型和引用类型），其标签以影子标签方式存储。
3. 对于堆中的每个对象，在其内部新建一个标签域。
4. 每次访问数据，对其相应的标签进行一次检查：检查当前时间与数据的时间标签记录的时间之差是否超过阈值，若超过阈值则执行校验检查，检查完将时间标签设置为当前时间；若没有超过阈值则不进行其他操作。
5. 每次更改数据，对数据的校验标签进行更新。
6. 对于$C = A\ op\ B$类型运算，比较A的时间标签和B的时间标签，将较早的值付给C的时间标签。
7. 对于$C = A\ op\ B$类型运算，对A的来源标签和B的来源标签进行按位或运算，结果赋给C的来源标签。
8. 如果校验过程中发现错误，则记下数据的来源标签s，对于之后所有读取的数据，如果其来源标签位s'含有出错数据的来源标记（$s \mid s' \neq 0$），则对数据进行校验。

采用CST方法还可以带来几个额外的好处：首先，通过本方法可以对程序进行安全控制。CST可以在程序运行时对用户的输入进行标记和跟踪，在发现恶意代码时，可以根据数据的来源标记判断该数据来源于哪个用户。该方法可用来防范多种类型的攻击，比如SQL注入攻击、格式字符串攻击、缓冲区溢出攻击等[128–131]。其次，该方法可以有助于隐私控制。智能手机的一大核心功能就是可以随时下载并使用第三方应用，为了提供相应的服务，这些应用往往会需要读取手机中的隐私数据，这些应用给人们带来便利的同时，也可能会造成隐私的泄露。通过本章提到的方法，可以对隐私数据进行标记，并对智能手机的网络进行监控，在网络收发数据时若发现隐私数据的标记，则在系统日志中记录下相应的信息，从而实现对第三方应用监控[126,132–135]。最后，该方法能够给调试程序带来便利。在程序发生错误或者崩溃时，通过分析数据的来源标记和时间标记，能够方便回溯和缩小排查的空间[136,137]。

本方法并不能杜绝软错误的发生，比如刚校验完的数据就发生了位翻转。本方法是对性能和可靠性的一个折中。另外，本方法只针对Java虚拟机中的数据流进行容错，即Java虚拟机的堆和栈空间，没有针对控制流。控制流部分的容错在本文第四章中给出，两个方法合起来可以实现对整个Java 虚拟机内存部分的容错。

本方法目前只实现了错误检测的功能，通过对本方法进行扩展，可以很容易实现错误恢复的功能，比如使用本文2.2.3节中介绍的具有错误恢复功能的编码方式。

5.4 实现与优化

基于龙芯平台的64位Java虚拟机，我们实现了CST方法。时间标签、来源标签和校验码被集成在一个标签结构中，这个标签结构在Java虚拟机中根据数据是存放在堆中还是栈中采用不同的存储方式。本节将介绍标签的实现方式和传递规则。在本节的最后，我们还对CST方法进行了一些优化。

5.4.1 标签存储

如何在Java虚拟机中存储标签会影响程序的性能代价和存储代价。在Java虚拟机中，数据主要是在Java栈和堆中存储。Java栈存储Java程序运行过程中需要的局部变量、操作数、方法参数等。堆中保存的则是Java程序运行过程中分配的对象[7]。

我们为Java栈和堆中的每个数据都分配了相应的标签。标签的结构如图5.1所示，每个标签占64位，其中校验码为32位，来源标签为15位，校验有效位为1位，时间标签为16位。

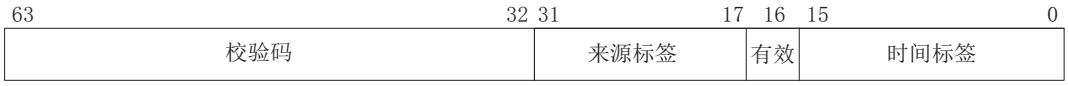


图 5.1 标签结构

Java栈中的数据。Java栈中主要存放的是基本数据类型和引用类型（reference类型）。Java虚拟机有八个基本数据类型，分别是bool、byte、char、int、short、long、float以及double，这八个基本数据类型以值的方式在Java虚拟机中存储和传递。除了基本数据类型，Java虚拟机还支持两种引用类型：对象和数组。引用类型不等于对象本身，对象的实例存放于堆中，引用类型一般是指向对象起始地址、以指针的方式在Java栈中存储和传递的。在64位Java虚拟机中，一个引用类型占64位[2,114]。

对于Java栈中的每一个数据，我们都为其创建一个影子标签。Java栈中可以存放的数据类型在64位Java虚拟机中都是64位对齐的，其中long类型和double类型占两个单位宽度，其他类型均占一个单元宽度。在本章的方法中，我们对Java栈进行了扩展，将Java栈中数据的体积均扩大了一倍用于存放影子标签，其中long型和double型扩大以后有64位数据并没有使用，这么处理是为了在64位Java虚拟机中实现对齐便于计算。每个对象的引用存放于Java栈中，引用类型同基本数据类型一样，也对应一个影子标签。

本节以执行字节码iadd（int型相加）前后Java栈顶的布局为例说明标签的存储。在图5.2中，O代表数据，ST代表来源标签，TT代表时间标签。在iadd 执行之前，Java栈顶依次为两个int型O_1和O_2，以及一个long型数据O_3（$O_3.W_1$和$O_3.W_2$分别表示long型的第一个字和第二个字）。在执行完iadd之后，栈顶依次为int型加法结果和long型O_3。由图5.2可见，long型的标签位于第一个字之上，long 型的第二个字之上的64位空间没有使用。由于本文使用的Java虚拟机的栈是向下生长，因此栈顶在低地址。计算结果的时

间标签为TT_1和TT_2中更小的，计算结果的来源标签则等于ST_1和ST_2按位或计算所得的结果。如果去掉标签位，则执行iadd前后的栈顶布局和没有采用CST方法的Java虚拟机的栈顶布局相同。

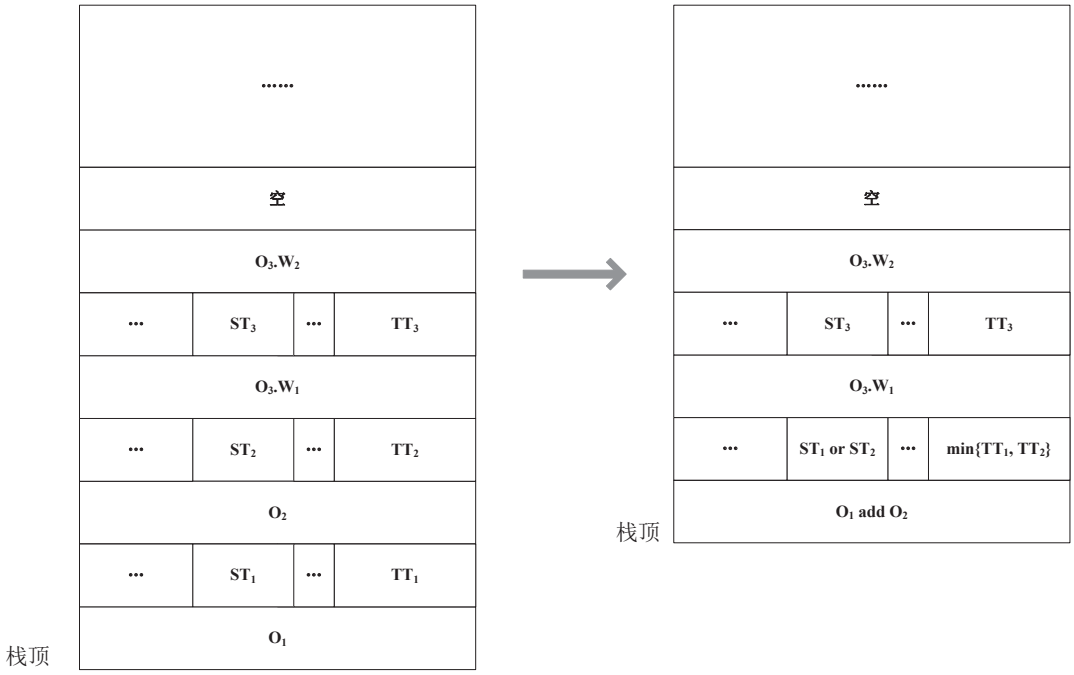


图 5.2　执行iadd前后Java栈顶的情况。O表示操作数，ST表示源标签，TT表示时间

在Java栈中还会存放一些Java虚拟机运行过程中需要的中间数据，本方法对这些中间数据也建立了标签以实现对这些数据的容错。这些中间数据包括Java栈帧压栈和出栈过程中所需要保存的寄存器sp、寄存器fp以及参数等。在Java虚拟机访问这些数据时，同样需要对标签位中相应的内容进行检查已决定是否进行校验操作。

堆中的数据。为堆中的数据创建标签主要有两种方法：对象级标签和字段级标签。对象级标签指的是为每一个对象分配一个标签。字段级标签指的是为对象里的每个字段分配一个标签。同对象级标签方式相比，字段级标签会造成更大的内存负担，但同时能够更准确地对错误进行检测。从性能角度来讲，两种方式难分伯仲，这是因为两种方式会各自带来性能代价。采用对象级标签，每次对对象字段的访问都需要对整个对象进行校验，另一方面，采用字段级标签，由于每个字段都对应标签，因此会影响Cache的效率[20]。本章采用的是对象级标签的方式，由于本章方法的特点是能够减少校验的次数，因此能够一定程度缓解采用对象级标签的给性能带来的影响。

对于堆中的数据，我们在每一个对象的实例中增加了一个64位的标签域。该标签位于对象头的末尾，在每个对象中位于固定偏移量的位置上。这种存储方式有利于计算校验码和访问对象字段等操作。

5.4.2 校验码

每个标签中的校验码为32位，其值为对校验的数据以32位为单位进行异或的结果。对于Java栈中的数据，每一个影子标签中校验码的值为数据的高32位与低32位异或的结果（对于引用类型，其校验码为64位指针的高32位和低32位异或的结果）。对于堆中的数据，每个对象的校验码为该对象中除了标签以外的其他数据以32位为单位进行异或的结果。我们固定了标签在每个对象中的相对位置，这样可以通过忽略固定位置的数据以便于校验码的计算。

校验码在数据创建的时候生成。每次改变了Java栈中的数据时，相应的校验码需要重新计算并更新，比如iconst字节码；同样的，每次改变了对象中的值，该对象的校验码也需要重新计算并更新，比如 putfield字节码。Java栈中的数据为64位，计算校验码仅需一个异或运算就可以完成，不会给系统带来过高性能损耗。但是重新计算对象的校验码往往需要进行多次访存和多次异或计算，这有可能会给系统带来较大的性能损耗，针对这一问题，本章进行了相应的优化，具体优化方法在5.4.7节中介绍。

5.4.3 时间标签

时间标签中记录的是该数据的创建时间或者上次校验时间。在数据创建时，时间标签随之创建，其值为数据创建时间。每次对数据访问时，均对比数据的时间标签与当前时间的差值，若大于设定的阈值，比如5000毫秒，则对该数据进行校验。校验完成后将时间标签更新为当前时间。本方法采用的时间标签为16位、以毫秒为单位，因此可以设定的阈值范围大概为1毫秒-65秒。

5.4.4 来源标签

来源标签用来标记数据的来源，本方法中来源标签占15位，可以追溯15种来源。在创建数据时，根据具体需要对不同的数据标记不同的来源位。例如，对用户A输入的数据由来源标签第1位标记，用户B输入的数据由来源标签第2位标记，来自硬盘文件的数据由来源标签第5位标记，那么一个来自硬盘中用户B创建的文件的数据的来源标签可能是0x12。如果不需要对来源进行标记，在创建标签时将数据的来源标签设置为0即可。当数据进行计算时，它们的来源标签也跟着进行按位或的运算。

在执行字节码的过程中，由于不同来源数据之间的计算，可能会导致错误数据以及不安全数据的扩散。通过标记来源信息，在纠错时根据数据的来源位标记可以追踪错误可能的来源和去向。在本方法中，如果某一数据检测出错误，那么我们认为该数据的来源标签标记的所有来源均可能为不可靠、不安全的。在接下来的校验过程中要对标记同样来源的数据进行校验。另外，开发者也可以根据需要实现更复杂的来源标签的生成和计算的逻辑。

5.4.5 本地方法

由于本地方法需要符合相应体系结构的接口定义，因此在参数传递的过程中无法使用自定义的标签结构。本章的实现是基于龙芯处理器的，因此在调用本地方法时我们需要舍弃参数的标签，构造符合MIPS ABI的寄存器和栈结构。在参数返回时，我们需要对返回结果创建新的标签：时间标签设置为当前时间，来源标签设置为该本地方法所有参数的来源标签按位或的结果。另外，来源标签也可设计为对不同的本地方法标记不同的来源位，这能一定程度上实现对本地方法调用的监测。

5.4.6 需要修改的字节码

我们的方法需要对很多字节码的翻译规则增加额外的代码。表5.1列出了所有我们修改过的字节码，并简要介绍了我们对这些字节码进行的修改。我们将这些字节码依据对栈顶数据和堆中数据的改变进行分类并在操作语义一列列出。在表5.1中，v_x表示Java栈中的数据，f_x 表示堆中的数据，D表示常量，$\tau(v)$表示数据v的标签，$t(v)$表示数据v的时间标签，$s(v)$表示数据v的来源标签，符号←表示赋值操作，比如$\tau(v_A) \leftarrow \tau(v_B)$表示将$v_B$ 的标签赋给v_A的标签位，$new(v)$表示为v新建一个标签。

5.4.7 优化

对于CST方法，本节进行了进一步的优化。

- **短生命期数据优化。**对于生存期比较短的数据，可以不进行创建标签和校验等操作。生存期比较短的数据包括在没有方法调用并且本身代码不长的方法中声明的局部变量，如内联函数中声明的局部变量。这一类数据存活期很短，因此不必对其进行校验。
- **更新校验码优化。**每次更新了对象中的值，都需要重新计算整个对象的校验码并更新原有的校验标签[20]。这会给程序带来较大的性能负担。针对这一问题，我们对更新对象校验码的过程进行了优化。设对象之前的校验码为c_P，更新之后的校验码为c_N，对象的目标值修改前为v_P，修改后为v_N，由异或运算的运算法则可以得出更新以后的校验码c_N的计算公式为：

$$c_N = v_P \ XOR \ v_N \ XOR \ c_P \tag{5-1}$$

  其中，v_P，v_N和c_P均为已知，因此校验码可以通过该公式直接计算得出。另外，由于数据在对象中的存储不是以32位对齐的，因此读取数据后，应该进行相应的移位操作，然后再计算出新的校验码。
- **栈顶优化。**Java虚拟机是一种基于栈操作的虚拟机，在指令执行之前和之后通常要进行出栈和压栈的操作。对于MIPS体系结构，这会带来很大的性能开销，因此有些Java虚拟机会专门使用一个寄存器作为栈顶寄存器用来存放栈顶的数据，

表 5.1 Java字节码对应的标签传播规则

字节码	操作语义	标签传播规则
iconst, lconst, fconst, dconst, aconst_null, bipush, sipush	$v_A \leftarrow D$	$\tau(v_A) \leftarrow new(D)$
iload, lload, fload, dload, aload	$v_A \leftarrow v_B$	$\tau(v_A) \leftarrow \tau(v_B)$
istore, lstore, fstore, dstore, astore	$v_A \leftarrow v_B$	$\tau(v_A) \leftarrow \tau(v_B)$
iaload, laload, faload, daload, aaload, baload, caload, saload	$v_A \leftarrow f_B$	$\tau(v_A) \leftarrow \tau(f_B)$
iastore, lastore, fastore, dastore, aastore, bastore, castore, sastore	$f_A \leftarrow v_B$	$\tau(f_A) \leftarrow \tau(v_B)$
dup, dup_x1, dup_x2, dup2, dup2_x1, dup2_x2	$v_A \leftarrow v_B$	$\tau(v_A) \leftarrow \tau(v_B)$
Xadd, Xsub, Xmul, Xdiv, Xrem, Xshl, Xshr, Xushr, Xand, Xor, Xxor	$v_A \leftarrow v_B \otimes v_C$	$t(v_A) \leftarrow min(t(v_B), t(v_C))$, $s(v_A) \leftarrow s(v_B)\|s(v_C)$
	$v_A \leftarrow v_A \otimes v_B$	$t(v_A) \leftarrow min(t(v_A), t(v_B))$, $s(v_A) \leftarrow s(v_A)\|s(v_B)$
Xneg	$v_A \leftarrow \otimes v_B$	$\tau(v_A) \leftarrow \tau(v_B)$
i2l, i2f, i2d, l2i, l2f, l2d, f2i, f2l, f2d, d2i, d2l, d2f, i2b, i2c, i2s	$v_A \leftarrow v_B$	$\tau(v_A) \leftarrow \tau(v_B)$
getstatic, getfield	$v_A \leftarrow f_B$	$\tau(v_A) \leftarrow \tau(f_B)$
putstatic, putfield	$f_A \leftarrow v_B$	$\tau(f_A) \leftarrow \tau(v_B)$
new, newarray, anewarray, multianewarray	$f_A \leftarrow D$	$\tau(f_A) \leftarrow new(obj)$, $\tau(v_B) \leftarrow new(D)$
arraylength	$v_A \leftarrow f_B$	$\tau(v_A) \leftarrow \tau(f_B)$
ldc, ldc_w, ldc2_w	$v_A \leftarrow D$	$\tau(v_A) \leftarrow new(D)$
swap	$v_A \leftrightarrow v_B$	$\tau(v_A) \leftrightarrow \tau(v_B)$
iinc	$v_A \leftarrow v_B$	$\tau(v_A) \leftarrow \tau(v_B + D)$

当一个字节码的输入正好是上一个自己码的输出时，直接使用栈顶寄存器中的值即可，无需压栈和出栈[115,116]。本章对这种方式进行了扩展，使用了一个寄存器专门作为栈顶标签寄存器，用来存放栈顶数据的标签。

- **获取当前时间优化。**每次创建标签、校验标签都需要获取系统当前时间，这会频繁地调用系统函数，造成性能影响。本章使用了一个寄存器专门用于存放当前时间，并周期性地调用时间函数对这个寄存器的值进行更新，使其近似等于系统当前时间。每次创建标签、校验标签时直接使用该寄存器中的值即可。对于寄存器数量较少无法使用专门寄存器的系统，可以使用一个64位的全局变量存放系统当前时间。

5.5 实验结果和分析

通过实验我们评测了本方法的性能代价、内存代价以及进行错误检测的效果。我们在龙芯版的64位OpenJDK 6中实现了该方法。实验使用的平台是龙芯3A 处理器，4G内存，操作系统为Linux2.6.36.3。我们对比测试了使用CST方法的Java虚拟机和原始的Java虚拟机。实验采用的测试程序为SPECjvm2008，其中一些测试程序的访存特性在表5.2中列出。第二列平均尺寸列出了每个测试程序创建的对象的平均大小，单位为字节。第三列和第四列分别列出了每个测试程序读取对象字段和更新对象字段的次数。最后一列则列出了每个测试程序访问对象的总次数。表5.2显示，大部分测试程序的对象尺寸都比较小，比如sunflow，对象的平均尺寸只有6个字节。通过这一点可以看出，使用对象级的操作码可能不会给程序带来过高的性能代价。另一方面，对象平均尺寸小意味着对象级的操作码可能会给程序带来较大内存代价。

5.5.1 性能损失

首先我们评测了CST方法对于Java程序和Java虚拟机性能产生的影响。实验结果如图5.3所示，柱状图的两列分别表示在不使用和使用5.4.7节中的优化方法时，CST方法给各个测试程序所带来的性能损耗。纵轴的数值表示跟不使用容错方法相比，CST给程序带来的性能负担，纵轴越低越好。实验结果显示，不使用优化方法的CST方法给测试程序带来的平均35%的性能损耗。优化方法给程序带来的性能提升非常明显，能够给有些测试程序带来超过50%的性能提升，比如compress和mpegaudio。在使用优化方法的情况下，CST方法给程序带来的性能损耗最大的scimark.fft.small和scimark.sor.small为34%，最少的sunflow为15%，平均26%。

表 5.2　测试程序列表

测试程序	平均尺寸	读	写	对象访问
compress	46	1442500642	288502445	1731003087
crypto.aes	15	3151060808	28874137	3179934945
crypto.rsa	19	71915399	3422886	75338285
crypto.signverify	38	517542706	5324274	522866980
mpegaudio	55	1372226347	80977419	1453203766
scimark.fft.large	67	109159623	10694652	119854275
scimark.lu.large	30	107535652	10513174	118048826
scimark.sor.large	30	107359064	10467816	117826880
scimark.sparse.large	57	107644127	10511475	118155602
scimark.fft.small	63	159131557	19105251	178236808
scimark.sor.small	32	83620135	6384202	90004337
scimark.sparse.small	41	74380688	4812940	79193628
scimark.monte_carlo	32	4047694087	673404300	4721098387
serial	8	927588905	178102327	1105691232
sunflow	6	1886066262	266867256	2152933518
xml.transform	8	1355626242	227637216	1583263458
xml.validation	9	685726058	115622184	801348242

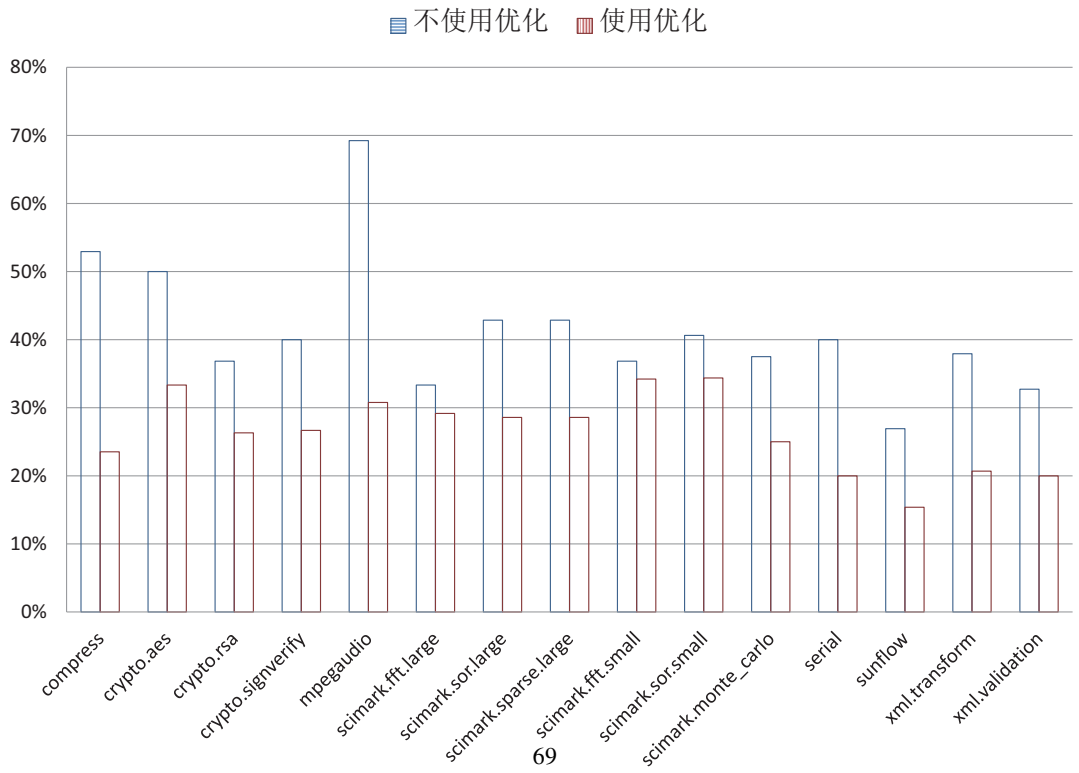


图 5.3　性能测试

接下来我们评估了不同的阈值对于程序性能的影响。图5.4显示了使用不同阈值，对于SPECjvm2008测试得分的影响。横轴表示阈值，纵轴表示跟不使用容错方法相比给程序带来的性能负担，纵轴越低表示给系统带来的性能开销越小。通过图中的折线图可以看出，在阈值小于10秒时，CST方法给系统带来的性能负载较大，很多测试程序超过了30%。当阈值增加到20秒时，系统的性能损耗明显下降，普遍低于30%。另外，对于不同的测试程序其性能开销随阈值变化的趋势也不尽相同。对于compress，随着阈值的增加，性能提升也比较明显；对于crypto程序，其性能损耗基本不随阈值的变化而变化。

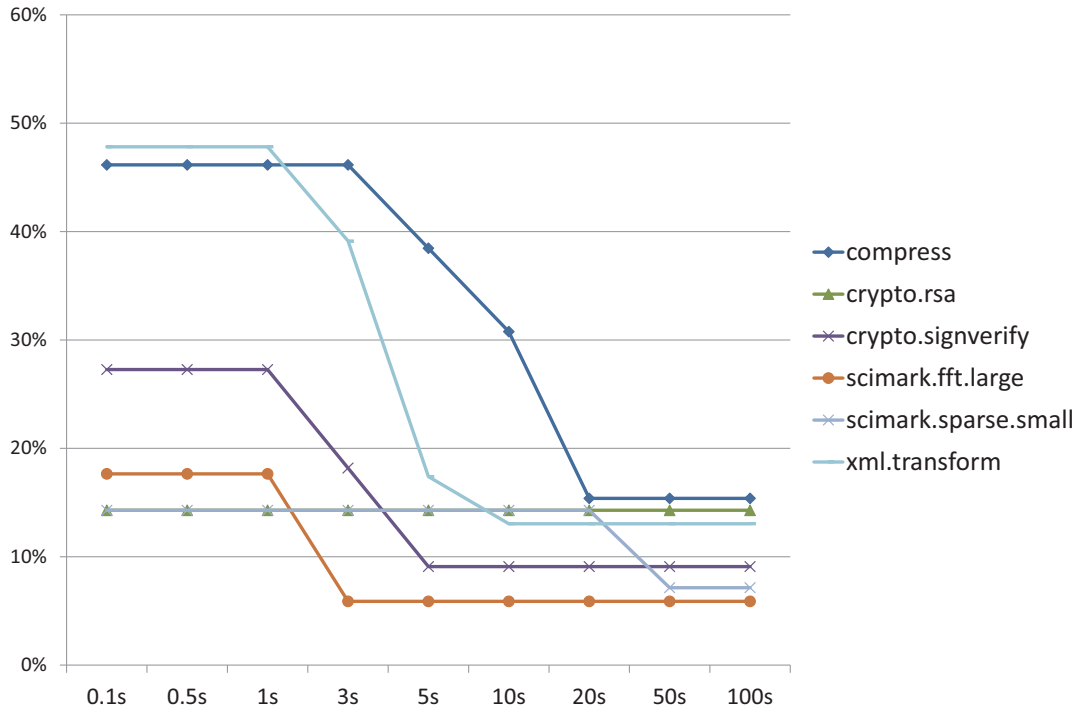


图 5.4　性能测试：阈值

5.5.2　内存消耗

图5.5显示了由于采用CST方法给堆空间分配对象的累积尺寸带来的增长。实验结果显示，由于增加了标签位，CST方法使得测试程序堆空间的累积尺寸增加了大概2%-14%，平均增加10%。由表5.2可见，测试程序的平均对象大小的范围为6字节至67字节之间。根据图5.5和表5.2可以看出，平均对象尺寸越小的测试程序，CST方法对其累积尺寸带来的增长率越明显；平均对象尺寸越大的测试程序，CST方法对其累积尺寸带来的增长率越小。比如，xml.transform的累积对象尺寸增加最明显，其平均对象尺寸为8字节；mpegaudio的累积对象尺寸增加较小，仅为3%，其平均对象尺寸为55字节。这是因为平均尺寸越小，则标签位在对象中占得比例越大，最终体现为带来更多的尺寸消耗。

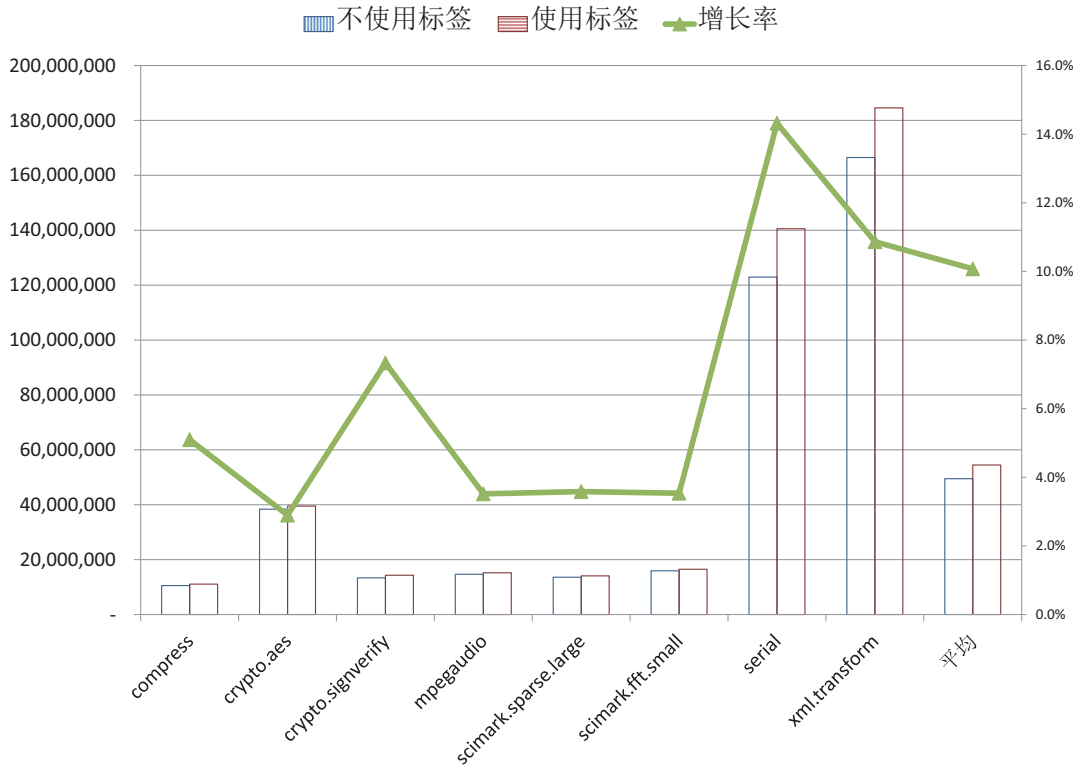


图 5.5　对象累积尺寸

为了测试标签位给栈空间带来的体积负担，我们编写了一个测试程序。该程序用来测试程序的嵌套深度，从而测试CST方法对栈空间的影响。Java 程序进行方法调用时，会在栈中保存相应的数据，因此，通过测试方法调用的深度可以评估栈空间体积增加的情况。我们的测试以1K字节为单位，逐步缩小栈空间的尺寸（由-xxs参数指定），从而测试出程序能够正确执行所需要的最小的栈空间。程序执行结果在图5.6中给出，由实验结果可见，在函数调用嵌套10次时，Java虚拟机使用最小的栈空间2K均可正确执行；当程序的函数调用嵌套1000次时，不采用和采用CST方法的Java虚拟机分别需要最少199K和215K的栈空间才能正确执行，栈空间的需求增长了8%；当程序的函数调用嵌套100000次时，不采用和采用CST方法的Java虚拟机分别需要最少9480K和11043K的栈空间才能正确执行，栈空间的需求增长了16.5%。

5.5.3　错误覆盖率

本节我们评测了CST方法进行错误检测的效果。为了模拟软错误，本章使用4.5.1中提到的软错误注入方法。该方法使用一个软错误注入线程，通过读取配置文件，周期性地像Java虚拟机中注入位翻转。可配置的参数为内存地址范围、最多翻转位数（Maximum Faulty Bits，简称MFB）以及最大注入间隔（Maximum Injection Interval，简称MII）。在本实验中，我们向整个Java虚拟机地址空间中注入错误，最大翻转位设为5位，最大注入间隔设为20秒。我们使用SPECjvm2008中的compiler.compiler作为测试

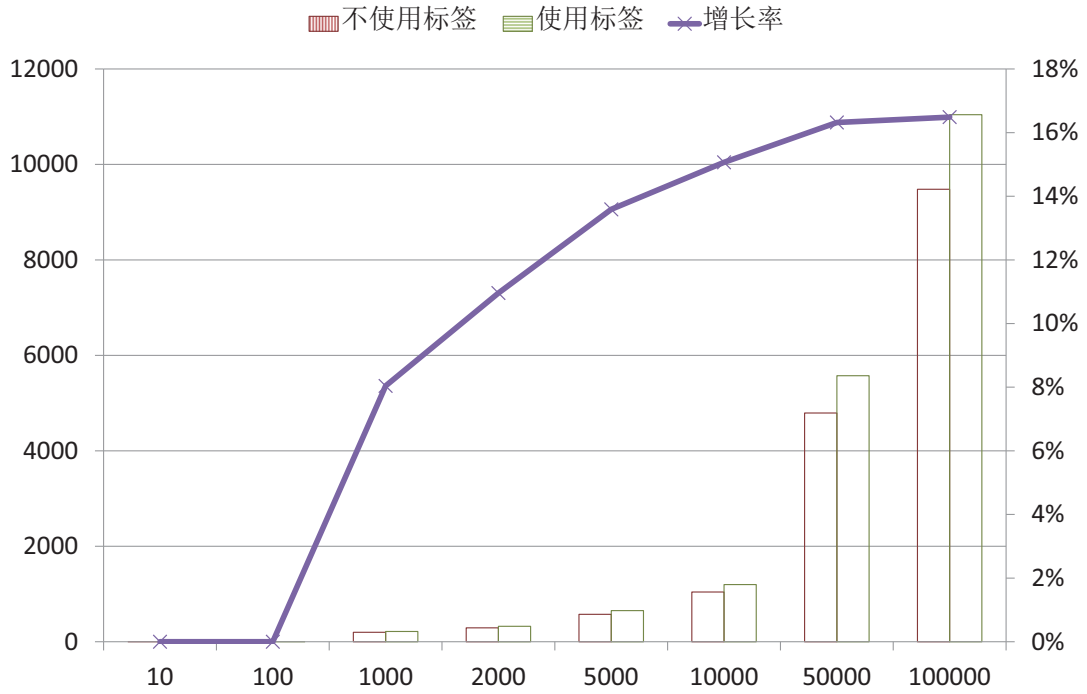


图 5.6　最小栈需要

程序，4线程执行（-bt参数为4）。Java虚拟机的初始堆空间和最大堆空间均设为2G（通过命令行参数-Xms和-Xmx设置）。

我们将实验分四组进行，每组实验执行100次。这四组实验分别使用不同的阈值：无穷大、50秒、5秒和1秒。阈值无穷大表示不使用CST方法进行数据校验。在实验过程中，一旦检测出错误，则程序停止执行。实验结果在表5.3中给出。当阈值为50秒、5秒和1秒的时候，能够正确完成执行的次数分别是18次、21次和19次。正确完成数区别不大是因为目前CST方法只实现了错误检测，而没有实现错误恢复。当阈值为50秒的时候，本章方法的效果已经体现。该方法能够检测出100次运行中的10次错误，程序错误完成数和崩溃数均有所减少。当阈值减小为5秒的时候，Java虚拟机的可靠性进一步提高。表5.3显示，当阈值从5秒降低为1秒时，Java虚拟机可靠性的提升不如阈值由50秒降为5秒时的明显。当阈值降为1秒时，本章方法能够检测出46%的错误。因为有20%的执行能够正确完成，因此CST方法可以检测出所有错误执行中的大概57%。因为我们使用的软错误线程是向整个Java虚拟机中注入错误，而CST只检测堆中和栈中的数据，因此方法的效果受到一定影响。

表 5.3 不同阈值对CST方法效果的影响

阈值	∞	50s	5s	1s
正确结果	19	18	21	19
错误结果	48	44	23	21
崩溃	29	26	11	12
其他	4	2	1	2
检测到错误	0	10	44	46

5.6 本章小结

本章提出了一种称为CST的Java虚拟机污点标记技术，用来实现对Java虚拟机的堆空间和栈空间中的数据进行错误检测。该方法动态追踪数据的时间信息和来源信息，对数据进行有差别的检查，从而消除了一些不必要的性能损失。本章还针对CST提出了一系列优化方法，包括短生命期数据优化、更新校验码优化、栈顶优化以及获取当前时间优化，这些优化能够大幅提升CST的性能。基于龙芯64位OpenJDK我们实现了该方法并进行了评测实验。实验显示，在阈值设为15秒时，该方法对Java虚拟机产生的性能代价不超过20%，该方法对堆空间中分配的对象累积体积平均增加10%，对最小栈空间需求增加了不超过17%。在阈值设为1秒时，CST方法可以检测出约57%的错误执行。

第六章　Java虚拟机访存与加载地址优化

6.1　引言

因为具有更大的地址空间，64位系统可以运行更大型的程序。但同时，在MIPS体系结构中，比如龙芯处理器，加载一个64位地址比加载一个32位地址需要更多的指令，这不仅增加了代码区的内存消耗，也给程序带来性能影响。本章提出了一种新的访存和地址加载的方式，实现了更好的程序性能和更小的代码体积。

MIPS指令格式固定长度的特点导致了加载地址的问题[138]。MIPS规定，所有指令都是32位长且常数项最多为16位（跳转指令除外），因此加载一个32位地址需要多条指令。在64位系统中，地址长度加倍，而指令格式不变，这使得加载地址的问题变得更为严重。为了解决这一问题，在硬件和软件方面都有一些相关技术的提出。在硬件方面，使用lui（Load-Upper-Immediate）这条特殊功能指令，通过两条指令可以加载一个32位地址；在软件方面，可以使用sp、gp寄存器专门用来保存堆栈指针和全局指针，或是使用全局偏移量表（GOT）保存所有全局变量的地址。

这些方法能缓解32位系统的情况，却无法有效适用于64位系统[139]。即使使用lui指令，加载一个64位地址也需要数条指令才能完成；指令的偏移量只有16位，对于数据较多的大型程序，一些数据可能无法通过sp和gp直接访问；GOT也存在类似的问题，对于大型程序，全局地址表可能会变得非常庞大，导致不能通过一个不变的全局指针访问。

本章将加载地址进行分析和分类，分别通过硬件和软件两种方式，对Java虚拟机生成代码的访存和取址进行了优化。在硬件方面，利用龙芯处理器中的双基址寄存器访存方式，本位提出了一种称为快速地址访问（Fast Memory Access）的方法，该方法选取一个寄存器用作全局寄存器来存储地址的高位部分，在每次访存前加载目标地址的低位部分，从而将加载地址并访存的指令数由6条降低为3条。利用这个全局寄存器，还可以使加载程序地址的指令数由6条缩减为3条。但是这种方式具有一定的局限性，为了解决这一问题，在软件方面，本章实现了一种优化的加载地址方法，利用在Java虚拟机中实际用到的地址位数并未达到64位这一特点，简化了对64位地址的加载过程，将加载地址的指令数由6条缩减为4条。通过对访存和取址的优化，对Java虚拟机的整体性能和生成代码的体积均产生了优化效果。

我们在龙芯64位OpenJDK上实现了该方法，并且在龙芯平台上评估了该方法对程序性能和内存消耗的影响。通过SPECjvm2008和Dacapo的评测，该方案使SPECjvm2008平均提升了5.1%，最高项提升了13.2%；Dacapo平均提升7.3%，最高

项提升24.9%。在内存消耗方面，该方法使Java虚拟机即时编译器生成代码的体积平均减少了16%。

本章接下来将会阐述我们的访存方法和取址方法，并测试该方法对Java虚拟机产生的效果。第二节介绍了相关工作；在第三节，会对Java虚拟机生成代码加载地址的行为进行分析，这部分是我们方法的依据和动机；第四节描述了双基址寄存器访存方法，以及实现细节；第五节介绍了基于软件优化的地址加载方法；实验结果以及实验分析会在第六节给出；第七节总结。

6.2 相关工作

Chow在[140]中针对RISC体系结构单一的访存方式，通过对编译过程进行改进，在软件上实现了对加载地址和访存的优化。这些方法包括：1.消除重复的lui指令，相邻的访存指令往往使用相同的高16位地址，这种情况下可以消除后面重复的lui指令；2. 使用公共的基地址寄存器，对不同地址的内存进行访问时，编译器尽量识别出这些地址的公共基地址，然后通过不同的偏移地址进行访问，从而减少地址加载；3. 合理安排全局变量区，使用gp寄存器指向一个叫做gp area的64k内存区，这个区域保存部分或者全部的全局静态变量，这样以gp 为基地址使用一条指令就可以访问gp area中的数据，因为gp area容量有限，该文将大小不超过阈值的变量放入gp area，并讨论了不同阈值对程序的影响。这些优化方式只针对静态编译方式的32位软件，并不能适用于64位Java虚拟机这种动态编译软件。

Srivastava在[141]中对各个模块的GAT（Global Address Table）在链接阶段进行了优化。GAT用于存储全局变量的地址，每一个编译的单元拥有一个GAT，在链接的时候将多个GAT合成一个GAT。Sribastava 对链接的过程进行了优化，比如在GAT足够小的时候直接使用寄存器gp访问变量、在不同的函数中使用同一个GAT和使用BSR代替general call从而消除函数入口和出口处的开销、进一步进行流水线调度等。这些优化方式是在对整个软件的各个模块全面分析的基础上进行的，必须要掌握整个程序的信息才可以进行。因此该优化方式并不适用于动态编译的程序。另外这种优化方式只针对加载全局变量的地址，对于加载局部变量不能产生优化效果。

Steven[142]提出了一种地址计算方法并且在一些有利的情况下，可以将典型的RISC五阶段流水线缩减为四个阶段。该方法限定栈的大小为2的幂，因此访存指令在流水线的ALU阶段执行的sp+offset可以通过按位或来完成，按位或只需要一级门电路即可实现，这样就可以将访存指令流水线的ALU和MEM合并成一个阶段执行，在访存指令后就可以立即使用读取的内容。Steven的方法只能用来简化栈的访问，并且在每次函数的出入口处会有一些代价。

Austin[143]提出了一种快速地址计算的方法来降低访存延迟。这个方法通过在流水线早期阶段预测访存地址，然后使用预测的地址访问数据Cache，若预测正确则使用

该值，若预测错误则按正常方式取址。基于该方法，Austin还提出了一些软件方法用于提高预测的准确度。通过改造编译器和链接器，对全局指针访问、栈指针访问和普通指针访问实现优化从而达到提高预测准确率的目的。

在汇编器处理la指令时，会根据具体地址的值，生成最优的指令序列，这些指令序列的长度不一。但是这种方式并不能很好的应用于Java虚拟机中，因为：1. Java虚拟机动态编译、动态生成可执行代码，还会因为重定位而对指令序列中的地址进行动态修改，这使得在编译时并不能确定la的目标地址的值是多少。或者生成了相应的指令序列，后来修改了地址，导致旧的指令序列并不能加载新的地址；2. 这种根据立即数的值来生成指令序列的方式，会为编译过程带来开销，在一次编译多次执行的静态编译下，这种方法是可取的，但是在Java虚拟机这种动态编译的方式中，这会增加编译线程的开销。

6.3 加载地址行为分析

被加载的地址分为数据地址和程序地址。在MIPS系统中，只有一种寻址方式，即：寄存器基地址+立即数偏移量（16位）。要访问内存中的数据需要先将相应的内存地址装入寄存器（Load Address，简称LA），这个地址可能是要访问内存的地址（即间接寻址），也可能是要访问内存的基地址。在程序跳转时，可以先将地址加载至寄存器然后使用寄存器跳转指令，这样就可以不受地址跳转指令中26位偏移量的限制。因此，在寄存器跳转之前，也需要先将地址加载至寄存器。加载一个地址至寄存器有多种途径，一种常用的方式是加载立即数，这种方式简单直接，大量应用于各软件当中。本章研究的对象为加载立即数至寄存器，后文提到的加载地址和LA均指加载立即数。加载地址通常由宏指令la实现，它是由指令序列实现的。

我们分析了Java虚拟机运行测试程序时，生成代码中的la指令比例和地址分布情况，在本节接下来的部分将介绍我们的观察。

6.3.1 加载地址的分类

Java虚拟机包括解释器和翻译器两部分。解释器中有一个模板表的结构，模板表存储了每条Java字节码所对应的本地代码，执行时以逐条解释字节码的方式执行字节码所对应的本地代码；编译器则是以Java方法或者循环体为单位对字节码进行编译，生成本地代码。

我们统计了模板表中la指令占所有指令的比例，还统计了SPECjvm2008中几个测试程序的执行频度最高的5个方法编译后产生的本地代码中la指令的比例。表6.1列出了实验结果。编译器生成代码的统计方法是记录程序运行过程中所有编译方法的执行次数以及其编译后的本地代码，执行完成后对5个执行频度最高的方法所有生成的本地代码进行统计。由于la指令是由多条指令组成（64位Java虚拟机中一条la 宏指令包括6条指

表 6.1　la指令数和比例

测试程序	32位			64位		
	总指令数	la指令数	%	总指令数	la 指令数	%
模板表	30416	5350	17.6	40848	16848	41.2
compiler.sunflow	904	182	20.1	563	258	45.8
compress	2046	572	28.0	3198	1836	57.4
crypto.aes	1358	292	21.5	1961	924	47.1
derby	1730	460	26.6	2632	1470	55.9
mpegaudio	2852	762	26.7	4220	2352	55.7
scimark.fft.large	1226	274	22.3	1763	912	51.7
sunflow	1086	288	26.5	1660	906	54.6
平均	1600	404	24.5	2285	1237	52.6

令），此处按照实际的指令数进行统计。由表6.1可见，64位模板表和64位编译器的生成代码中，la指令所占比例非常大，高达50%左右。表6.1最后一行列出的是编译器生成代码中la指令的平均数，由此可见解释器模板表中la指令的比例要低于编译器生成代码中la指令的比例。另外，32位Java虚拟机生成的la指令的比例要远低于64位Java虚拟机，这也说明la指令给64位Java虚拟机带来更为严重的影响。

在程序执行过程中，可以将LA分成三类：加载程序地址（Load PC，简称LPC）、加载数据地址后访问内存（Load/Store After Load Address，简称LSLA）、加载数据地址后不访问内存（General Load Address，简称GLA）。LSLA和GLA的区别就是加载完数据地址后会不会对相应的内存进行读写。表6.2给出了64位Java虚拟机运行各个测试程序时，LA的执行次数以及这三部分所占的比例。

表6.2中，la指令数表示la的执行次数，若要计算实际执行的指令数，还应该再乘以la指令序列的长度。除了执行生成代码，Java 虚拟机还需要执行其他代码，比如编译线程、垃圾收集等，因此表6.2中la指令的比例要低于表6.1，但是仍然占有很大比例。由表6.2可以看出，derby的LA执行频度非常高，这是因为derby是一个数据库测试程序，对数据访问、对象分配的执行比较多。大部分科学计算程序的LSLA的比例比较高，比如scimark.lu.large，这是因为科学计算程序评测的是程序进行密集的科学计算时的访存模式，会较多地访存[117]。

6.3.2　地址分布

我们对64位Java虚拟机执行生成代码时，所有加载的地址的分布进行了统计。图6.1显示了其中4个测试程序的地址分布。

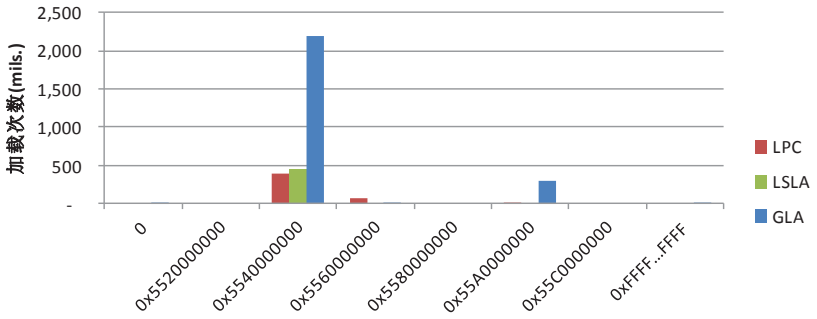
由图6.1可见，各个测试程序加载的地址分布趋势大体相同，加载的地址都非常集中地分布在两个区域，其他区域的地址几乎没有加载。比如compiler.sunflow，第一个区域的地址范围从0x5520000000至0x5560000000，第二个

表 6.2 la指令分类

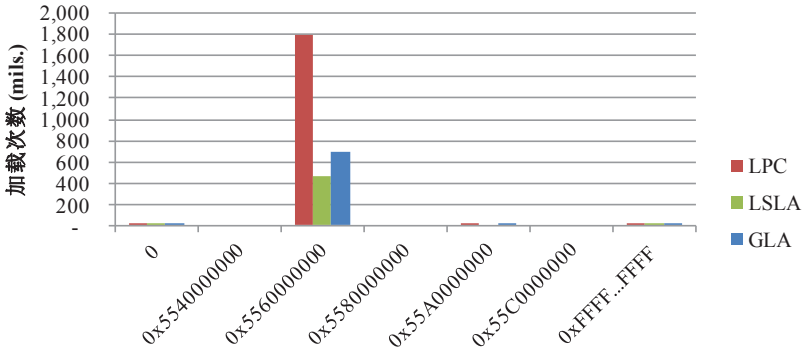
测试程序	指令总数 (Mil.)	la指令数 (Mil.)	la指令比例 (%)	加载地址		
				LPC (%)	LSLA (%)	GLA (%)
compiler.compiler	101,000	3,060	3.02	14.23	13.73	72.03
compiler.sunflow	104,000	3,333	3.21	13.78	13.35	72.88
compress	187,000	7,480	4.01	8.67	8.77	82.56
crypto.aes	257,000	13,835	5.38	3.10	2.93	93.96
crypto.rsa	230,000	2,729	1.19	60.25	15.57	24.17
crypto.signverify	212,000	6,090	2.87	15.84	9.00	75.16
derby	299,000	24,014	8.03	10.53	10.42	79.04
mpegaudio	206,000	13,195	6.40	2.59	3.50	93.91
scimark.fft.large	72,000	1,715	2.39	26.67	25.89	47.45
scimark.lu.large	188,000	12,012	6.41	48.41	48.30	3.29
scimark.sor.large	222,000	2,598	1.17	43.59	43.15	13.27
scimark.sparse.large	198,000	4,968	2.51	46.65	46.42	6.95
scimark.fft.small	231,000	1,899	0.82	19.55	19.38	61.05
scimark.lu.small	152,000	1,934	1.27	38.62	38.16	23.22
scimark.sor.small	212,000	1,937	0.91	42.23	41.61	16.18
scimark.sparse.small	136,000	1,934	1.42	45.38	44.67	9.94
scimark.monte_carlo	201,000	8,565	4.26	3.70	3.64	92.67
serial	139,000	3,061	2.21	18.86	15.68	65.47
sunflow	321,000	5,756	1.80	13.85	14.09	72.05
xml.transform	179,000	5,777	3.23	18.55	14.07	67.37
xml.validation	92,000	1,239	1.34	14.61	13.32	72.11
avg.			3.5	23.25	20.69	56.06

区域从0x5580000000至0x55A0000000。LPC、LSLA和GLA在第一个区域均有较多的分布，这个区域主要是Java虚拟机的方法区[2]；第二个区域基本全部由GLA构成，并且数量要远低于第一个区域，这部分区域是Java虚拟机的堆区。对于不同的测试程序，地址分布又有一些差别：在compiler.sunflow中，程序加载堆区的地址较多；在cypto.rsa中，程序加载PC的比例较高，加载堆区的地址特别少；在derby中，加载地址次数非常多，会达到30G次以上；在mpeg中，GLA的比例非常高。这些特点与表6.2也是相吻合的。

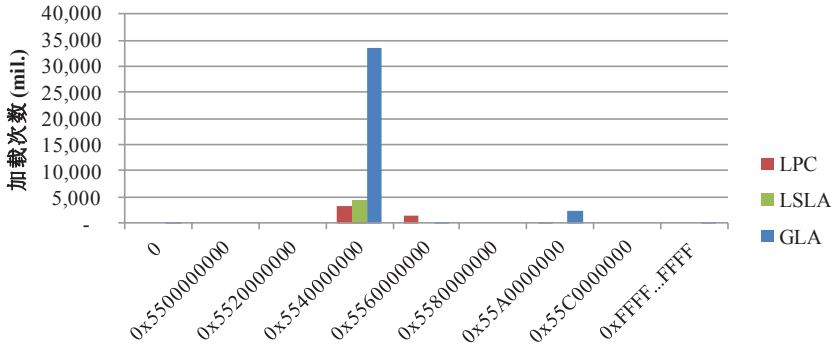
在地址分布统计中，会出现一些地址为-1的情况，在这4个测试程序中分别出现了269000、462000、229000和218000次。地址出现-1的情况是因为在即时编译器在编译方法调用的字节码时，会先将跳转的目标地址填写为-1，在解析出目标地址后再对跳转地址进行修改。



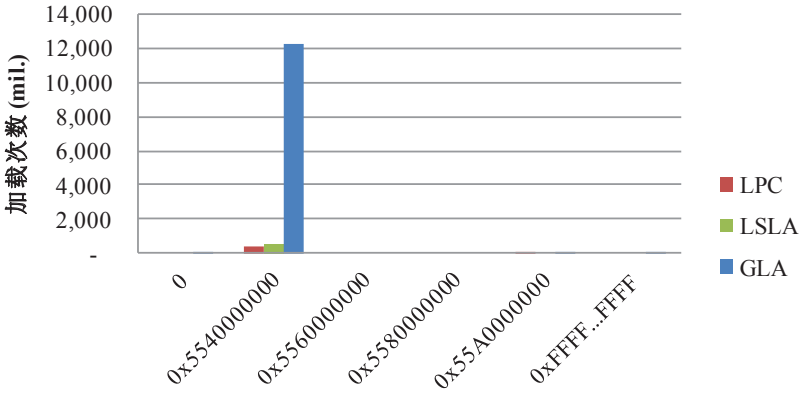
(a) compiler.sunflow地址分布



(b) crypto.rsa地址分布



(c) derby地址分布

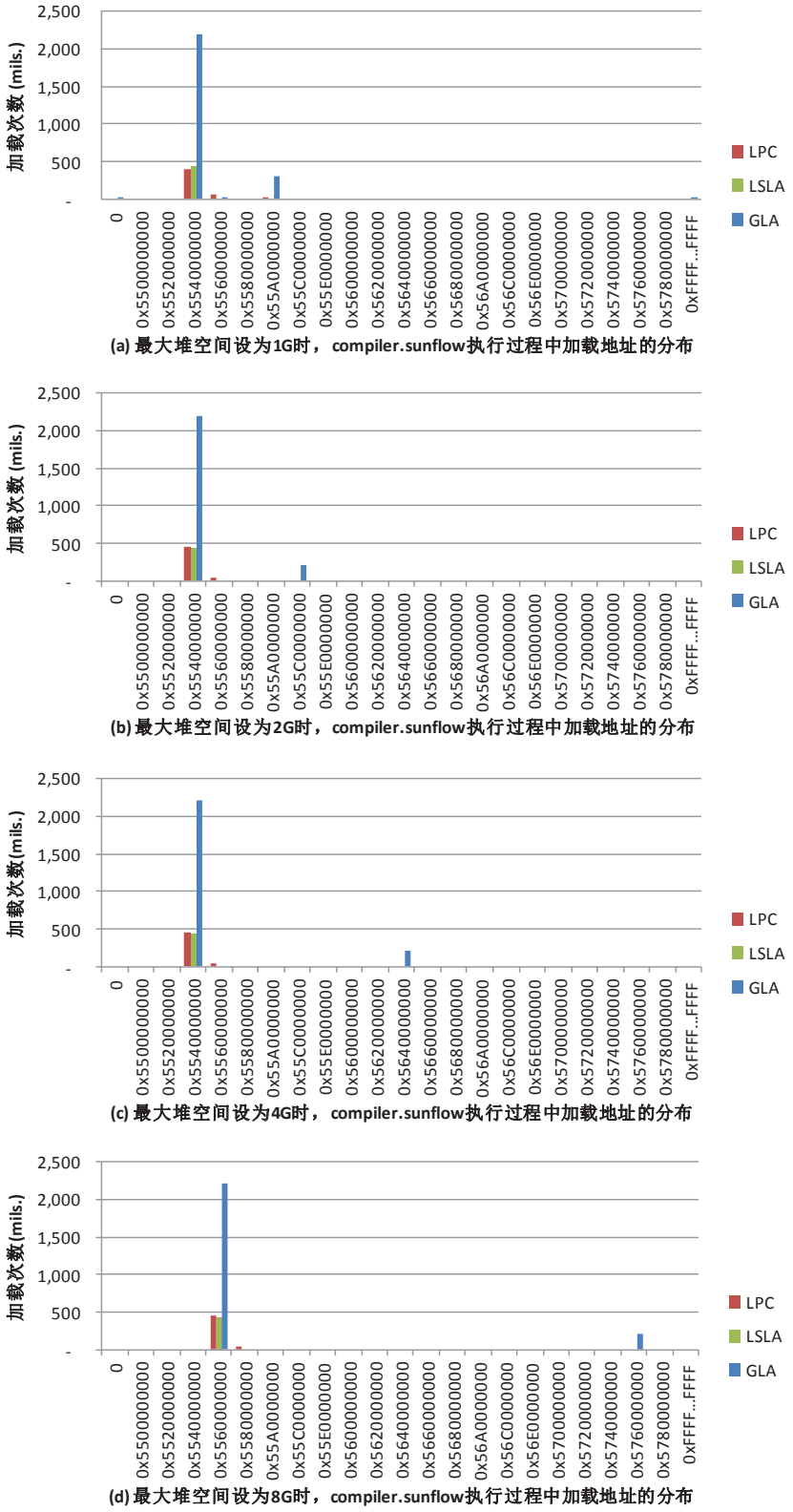


(d) mpegaudio地址分布

图 6.1　不同测试程序的地址分布

Java虚拟机的堆设置为不同大小，会对地址的分布产生影响。以compiler.sunflow为

例，最大堆分别设置为1G、2G、4G和8G时，地址的分布如图6.2所示。由图6.2可见，不同的堆大小，对于方法区的地址分布没有影响，对于堆空间的地址分布产生了平移的影响。其他测试程序结果类似，本章不一一列出。



(a) 最大堆空间设为1G时，compiler.sunflow执行过程中加载地址的分布

(b) 最大堆空间设为2G时，compiler.sunflow执行过程中加载地址的分布

(c) 最大堆空间设为4G时，compiler.sunflow执行过程中加载地址的分布

(d) 最大堆空间设为8G时，compiler.sunflow执行过程中加载地址的分布

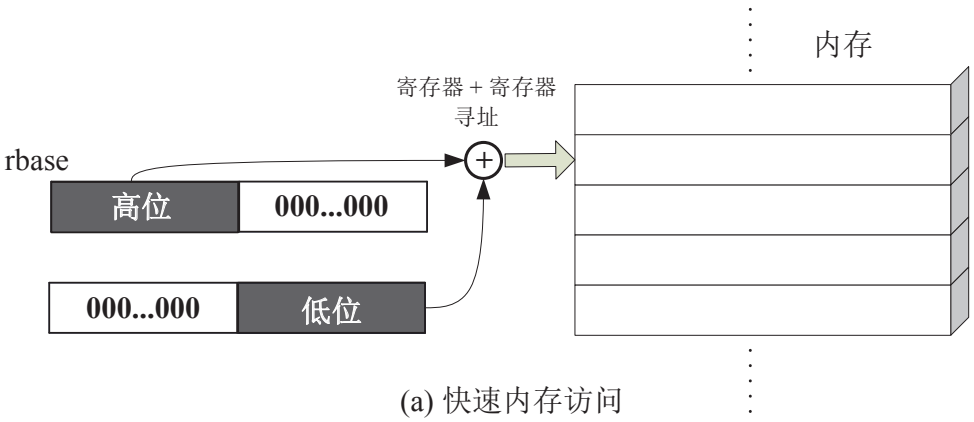
图 6.2 不对堆大小的地址分布

6.4 快速内存访问

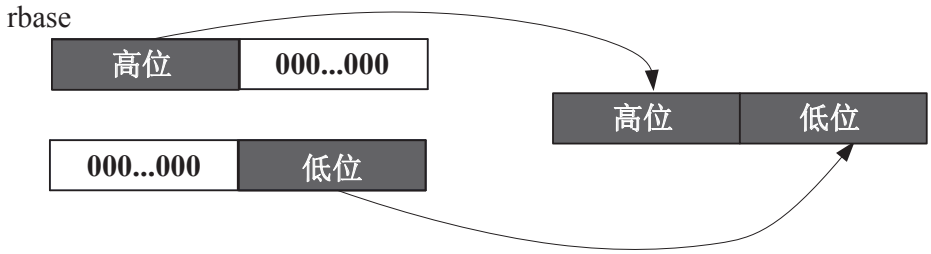
6.4.1 概述

在加载了内存地址之后，很多情况下会对该地址对应的内容进行读或写操作，也就是上文提到的LSLA。针对这一种情况，我们通过在硬件上实现双寄存器访存方式，实现了减少访存指令数的目的。

通过6.3.2节地址分布的实验我们发现，在Java虚拟机生成代码的执行过程中，大部分地址的高32位都是相同的。因此，我们选择一个通用寄存器作为全局寄存器rbase来保存目标地址的高32位。需要访存时，在一个临时寄存器rt中加载目标地址的低32位，然后以rbase和rt作为参数，使用一条双基址寄存器访存指令就可以完成对目标地址的访问。通过该方法，每次访存之前只需要加载目标地址的低32位即可，这基本接近了32位Java虚拟机的访存开销。图6.3(a)给出了快速内存访问的基本原理。



(a) 快速内存访问



(b) 加载程序地址

图 6.3　访存和加载程序地址的算法示意图

该方法只适用于LSLA，无法适用于LPC和GLA，因为后两种方式并不访问内存。对于后两种地址加载，本章将通过纯软件方式进行优化，具体方法在6.5节给出。

由于双基址寻址方式的目标地址为rbase和rt符号位扩展后相加的结果，而rt中只加载了低32位地址，因此这种方式适用于访问rbase的正、负2G空间之内的地址。我们将这种访存方式应用于Java虚拟机的方法区，由6.3.2节地址分布实验可见，Java虚拟机方

法区分布集中、体积较小而且分布不随堆大小的改变而改变。在Java程序运行时可以保证加载的地址落在rbase的偏移范围内。对于堆空间的访问，当堆空间较小时，可以使用这种方式访问堆空间，但是在堆空间设置的较大时则可能需要在访存前更改rbase中的值，这就增加了访存代价。加之大部分LSLA均发生在方法区（见图6.1和图6.2），故本章仅对Java虚拟机方法区的访存使用了双基址的优化方式。

该方法需要改造Java虚拟机的模板表和即时编译器，使程序运行过程中尽量避免使用rbase，从而保证在访存时rbase中保存着正确的值。因为MIPS处理器的通用寄存器数量相对较多，选取一个寄存器用作全局寄存器并不会对整个系统带来较大的开销。如果不能避免要使用rbase（更改寄存器中的值），则需要在每次使用rbase之后恢复rbase中的值，在我们的方法中，使用两条指令即可完成rbase的恢复。

6.4.2 实现细节

龙芯GS464处理器核实现了“(base) + (index) + disp8”的定点、浮点内存存取的双基址寻址方式[100]。新加入的指令有：gsLWB、gsLHX、gsLWX、gsLDX、gsLWXC1、gsLDXC1、gsSBX、gsSHX、gsSWX、gsSDX、gsSDXC1。指令格式为（以读字节为例）：

```
gsLWX base, rd, rt, off
```

实现功能为：GPR[rd] ← memory[GPR[base]+GPR[index]+offset]。先将带符号的8位offset和寄存器base中的内容以及寄存器index中的内容相加，得到有效地址，再根据这个对齐的有效地址从内存中取出8位数据，将这个数据进行符号位扩展后存到寄存器rd中。

以读取一个32位定点数据为例，龙芯Java虚拟机即时编译器生成的访存指令序列如图6.4所示。由此可见，该方法大幅减少了64位访存指令序列的长度，优化后的访存序列只用了3条指令。不仅减小了生成代码的体积，降低内存消耗，还提高了程序性能。

我们选择MIPS中的Callee Saved寄存器作为rbase，这样在Java虚拟机调用C函数（本地方法调用）之后不需恢复rbase。在Java虚拟机生成代码的执行过程中，若更改了寄存器rbase中的值，在使用完之后需要调用函数reinit_rbase()来恢复rbase中原来的值（0x5500000000）。reinit_rbase()实现方式在图6.4(d)中列出。

```
lui   rt, hi (addr) //高16位
lw    rd, lo (addr) (rt)//低16位
```

(a) 32位Java虚拟机

```
lui   rt, hi (addr >> 32)
ori   rt, rt, lo (addr >> 32)
dsll rt, rt, 16
ori   rt, rt, hi (addr)
dsll rt, rt, 16
lw    rd, lo (addr) (rt)
```

(b) 原始的64位Java虚拟机

```
lui   rt, hi (addr)
ori   rt, rt, lo(addr)
gsLWX  rbase, rd, rt, 0
```

(c) 快速内存访问

```
dadd       rd, zero, 0x55
dsll.32    rd, 0
```

(d) reinit_rbase函数

图 6.4　优化前后的访存指令序列

6.5　加载地址优化

本节我们通过纯软件的方式实现了对LPC和GLA的优化。这两种优化方式是对快速内存访问的补充，同时，这两种优化方式独立于快速内存访问优化，在没有双寄存器寻址的支持下也可以使用。下面分别介绍这两种优化方法。

6.5.1　LPC优化

对于加载程序地址，即LPC，也可以利用rbase进行优化。加载地址时，先加载目标地址的低32位至寄存器，结果再与rbase相加即可完成地址的加载。优化后的LPC的过程如图6.3(b)所示。该方式同双基址访存原理相同，都是加载低32位地址再与rbase相加。由于程序地址均位于方法区中，堆中的地址均是数据地址，因此Java虚拟机中所加载的程序地址都位于方法区（见6.3.2节地址分布实验），使用这种方式可以保证程序的正确性。对于GLA，只有在地址位于rbase正负2G空间内的时候才可以使用这种方式。有些情况下GLA所加载的地址在编译时并不能确定位于哪个区域，因此本章没有对GLA使用这种优化方式。

6.5.2　GLA优化

通过表6.2可见，GLA在各个测试程序的运行过程中都占有较大比例，但是由于GLA加载的地址经常会出现在堆区，超出rbase的正负2G的范围，因此不能使用rbase进行优化。针对GLA情况，本章使用了一种优化的地址计算方式。在Java虚拟机中，我们发现并不是所有地址位都得到了使用，因此，在加载地址的时候我们只需要载入地址的有效位即可。更进一步的，如6.1和图6.2所示，在Java虚拟机中几乎所有被加载地址的高位均为0，因此我们选择地址的低48位作为有效位，这样既可以保证

足够的访存空间，又可以使用尽量少的指令数。加载地址时，使用3条指令分别加载目标地址的高、中、低三部分（3个16位数），再加上1条移位指令，就可以完成一个48位地址的加载。这样使用4 条指令就可以完成一个地址的加载。这个地址既可以是堆中的地址，也可以是方法区中的地址。

在Java虚拟机初始化的过程中，我们对地址空间进行相应的控制，以保证地址的分配符合本章的要求。在6.3.2节实验中提到，会加载一部分-1 地址。因为我们方法中加载的地址是符号位扩展，所以可以保证加载-1时也是正确的。 LPC和GLA的具体实现细节见图6.5。通过优化，本章将LPC由6条指令优化至3条指令；将GLA由6条指令优化至4条指令。

```
lui       rt, hi (addr >> 32)
ori       rt, rt, lo (addr >> 32)
dsll      rt, rt, 16
ori       rt, rt, hi (addr)
dsll      rt, rt, 16
ori       rt, rt, lo (addr)
```

(a) 加载64位地址（优化前）

```
lui       rt, hi (addr)
ori       rt, rt, lo(addr)
dadd      rt, rbase, rt
```

(b) LPC优化

```
lui       rd, lo (addr >> 32)
ori       rd, rd, hi (addr)
dsll      rd, rd, 16
ori       rd, rd, lo (addr)
```

(c) GLA优化

图 6.5 优化前后加载地址指令序列

6.6 实验评测

为了评测本章方法的效果，我们使用了测试程序SPECjvm2008和Dacapo对方法的性能效果和内存效果进行了评测。

6.6.1 实验平台介绍

硬件平台

我们的硬件平台是龙芯3A处理器，2G物理内存，操作系统为Linux 2.6.36。龙芯3A处理器是一款面向服务器应用以及科学计算的多核处理器，采用与MIPS64兼容的RISC处理器核。龙芯3A处理器片内集成4个64位四发射超标量GS464高性能处理器核，每个GS464包含两个定点运算单元、两个浮点运算单元和一个访存部件，支持Non-Blocking Cache及Load-Speculation等访存优化技术，使用64K一级数据Cache和64K一级指令Cache，均为4路组相连，使用写回策略。龙芯3A拥有4M二级片上Cache，多个GSG464核以及二级Cache模块通过AXI互联网络形成一个分布式共享二级Cache的多核结构。龙芯3A的指令系统与MIPS64兼容，并支持龙芯扩展指令集。

Java虚拟机

OpenJDK[2], [15]是Sun公司为Java平台构建的Java开发环境的开源版本。Sun公司在2006年对Java 开放源代码，于2009年正式发布OpenJDK。OpenJDK是最主流的Java虚拟机开源平台之一，它的核心是Java Hotspot VM。Java程序首先由解释器开始执行，当检测到有热点方法或热点循环时，即Hotspot，会将热点调度至即时编译器（Just-In-Time Compiler，简称JIT Compiler），将热点字节码编译为本地代码，即生成代码（Generated Code）。Hotspot VM包含了两个JIT Compiler：Client Compiler和Server Compiler。Client Compiler主要用于桌面应用，兼顾执行效率和编译时间，在响应时间和执行速度上获得一个平衡；Server Compiler主要用于长时间运行的服务器程序，具有较高的优化程度和较高的编译代价。

我们完成了OpenJDK6在龙芯平台上的移植工作，包括32位和64位版本的解释器, Client Compiler和Server Compiler。我们移植的版本基于OpenJDK6 Build 14.0 b16。本章采用的实验环境为64位Client Compiler。

6.6.2 性能测试

在性能评测中，我们分别使用不同的优化方法对测试程序进行了性能测试。具体的优化配置如下：

- 基准测试：所有优化方法均不使用；
- Fast Memory Access：只是用双寄存器寻址方式优化；
- Software Support：只使用LPC和GLA优化；
- Both：所有优化方式均使用。

图6.6显示了在使用不同的优化方式下64位Java虚拟机执行SPECjvm2008的结果。堆大小为1G，单线程执行。y轴是同基准测试相比较的结果。

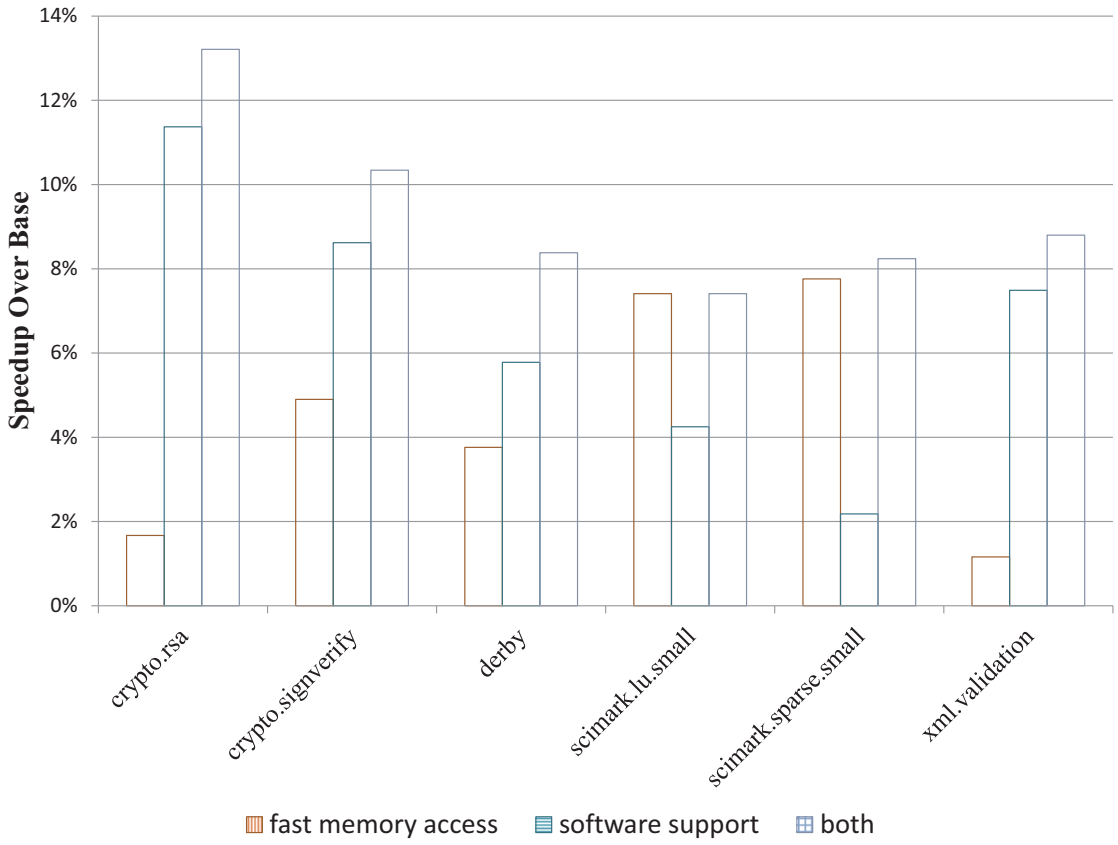


图 6.6 SPECjvm2008性能评测

由图6.6可见，crypto.rsa的提升比较明显，达到13.21%。双寄存器访存优化对于scimark.lu.small和scimark.sparse.small提升效果较明显，这是因为这两个测试程序中LSLA的比例较高。其他测试程序则是LPC和GLA的优化效果更为明显。双寄存器访存、软件优化和同时使用所有优化分别使SPECjvm2008的总分提升了2.6%、4.5%和5.1%。

我们还对Dacapo进行了评测，评测结果在图6.7中给出。由图6.7可见，双寄存器访存对avrora产生了轻微的负向的结果，这可能是因为avrora中LSLA的情况很少，优化效果不足以弥补占用了一个全局寄存器所产生的代价。方法对xalan的优化效果最明显，提高了23.9%。这是因为xalan执行时分配对象的数量要远多于其他测试用例，因此会产生大量访存和取址代码[144]。双寄存器访存、软件优化和同时使用所有优化分别使Dacapo平均提升了4.4%、6.0%和7.3%。

通过性能评测我们发现，有的测试程序通过双寄存器访存获得更大的性能提升而有些则是通过LPC和GLA优化。从整体看，双寄存器访存的优化效果要低于软件方式，这是因为在大部分测试程序中，LSLA的情况要远少于LPC和GLA的和。个别用例使用双寄存器访存会产生负向结果，而使用LPC优化和GLA优化的效果都是正向的。

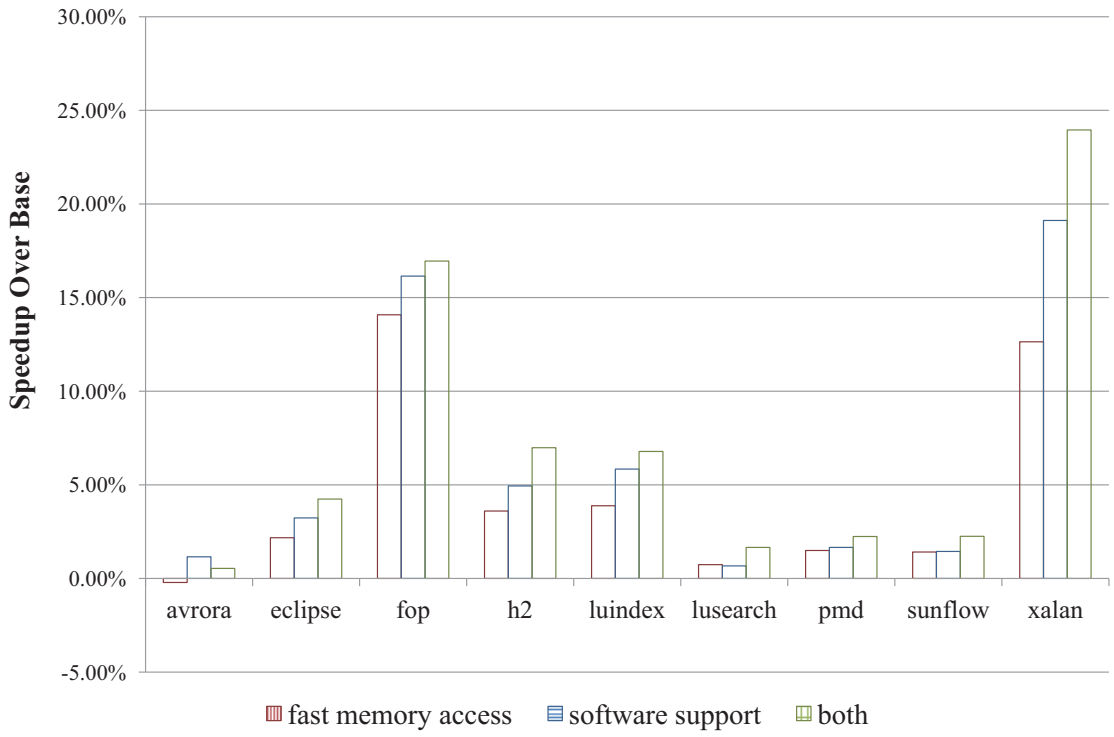


图 6.7　Dacapo性能评测

6.6.3　内存消耗测试

在6.3.1节中，我们统计了la指令在模板表中的比例和la指令在各测试程序生成代码中执行频率最高的5个方法中所占的比例。本节统计了优化后la指令所占的比例。在表6.3中可以看到，解释器模板表中la指令的比例由41.2%降低到31.8%，模板表指令数由40848 降至35236，降低了13.8%。编译器生成代码中la指令的平均比例由52.6%降低为39.6%，生成代码的平均体积由原来的2285条指令降低为1930条指令，降低了16%。

表 6.3　优化前后la指令数和比例

测试程序	64位（优化前）			64位（优化后）		
	总指令数	la指令数	%	总指令数	la指令数	%
compiler.sunflow	563	258	45.8	467	168	36.0
compress	3198	1836	57.4	2892	1200	41.5
crypto.aes	1961	924	47.1	1733	620	35.8
derby	2632	1470	55.9	2111	932	44.1
mpegaudio	4220	2352	55.7	3400	1536	45.2
scimark.fft.large	1763	912	51.7	1581	540	34.2
sunflow	1660	906	54.6	1327	536	40.4
compiler avg.	2285	1237	52.6	1930	790	39.6

6.7 本章小结

本章提出了一种从硬件和软件两方面优化MIPS 64位Java虚拟机的访存和取址的方法，减少了生成代码的指令数，提高了程序性能。本章使用一个全局寄存器来保存地址的高位，通过在硬件上实现双基址寄存器指令，简化了Java虚拟机方法区的访存过程；优化了PC的加载过程，指令数基本接近32 位系统。对于堆空间，本章提出了一种软件方式来优化取址。

我们在OpenJDK6上实现了我们的方法，并在龙芯平台上做了测试。最终的评测结果显示，我们的方法可以使SPECjvm2008整体性能提升5.1%，使Dacapo整体提升7.3%，个别用例性能提升近25%。在内存消耗方面，本方法可以使Java虚拟机编译器生成代码的体积减少16%。

第七章　总结及未来工作

近年来，随着集成电路制造工艺的持续发展，现代计算机系统的处理器性能和存储性能均得到了大幅度提高。但同时，现代计算机系统也正面临着越来越严峻的来自软错误的威胁。一些应用于特殊环境下的计算机系统，比如航天设备，更是面临着来自宇宙射线等极端条件所导致的挑战。因此，针对软错误的错误检测与错误恢复技术变得愈发重要。在软件层面实现容错技术，相对于硬件层面，具有成本低、灵活性好等特点。在Java 虚拟机层面进行容错由于其移植性好等特点具有很大的优势。但是从软件层面实现容错技术往往会给计算机系统带来高额的性能开销，在Java虚拟机层面进行容错亦是如此。本文通过对Java虚拟机进行研究和改进，结合软件再生、动态污点分析等技术，在保证系统性能的前提下，实现了从软件层面提高计算机系统的可靠性。再次基础上，本文进一步提出了Java虚拟机访存和取址的优化方法。同时，实现一个高可靠、高性能的Java虚拟机对于以龙芯为代表的国产处理器破除x86体系壁垒、丰富软件环境、提高国产计算机系统可靠性均具有重要的现实意义。

7.1　本文工作总结

本文主要工作如下：

1. 提出并实现了一种针对Java虚拟机控制流的软错误恢复方法。该方法结合软件再生技术，以Java虚拟机进行软件再时所面临的问题为出发点，利用Java虚拟机模块化的特点，实现了一种预防式的、主动式的容错技术。该方法在Java运行过程中，主动地进行软件再生过程，从而消除Java虚拟机内有可能存在的软错误。通过该方法可以实现对Java虚拟机方法区中的任意多位的软错误进行恢复。该方法利用Java虚拟机模块间以及模块内部良好的局部性特点，把软件再生过程进行拆分和优化，使得软件再生可以分步骤进行、并行进行且并发进行，大幅度提高了软件再生的吞吐率，降低了软件再生对Java的响应时间和程序性能的影响。该方法与其他容错技术可同时使用，作为其他容错技术的预防和补充。实验显示，该方法在基本不影响Java虚拟机性能的前提下，有效的提升了Java虚拟机的可靠性。
2. 提出了并实现了一种针对Java虚拟机数据流的软错误恢复方法。该方法结合动态污点分析技术，对Java虚拟机栈中和堆中的所有数据进行动态追踪和分析，对符合校验条件的数据进行错误检测。该方法利用数据的时间信息和来源信息，对数据进行有差别的检查，从而消除了一些不必要的性能损失。针对该方法，本文还提出了一系列优化方法，包括短生命期数据优化、更新校验码优化、栈顶

优化以及获取当前时间优化，这些优化能够大幅降低错误检测对Java虚拟机造成的性能影响。实验显示，在校验阈值设为15秒时，该方法对Java虚拟机产生的性能代价不超过20%，该方法对堆空间中分配的对象累积体积平均增加10%，对最小栈空间需求增加了不超过17%。

3. 提出了一种基于龙芯处理器的64位Java虚拟机的访存和取址优化方法。64位MIPS系统在进行地址加载时需要多条指令才能完成，这影响了Java 虚拟机的生成代码的执行效率。针对这一问题，本文利用双基址访存指令，对64位Java虚拟机方法区的取址并访存过程实现了优化。该优化可以使访存指令数接近32位Java虚拟机。对于其他地址加载过程，本文提出了纯软件方式的优化办法。最终的评测结果显示，该方法可以使SPECjvm2008整体性能提升5.1%，使Dacapo整体提升7.3%，个别测试程序性能提升近25%。在内存消耗方面，该方法可以使Java虚拟机编译器生成代码的体积减少16%。

7.2 下一步研究方向

针对Java虚拟机的容错技术和性能优化技术需要大量的研究工作和工程投入，本文开展了一些初步的研究工作，以下几方面的问题值得在今后的工作中进行进一步深入的研究：

1. 本文第三章提出的Java虚拟机热再生方法仅实现了对方法区的再生，未来可以进行对堆和栈中的数据进行一定程度的再生过程，以实现对包括数据流在内的整个Java虚拟机进行软错误恢复。
2. 本文第四章提出的CST方法，采用的校验方式为校验和的方式，即按位异或。未来可以进行其他编码方式的校验，比如实现具有错误恢复功能的编码，比如实现Arithmetic Code，这样可以在数据计算时无需进行编解码操作，提升程序的效率。
3. 进一步进行主动性、预防性容错技术的研究，比如在Java虚拟机中使用冗余的数据，在Java程序运行负载不高的时候，对冗余的数据内容进行错误检测。这种方式有别于在访问时才进行错误检测的方式，可以提前发现错误并进行恢复，可以降低程序的响应时间。
4. 进一步进行对局部数据或代码进行容错的研究，比如在Java虚拟机中增加异常处理、变量声明、线程声明等关键字，实现对部分重要的异常、变量或者线程进行高错误覆盖率的错误检测。

参考文献

[1] WEAVER C, EMER J, MUKHERJEE S S, et al. Techniques to reduce the soft error rate of a high-performance microprocessor[C]//ACM SIGARCH Computer Architecture News. .[S.l.]: [s.n.] , 2004，32:264.

[2] KOTZMANN T, WIMMER C, MÖSSENBÖCK H, et al. Design of the Java HotSpot Client Compiler for Java 6[J]. ACM Transactions on Architecture and Code Optimization (TACO), 2008, 5(1):7.

[3] BORKAR S. Designing reliable systems from unreliable components: the challenges of transistor variability and degradation[J]. Micro, IEEE, 2005, 25(6):10–16.

[4] FAROOQ M, IQBAL M W, RANA T A, et al. Comparative Analysis Of Fault-Tolerance Techniques For Space Applications[J]. VFAST Transactions on Software Engineering, 2014, 3(2):1–10.

[5] TABER A, NORMAND E. Single event upset in avionics[J]. IEEE Transactions on Nuclear Science (Institute of Electrical and Electronics Engineers);(United States), 1993, 40(2).

[6] MUKHERJEE S. Architecture design for soft errors[M].[S.l.]: Morgan Kaufmann, 2011.

[7] LINDHOLM T, YELLIN F, BRACHA G, et al. The Java virtual machine specification[M].[S.l.]: Pearson Education, 2014.

[8] STILKERICH M, THOMM I, WAWERSICH C, et al. Tailor-made JVMs for statically configured embedded systems[J]. Concurrency and Computation: Practice and Experience, 2012, 24(8):789–812.

[9] ULBRICH P, KAPITZA R, HARKORT C, et al. I4Copter: An adaptable and modular quadrotor platform[C]//Proceedings of the 2011 ACM Symposium on Applied Computing. .[S.l.]: [s.n.] , 2011:380–386.

[10] BURKE M G, CHOI J D, FINK S, et al. The Jalapeno dynamic optimizing compiler for Java[C]//Proceedings of the ACM 1999 conference on Java Grande. .[S.l.]: [s.n.] , 1999:129–141.

[11] CIERNIAK M, LUEH G Y, STICHNOTH J M. Practicing JUDO: Java under dynamic optimizations[C]//ACM SIGPLAN Notices. .[S.l.]: [s.n.] , 2000，35:13–26.

[12] ISHIZAKI K, KAWAHITO M, YASUE T, et al. Design, implementation, and evaluation of optimizations in a just-in-time compiler[C]//Proceedings of the ACM 1999 conference on Java Grande. .[S.l.]: [s.n.] , 1999:119–128.

[13] ISHIZAKI K, KAWAHITO M, YASUE T, et al. A study of devirtualization techniques for a Java Just-In-Time compiler[C]//ACM SIGPLAN Notices. .[S.l.]: [s.n.] , 2000，35:294–310.

[14] SUGANUMA T, OGASAWARA T, TAKEUCHI M, et al. Overview of the IBM Java just-in-time compiler[J]. IBM systems Journal, 2000, 39(1):175–193.

[15] PALECZNY M, VICK C, CLICK C. The Java hotspotTM Server Compiler[C]//Proceedings of the 2001 Symposium on Java TM Virtual Machine Research and Technology Symposium-Volume 1. .[S.l.]: [s.n.] , 2001:1–1.

[16] CHEN D, MESSER A, BERNADAT P, et al. JVM Susceptibility to Memory Errors.[C]//Java Virtual Machine Research and Technology Symposium. .[S.l.]: [s.n.] , 2001:67–78.

[17] HOLIDAY M R. Fault-tolerant java virtual machine[P].[S.l.]: Google Patents. US Patent 6,421,739.

[18] FRIEDMAN R, KAMA A. Transparent fault-tolerant Java virtual machine[C]//Reliable Distributed Systems, 2003. Proceedings. 22nd International Symposium on. .[S.l.]: [s.n.] , 2003:319–328.

[19] NAPPER J, ALVISI L, VIN H. A fault-tolerant java virtual machine[C]//2013 43rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). .[S.l.]: [s.n.] , 2003:425–425.

[20] CHEN G, KANDEMIR M. Improving java virtual machine reliability for memory-constrained embedded systems[C]//Proceedings of the 42nd annual Design Automation Conference. .[S.l.]: [s.n.] , 2005:690–695.

[21] CHEN G, KANDEMIR M, VIJAYKRISHNAN N, et al. Analyzing heap error behavior in embedded JVM environments[C]//Proceedings of the 2nd IEEE/ACM/IFIP international conference on Hardware/software codesign and system synthesis. .[S.l.]: [s.n.] , 2004:230–235.

[22] STILKERICH I, STROTZ M, ERHARDT C, et al. A JVM for soft-error-prone embedded systems[C]//ACM SIGPLAN Notices. .[S.l.]: [s.n.] , 2013，48:21–32.

[23] REINHARDT S K, MUKHERJEE S S. Transient fault detection via simultaneous multithreading[M]. Vol. 28.[S.l.]: ACM, 2000.

[24] SOMERS J, DIRECTOR F, GRAHAM S. Stratus ftServer–Intel Fault Tolerant Platform[C]//Rap. tech. Intel Developer Forum, Fall. .[S.l.]: [s.n.] , 2002.

[25] WOOD A, JARDINE R, BARTLETT W. Data integrity in HP nonstop servers[C]//Workshop on SELSE. .[S.l.]: [s.n.] , 2006.

[26] OH N, SHIRVANI P P, MCCLUSKEY E J. Error detection by duplicated instructions in super-scalar processors[J]. Reliability, IEEE Transactions on, 2002, 51(1):63–75.

[27] REIS G A, CHANG J, VACHHARAJANI N, et al. SWIFT: Software implemented fault tolerance[C]//Proceedings of the international symposium on Code generation and optimization. .[S.l.]: [s.n.] , 2005:243–254.

[28] REIS G A, CHANG J, AUGUST D I, et al. Configurable transient fault detection via dynamic binary translation[C]//IN: PROCEEDINGS OF THE 2ND WORKSHOP ON ARCHITECTURAL RELIABILITY. .[S.l.]: [s.n.] , 2006.

[29] REIS G, CHANG J, VACHHARAJANI N, et al. Design and evaluation of hybrid fault-detection systems[C]//Computer Architecture, 2005. ISCA'05. Proceedings. 32nd International Symposium on. .[S.l.]: [s.n.] , 2005:148–159.

[30] SODANI A, SOHI G S. Dynamic instruction reuse[M]. Vol. 25.[S.l.]: ACM, 1997.

[31] GOMAA M, VIJAYKUMAR T, et al. Opportunistic transient-fault detection[C]//Computer Architecture, 2005. ISCA'05. Proceedings. 32nd International Symposium on. .[S.l.]: [s.n.] , 2005:172–183.

[32] PETERSON W W, BROWN D T. Cyclic codes for error detection[J]. Proceedings of the IRE, 1961, 49(1):228–235.

[33] PETERSON W W, WELDON E J. Error-correcting codes[M].[S.l.]: MIT press, 1972.

[34] AVIZIENIS A. Arithmetic algorithms for error-coded operands[J]. Computers, IEEE Transactions on, 1973, 100(6):567–572.

[35] IACOBOVICI S. Residue-based error detection for a shift operation[P].[S.l.]: Google Patents. US Patent 7,543,007.

[36] LO J C. Reliable floating-point arithmetic algorithms for error-coded operands[J]. Computers, IEEE Transactions on, 1994, 43(4):400–412.

[37] WEBB C. z6-The Next-Generation Mainframe Micropro cessor[J]. Hot Chips, August, 2007.

[38] NOUFAL I A, NICOLAIDIS M. A CAD framework for generating self-checking multipliers based on residue codes[C]//Proceedings of the conference on Design, automation and test in Europe. .[S.l.]: [s.n.] , 1999:29.

[39] FURUYA K, AKITA Y, TOHMA Y. Logic design of fault-tolerant dividers based on data complementation strategy[C]//Proc. 13th Symp. Fault-Tolerant Comput. .[S.l.]: [s.n.] , 1983:306–313.

[40] LO J C, THANAWASTIEN S, RAO T. Concurrent error detection in arithmetic and logical operations using Berger codes[C]//Computer Arithmetic, 1989., Proceedings of 9th Symposium on. .[S.l.]: [s.n.] , 1989:233–240.

[41] TIPSUWANPORN V, RODKAMTUI J. Fault tolerant control system in critical process based on Ethernet network[C]//Robotics, Automation and Mechatronics, 2006 IEEE Conference on. .[S.l.]: [s.n.] , 2006:1–4.

[42] BERNICK D, BRUCKERT B, VIGNA P D, et al. NonStop® advanced architecture[C]//Dependable Systems and Networks, 2005. DSN 2005. Proceedings. International Conference on. .[S.l.]: [s.n.] , 2005:12–21.

[43] AVIZIENIS A. The N-version approach to fault-tolerant software[J]. IEEE Transactions on software engineering, 1985(12):1491–1501.

[44] RANDELL B, XU J. The evolution of the recovery block concept[J]. Software Fault Tolerance, 1995, 1:21.

[45] ELNOZAHY E N, ALVISI L, WANG Y M, et al. A survey of rollback-recovery protocols in message-passing systems[J]. ACM Computing Surveys (CSUR), 2002, 34(3):375–408.

[46] LEE I, BASOGLU M, SULLIVAN M, et al. Survey of error and fault detection mechanisms[J]. University of Texas at Austin, Tech. Rep, 2011.

[47] AUSTIN T M. DIVA: A reliable substrate for deep submicron microarchitecture design[C]//Microarchitecture, 1999. MICRO-32. Proceedings. 32nd Annual International Symposium on. .[S.l.]: [s.n.] , 1999:196–207.

[48] SLEGEL T J, AVERILL III R M, CHECK M, et al. IBM's S/390 G5 microprocessor design[J]. Micro, IEEE, 1999, 19(2):12–23.

[49] MUKHERJEE S S, KONTZ M, REINHARDT S K. Detailed design and evaluation of redundant multi-threading alternatives[C]//Computer Architecture, 2002. Proceedings. 29th Annual International Symposium on. .[S.l.]: [s.n.] , 2002:99–110.

[50] KANAWATI G A, NAIR V S, KRISHNAMURTHY N, et al. Evaluation of integrated system-level checks for on-line error detection[C]//Computer Performance and Dependability Symposium, 1996., Proceedings of IEEE International. .[S.l.]: [s.n.] , 1996:292–301.

[51] HANGAL S, VAHIA D, MANOVIT C, et al. TSOtool: A program for verifying memory systems using the memory consistency model[C]//ACM SIGARCH Computer Architecture News. .[S.l.]: [s.n.] , 2004，32:114.

[52] REBAUDENGO M, REORDA M S, TORCHIANO M, et al. Soft-error detection through software fault-tolerance techniques[C]//Defect and Fault Tolerance in VLSI Systems, 1999. DFT'99. International Symposium on. .[S.l.]: [s.n.] , 1999:210–218.

[53] BALASUBRAMANIAN V, BANERJEE P. Compiler-assisted synthesis of algorithm-based checking in multiprocessors[J]. IEEE Transactions on Computers, 1990(4):436–446.

[54] AL-YAMANI A, OH N, MCCLUSKEY E J, et al. Performance evaluation of checksum-based abft[C]//Defect and Fault Tolerance in VLSI Systems, 2001. Proceedings. 2001 IEEE International Symposium on. .[S.l.]: [s.n.] , 2001:461–466.

[55] HUANG K H, ABRAHAM J, et al. Algorithm-based fault tolerance for matrix operations[J]. Computers, IEEE Transactions on, 1984, 100(6):518–528.

[56] BANERJEE P, RAHMEH J T, STUNKEL C, et al. Algorithm-based fault tolerance on a hypercube multiprocessor[J]. Computers, IEEE Transactions on, 1990, 39(9):1132–1145.

[57] JOU J Y, ABRAHAM J, et al. Fault-tolerant FFT networks[J]. Computers, IEEE Transactions on, 1988, 37(5):548–561.

[58] MISHRA A, BANERJEE P. An algorithm-based error detection scheme for the multigrid method[J]. Computers, IEEE Transactions on, 2003, 52(9):1089–1099.

[59] ANDREWS D M. Using executable assertions for testing and fault tolerance[C]//9th Fault-Tolerance Computing Symposium, Madison, Wisconsin, USA. .[S.l.]: [s.n.] , 1979.

[60] MAHMOOD A, MCCLUSKEY E J. Concurrent error detection using watchdog processors-a survey[J]. Computers, IEEE Transactions on, 1988, 37(2):160–174.

[61] RELA M Z, MADEIRA H, SILVA J G. Experimental evaluation of the fail-silent behaviour in programs with consistency checks[C]//Fault Tolerant Computing, 1996., Proceedings of Annual Symposium on. .[S.l.]: [s.n.] , 1996:394–403.

[62] COTRONEO D, ORLANDO S, RUSSO S. Failure classification and analysis of the java virtual machine[C]//Distributed Computing Systems, 2006. ICDCS 2006. 26th IEEE International Conference on. .[S.l.]: [s.n.] , 2006:17–17.

[63] DOMASCHKA J, HAUCK F J, REISER H P, et al. Deterministic multithreading for java-based replicated objects[C]//proceedings of the International Conference on Parallel and Distributed Computing and Systems. .[S.l.]: [s.n.] , 2006.

[64] SILVA J M, SIMAO J, VEIGA L. Ditto–Deterministic Execution Replayability-as-a-Service for Java VM on Multiprocessors[M]//Middleware 2013.[S.l.]: Springer, 2013:405–424.

[65] SØRENSEN M, HANSEN R. TinyVM: A Fault-Tolerant Virtual Machine[J]. 2014.

[66] DE KRUIJF M, NOMURA S, SANKARALINGAM K. Relax: An architectural framework for software recovery of hardware faults[C]//ACM SIGARCH Computer Architecture News. .[S.l.]: [s.n.] , 2010，38:497–508.

[67] YALCIN G, UNSAL O, HUR I, et al. Faultm: fault-tolerance using hardware transactional memory[C]//Pespma 2010-Workshop on Parallel Execution of Sequential Programs on Multi-core Architecture. .[S.l.]: [s.n.] , 2010.

[68] MEIXNER A, BAUER M E, SORIN D J. Argus: Low-cost, comprehensive error detection in simple cores[C]//Microarchitecture, 2007. MICRO 2007. 40th Annual IEEE/ACM International Symposium on. .[S.l.]: [s.n.] , 2007:210–222.

[69] YANG X J, GAO L. Error flow model: Modeling and analysis of software propagating hardware faults.[J]. Ruan Jian Xue Bao(Journal of Software), 2007, 18(4):808–820.

[70] LI S, LI X. Soft error propagation in floating-point programs[C]//Performance Computing and Communications Conference (IPCCC), 2010 IEEE 29th International. .[S.l.]: [s.n.] , 2010:239–246.

[71] BENSO A, CARLO S D, NATALE G D, et al. Static analysis of SEU effects on software applications[C]//Test Conference, 2002. Proceedings. International. .[S.l.]: [s.n.] , 2002:500–508.

[72] SRIDHARAN V, KAELI D R. Quantifying software vulnerability[C]//Proceedings of the 2008 workshop on Radiation effects and fault tolerance in nanometer technologies. .[S.l.]: [s.n.] , 2008:323–328.

[73] XU J, SHEN R, TAN Q. PRASE: an approach for program reliability analysis with soft errors[C]//Dependable Computing, 2008. PRDC'08. 14th IEEE Pacific Rim International Symposium on. .[S.l.]: [s.n.] , 2008:240–247.

[74] HILLER M, JHUMKA A, SURI N. EPIC: Profiling the propagation and effect of data errors in software[J]. Computers, IEEE Transactions on, 2004, 53(5):512–530.

[75] SEGALL Z, VRSALOVIC D, SIEWIOREK D, et al. Fiat-fault injection based automated testing environment[C]//Fault-Tolerant Computing, 1995, Highlights from Twenty-Five Years., Twenty-Fifth International Symposium on. .[S.l.]: [s.n.] , 1995:394.

[76] KANAWATI G A, KANAWATI N A, ABRAHAM J A. FERRARI: A tool for the validation of system dependability properties[C]//Fault-Tolerant Computing, 1992. FTCS-22. Digest of Papers., Twenty-Second International Symposium on. .[S.l.]: [s.n.] , 1992:336–344.

[77] KAO W L, IYER R K, TANG D. FINE: A fault injection and monitoring environment for tracing the UNIX system behavior under faults[J]. Software Engineering, IEEE Transactions on, 1993, 19(11):1105–1118.

[78] CARREIRA J, MADEIRA H, SILVA J G, et al. Xception: Software fault injection and monitoring in processor functional units[J]. Dependable Computing and Fault Tolerant Systems, 1998, 10:245–266.

[79] SUNDARESAN V, MAIER D, RAMARAO P, et al. Experiences with multi-threading and dynamic class loading in a Java just-in-time compiler[C]//Proceedings of the International Symposium on Code Generation and Optimization. .[S.l.]: [s.n.] , 2006:87–97.

[80] TENE G, IYENGAR B, WOLF M. C4: The continuously concurrent compacting collector[J]. ACM SIGPLAN Notices, 2011, 46(11):79–88.

[81] ALPERN B, ATTANASIO C R, COCCHI A, et al. Implementing jalapeño in Java[J]. ACM SIGPLAN Notices, 1999, 34(10):314–324.

[82] ALPERN B, ATTANASIO C R, BARTON J J, et al. The Jalapeno virtual machine[J]. IBM Systems Journal, 2000, 39(1):211–238.

[83] ROGERS I, GROVE D. The strength of metacircular virtual machines: Jikes RVM[J]. Beautiful Architecture: Leading Thinkers Reveal the Hidden Beauty in Software Design, 2009:235.

[84] WIMMER C, HAUPT M, VAN DE VANTER M L, et al. Maxine: An approachable virtual machine for, and in, java[J]. ACM Transactions on Architecture and Code Optimization (TACO), 2013, 9(4):30.

[85] PALACZ K, BAKER J, FLACK C, et al. Engineering a common intermediate representation for the Ovm framework[J]. Science of Computer Programming, 2005, 57(3):357–378.

[86] MCGHAN H, O'CONNOR M. Picojava: A direct execution engine for java bytecode[J]. Computer, 1998, 31(10):22–30.

[87] CONNOR M J, TREMBLAY M. picoJava-I: The Java virtual machine in hardware[J]. Micro, IEEE, 1997, 17(2):45–53.

[88] PUFFITSCH W, SCHOEBERL M. picoJava-II in an FPGA[C]//Proceedings of the 5th international workshop on Java technologies for real-time and embedded systems. .[S.l.]: [s.n.] , 2007:213–221.

[89] HALFHILL T R. Imsys hedges bets on Java[J]. Microprocessor Report, August, 2000.

[90] IMSYS A. the Cjip Technical Reference Manual[J]. Imsys AB, Sweden, v0, 2003, 23.

[91] HARDIN D S. aJile Systems: Low-power direct-execution Java microprocessors for real-time and networked embedded applications[J]. White paper available in http://www. ajile. com/downloads/aJile-white-paper. pdf, 2006.

[92] HARDIN D S. Real-time objects on the bare metal: an efficient hardware realization of the java tm virtual machine[C]//Object-Oriented Real-Time Distributed Computing, 2001. ISORC-2001. Proceedings. Fourth IEEE International Symposium on. .[S.l.]: [s.n.] , 2001:53–59.

[93] BOSE B, TUNA M E, NAGY J M. LavaCORE trade; configurable Java trade; processor core[C]//Aerospace Conference Proceedings, 2002. IEEE. .[S.l.]: [s.n.] , 2002，4:4–1953.

[94] KREUZINGER J, BRINKSCHULTE U, PFEFFER M, et al. Real-time event-handling and scheduling on a multithreaded Java microcontroller[J]. Microprocessors and Microsystems, 2003, 27(1):19–31.

[95] BECK A C S, CAIRO L. Low power java processor for embedded applications[M]//VLSI-SOC: From Systems to Chips.[S.l.]: Springer, 2006:213–228.

[96] SCHOEBERL M. Jop: A java optimized processor for embedded real-time systems[D].[S.l.]: Technical University of DenmarkDanmarks Tekniske Universitet, Department of Informatics and Mathematical ModelingInstitut for Informatik og Matematisk Modellering, 2008.

[97] SCHOEBERL M. A Java processor architecture for embedded real-time systems[J]. Journal of Systems Architecture, 2008, 54(1):265–286.

[98] TAN Y Y, YAU C, LO K, et al. Design and implementation of a java processor[J]. IEE Proceedings-Computers and Digital Techniques, 2006, 153(1):20–30.

[99] HU W W, ZHANG F X, LI Z S. Microarchitecture of the Godson-2 processor[J]. Journal of Computer Science and Technology, 2005, 20(2):243–249.

[100] HU W, WANG J, GAO X, et al. Godson-3: A scalable multicore RISC processor with X86 emulation[J]. IEEE micro, 2009, 29(2):17–29.

[101] GAO X, CHEN Y J, WANG H D, et al. System architecture of Godson-3 multi-core processors[J]. Journal of Computer Science and Technology, 2010, 25(2):181–191.

[102] HU W, WANG R, CHEN Y, et al. Godson-3B: A 1GHz 40W 8-core 128GFlops processor in 65nm CMOS[C]//Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 2011 IEEE International. .[S.l.]: [s.n.] , 2011:76–78.

[103] HU W, ZHANG Y, YANG L, et al. Godson-3B1500: A 32nm 1.35 GHz 40W 172.8 GFLOPS 8-core processor[C]//Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 2013 IEEE International. .[S.l.]: [s.n.] , 2013:54–55.

[104] HU W, YANG L, FAN B, et al. An 8-Core MIPS-Compatible Processor in 32/28 nm Bulk CMOS[J]. Solid-State Circuits, IEEE Journal of, 2014, 49(1):41–49.

[105] CAI S, YANG Y, LIN C, et al. JVM virtual method invoking optimization based on CAM table[C]//Networking, Architecture and Storage (NAS), 2011 6th IEEE International Conference on. .[S.l.]: [s.n.] , 2011:122–129.

[106] QI A, SONGSONG C, JIAN W. JVM JIT Compiler Optimization Based on Cache Locking Mechanism[J]. Journal of Computer Research and Development, 2012:S1.

[107] HOPKINS JR A L, SMITH III T, LALA J H. FTMPłA highly reliable fault-tolerant multiprocess for aircraft[J]. Proceedings of the IEEE, 1978, 66(10):1221–1239.

[108] LYONS R E, VANDERKULK W. The use of triple-modular redundancy to improve computer reliability[J]. IBM Journal of Research and Development, 1962, 6(2):200–209.

[109] OH N, SHIRVANI P P, MCCLUSKEY E J. Control-flow checking by software signatures[J]. Reliability, IEEE Transactions on, 2002, 51(1):111–122.

[110] COTRONEO D, NATELLA R, PIETRANTUONO R, et al. A survey of software aging and rejuvenation studies[J]. ACM Journal on Emerging Technologies in Computing Systems (JETC), 2014, 10(1):8.

[111] HUANG Y, KINTALA C, KOLETTIS N, et al. Software rejuvenation: Analysis, module and applications[C]//Fault-Tolerant Computing, 1995. FTCS-25. Digest of Papers., Twenty-Fifth International Symposium on. .[S.l.]: [s.n.] , 1995:381–390.

[112] KOURAI K, CHIBA S. A fast rejuvenation technique for server consolidation with virtual machines[C]//Dependable Systems and Networks, 2007. DSN'07. 37th Annual IEEE/IFIP International Conference on. .[S.l.]: [s.n.] , 2007:245–255.

[113] COTRONEO D, ORLANDO S, PIETRANTUONO R, et al. A measurement-based ageing analysis of the JVM[J]. Software Testing, Verification and Reliability, 2013, 23(3):199–239.

[114] AO Q, JIN G, SU W, et al. Optimizing Memory Access with Fast Address Computation on a MIPS Architecture[C]//Networking, Architecture, and Storage (NAS), 2014 9th IEEE International Conference on. .[S.l.]: [s.n.] , 2014:143–147.

[115] VENNERS B. Inside the Java virtual machine[M].[S.l.]: McGraw-Hill, Inc., 1996.

[116] LINDHOLM T, YELLIN F. Java virtual machine specification[M].[S.l.]: Addison-Wesley Longman Publishing Co., Inc., 1999.

[117] SHIV K, CHOW K, WANG Y, et al. SPECjvm2008 performance characterization[M]//Computer Performance Evaluation and Benchmarking.[S.l.]: Springer, 2009:17–35.

[118] MEIXNER A, SORIN D J. Error detection using dynamic dataflow verification[C]//Parallel Architecture and Compilation Techniques, 2007. PACT 2007. 16th International Conference on. .[S.l.]: [s.n.] , 2007:104–118.

[119] BENSO A, CHIUSANO S, PRINETTO P, et al. AC/C++ source-to-source compiler for dependable applications[C]//Dependable Systems and Networks, 2000. DSN 2000. Proceedings International Conference on. .[S.l.]: [s.n.] , 2000:71–78.

[120] NEWSOME J, SONG D. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software[J]. 2005.

[121] JEONG J, GUO S, GU Y, et al. TBD: Trajectory-based data forwarding for light-traffic vehicular networks[C]//Distributed Computing Systems, 2009. ICDCS'09. 29th IEEE International Conference on. .[S.l.]: [s.n.] , 2009:231–238.

[122] ZHANG Y, ZHAO J, CAO G. On scheduling vehicle-roadside data access[C]//Proceedings of the fourth ACM international workshop on Vehicular ad hoc networks. .[S.l.]: [s.n.] , 2007:9–18.

[123] YIN H, SONG D, EGELE M, et al. Panorama: capturing system-wide information flow for malware detection and analysis[C]//Proceedings of the 14th ACM conference on Computer and communications security. .[S.l.]: [s.n.] , 2007:116–127.

[124] KANG M G, MCCAMANT S, POOSANKAM P, et al. DTA++: Dynamic Taint Analysis with Targeted Control-Flow Propagation.[C]//NDSS. .[S.l.]: [s.n.] , 2011.

[125] HALDAR V, CHANDRA D, FRANZ M. Dynamic taint propagation for Java[C]//Computer Security Applications Conference, 21st Annual. .[S.l.]: [s.n.] , 2005:9–pp.

[126] ENCK W, GILBERT P, HAN S, et al. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones[J]. ACM Trans. Comput. Syst., 2014, 32(2):5:1–5:29. http://doi.acm.org/10.1145/2619091.

[127] BELL J S, KAISER G E. Phosphor: Illuminating Dynamic Data Flow in the JVM[J]. 2014.

[128] BELL J, KAISER G. Dynamic taint tracking for Java with phosphor[C]//Proceedings of the 2015 International Symposium on Software Testing and Analysis. .[S.l.]: [s.n.] , 2015:409–413.

[129] SUH G E, LEE J W, ZHANG D, et al. Secure program execution via dynamic information flow tracking[C]//Acm Sigplan Notices. .[S.l.]: [s.n.] , 2004，39:85–96.

[130] SON S, MCKINLEY K S, SHMATIKOV V. Diglossia: detecting code injection attacks with precision and efficiency[C]//Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security. .[S.l.]: [s.n.] , 2013:1181–1192.

[131] HALFOND W G, ORSO A, MANOLIOS P. Using positive tainting and syntax-aware evaluation to counter SQL injection attacks[C]//Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering. .[S.l.]: [s.n.] , 2006:175–185.

[132] AZADMANESH M R, SHARIFI M. Towards a system-wide and transparent security mechanism using language-level information flow control[C]//Proceedings of the 3rd international conference on Security of information and networks. .[S.l.]: [s.n.] , 2010:19–26.

[133] CHANG W, STREIFF B, LIN C. Efficient and extensible security enforcement using dynamic data flow analysis[C]//Proceedings of the 15th ACM conference on Computer and communications security. .[S.l.]: [s.n.] , 2008:39–50.

[134] MIGLIAVACCA M, PAPAGIANNIS I, EYERS D M, et al. DEFCON: High-Performance Event Processing with Information Security.[C]//USENIX Annual Technical Conference. .[S.l.]: [s.n.] , 2010.

[135] ROY I, PORTER D E, BOND M D, et al. Laminar: practical fine-grained decentralized information flow control[M]. Vol. 44.[S.l.]: ACM, 2009.

[136] LEEK T R, BAKER G Z, BROWN R E, et al. Coverage maximization using dynamic taint tracing[R].[S.l.]: DTIC Document, 2007.

[137] GANAI M, LEE D, GUPTA A. DTAM: dynamic taint analysis of multi-threaded programs for relevancy[C]//Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering. .[S.l.]: [s.n.] , 2012:46.

[138] SWEETMAN D. See MIPS run[M].[S.l.]: Morgan Kaufmann, 2010.

[139] VENSTERMANS K, EECKHOUT L, DE BOSSCHERE K. 64-bit versus 32-bit Virtual Machines for Java[J]. Software: Practice and Experience, 2006, 36(1):1–26.

[140] CHOW F, CORRELL S, HIMELSTEIN M, et al. How many addressing modes are enough?[C]//ACM SIGARCH Computer Architecture News. .[S.l.]: [s.n.] , 1987，15:117–121.

[141] SRIVASTAVA A, WALL D W. Link-time optimization of address calculation on a 64-bit architecture[M]. Vol. 29.[S.l.]: ACM, 1994.

[142] STEVEN G B. A novel effective address calculation mechanism for RISC microprocessors[J]. ACM SIGARCH Computer Architecture News, 1988, 16(4):150–156.

[143] AUSTIN T M, PNEVMATIKATOS D N, SOHI G S. Streamlining data cache access with fast address calculation[J]. ACM SIGARCH Computer Architecture News, 1995, 23(2):369–380.

[144] BLACKBURN S M, GARNER R, HOFFMANN C, et al. The DaCapo benchmarks: Java benchmarking development and analysis[C]//ACM Sigplan Notices. .[S.l.]: [s.n.] , 2006，41:169–190.

致　谢

感谢我的导师胡伟武研究员。感谢胡老师言传身教，教我做人、做事、做学问。感谢胡老师为龙芯做的一切，为我们攻读博士学位营造了优越的环境、一流的平台、宽松的氛围，为我们的生活排除后顾之忧。在这里我们肩负使命，为项目做出实际贡献，在这里我们可以自由发挥，追求并实现自己的想法。在这里既锻炼了我的动手能力，也提高了我的科研水平。胡老师渊博的学识、积极进取的精神、踏实严谨的作风、饱满的激情、崇高的理想，无不让学生敬佩，是学生一生学习的榜样。感谢胡老师为中国计算机事业做出的贡献!

感谢我的导师张福新老师。张老师的学识与为人让人钦佩。从本科毕设开始，张老师就对我进行了孜孜不倦的教诲。在我选题迷茫的时候，张老师跟我长谈，给我指点迷津。在我懒惰的时候，张老师的批评和以身作则，及时让我清醒继续前行。在我努力工作时，张老师的表扬和鼓励，让我如浴春风、加倍努力。

感谢章隆兵老师和王剑老师。在龙芯实验室的两年时间里，章老师在科研上不断地给我指导与帮助，使我收益颇丰。感谢王剑老师对我的帮助，王老师总是那么和蔼可亲，给予学生全力的支持和帮助。

感谢靳国杰，他在工作和科研方面均给了我莫大的帮助。他的专业学识、他的踏实肯干、他的昂扬斗志都是我学习的榜样。

感谢蔡嵩松，他既是我的师兄也是我的朋友，感谢他对我论文上的大力帮助。感谢我们在Java组的快乐时光。

感谢高翔、张戈、刘奇、张珩、肖俊华、胡明昌、章立生、吴少刚、晏华，他们或为我的老师、或为我曾经的领导，给我的工作和学习很多帮助。

感谢刘凤琴老师、杨杰老师、冯钢老师、李丹老师以及研究生部的所有老师给我的帮助，让我能够将更多的精力投入到科研和学习当中。

感谢我的挚友曲政、郭翼飞、高然，在读博期间，感谢他们的陪伴与理解，感谢与他们欢度的日子。有了他们，在北京的生活少了许多辛苦，多了许多快乐。

最后特别要感谢我的家人。感谢我的父母，是他们养育了我、小心翼翼地呵护着我、无条件地支持着我。他们为我付出了最多，却也承受着我的不成熟。在我不如意、不顺利的时候，可能会对他们肆无忌惮地表露，他们总是默默承受并给我鼓励与支持。从今开始，我终于踏入社会，希望能以我加倍的努力拼搏来报答父母的无法报答的恩情。感谢兄嫂、感谢戈俊红女士，感谢他们无私的奉献和帮助，让我在北京有亲情，有温暖。

谨以此文献给所有关心、帮助和支持我的人。

纪念导师夏培肃先生!

作者简介

姓名：敖琪　　性别：男　　出生日期：1986.4.10　　籍贯：辽宁朝阳

2008.9 – 现在　　中国科学院计算技术所攻读博士研究生

2004.9 – 2008.7　　北京交通大学本科生

【攻读博士学位期间发表的论文和专利】

[1] Qi Ao, Longbing Zhang, Shuai Chen, Jie Fu, Bin Liao, “A Fault-Tolerant Java Virtual Machine Using Fast Rejuvenation for Soft-Error-Prone Systems,” The 17th IEEE International Conference on High Performance Computing and Communications (HPCC), 2015.

[2] Qi Ao, Guojie Jin, Wen Su, Songsong Cai, “Optimizing Memory Access with Fast Address Computation on a MIPS Architecture,” The 9th IEEE International Conference on Networking, Architecture, Storage (NAS), 2014.

[3] Shuai Chen, Qi Ao, Hua Jing, Xiaobing Shi, Shiguang Meng, Xinke Chen, “An 8Gb/s 0.75 mW/Gb/s injection-locked receiver with constant jitter tracking bandwidth and accurate quadrature clock generation in 40nm CMOS,” The 21st IEEE International Conference on Electronics, Circuits and Systems (ICECS), 2014.

[4] 敖琪，蔡嵩松，王剑，“基于硬件cache锁机制的Java虚拟机即时编译器优化”，计算机研究与发展，2012 (S1): 185-190.

[5] 敖琪，靳国杰，高翔，“64位虚拟机处理访存指令的方法及装置”，专利号：2014102168280

[6] 敖琪，“Java虚拟机中的软件再生方法及装置”，在审理

【攻读博士学位期间的获奖情况】

[1] 2009年被评为中国科学院“三好学生”

[2] 2010年被评为中国科学院“三好学生”

[3] 2011年被评为中国科学院“三好学生”

[4] 2012年被评为中国科学院“三好学生”